COMPANION GUIDE 3 - EXECUTABLE EDITION

# Mitophagy-Lysosome Coupling in Aging Microglia

A tested, cell-by-cell pipeline for a multi-cohort, animal-level, human-replicated and genetically anchored study - with expected outputs, expected figures and specific troubleshooting for every step

| Item | Detail |
|---|---|
| Prepared for | Hashmi |
| Replaces | Guide 2 (whole-notebook blocks). Same toolchain: Anaconda, Jupyter, Scanpy. |
| What is new | Every notebook split into cells; each cell followed by its expected output (real, from a synthetic benchmark with a known planted truth), expected figure and a problem/fix table; 14 ready-to-run .ipynb notebooks; a failure-mode gallery; a tested helper library. |
| Bundle | microglia-coupling.zip (delivered with this PDF): notebooks, src, environment.yml, licence, templates. |
| Needs | 16 GB RAM (32 GB for the human cohort), about 30 GB disk, internet for downloads. No paid accounts. |

# Read this first

## Was Guide 2 just copy-paste with basic troubleshooting?

Largely, yes. It gave correct designs and scripts, but in whole-notebook blocks, with a short generic troubleshooting table, no example outputs and no example figures. You could not tell whether a run was going well. This guide fixes that.

## What changed

- **Cell by cell.** Every notebook is split into cells with a one-paragraph purpose, the code, and what to expect.
- **Expected outputs that are real.** Instead of invented numbers, the pipeline was built and run end to end on a SYNTHETIC benchmark with a known planted effect. The green boxes are the actual outputs of that run, so what you see is what correct code produces.
- **Expected figures.** Every figure shown was produced by the code in the same cell.
- **"How to read it" boxes** on the key cells interpret the numbers (what is good, what would be a warning).
- **Specific troubleshooting** per cell: real error messages, causes and fixes, including bugs that were found and fixed while testing.
- **Failure-mode gallery** (Notebook 13): what a true effect, no effect, a depth artefact and a composition artefact look like with the same code.
- **Ready-to-run notebooks.** The bundle contains all code as .ipynb files; you do not copy code from the PDF.

## Reading the boxes

| Box | Meaning |
|---|---|
| Green: EXPECTED OUTPUT | Real output of the exact code above it, run on the synthetic benchmark (planted truth known). Your numbers should be close in the smoke test. |
| Orange: FORMAT EXAMPLE | Schematic. The step needs real data, the internet or an external program (GEO, MAGMA, STRING, scVI, ...) and could not be run here. Shows the layout of the output only; the numbers are invented. |
| Blue/green: How to read it | What the numbers mean and which pattern would be a warning. |
| Red table: If you see... | Specific problems and their fixes. |

**Honest limits**

The benchmark proves the code runs and recovers planted effects; it says nothing about real biology, and real effects are usually smaller and noisier. Several steps (downloads, MAGMA, STRING, scVI, CollecTRI) could not be executed in the test environment: they are marked and described, not simulated. Accession numbers marked [verify] must be checked on the source page. No guide can guarantee acceptance in a top journal; it can remove the weaknesses reviewers reject for (pseudoreplication, circular gene lists, no null model, no replication, no human or genetic anchor) and show you how to recognise artefacts.

# Contents


Was Guide 2 just copy-paste with basic troubleshooting? . . . 2

What changed . . . 2

Reading the boxes . . . 2

**Part 1. Strategy: the paper you are now building 5**

1.1 What was wrong with the first design, and what replaces it . . . 5

1.2 The central question and the hypotheses . . . 5

1.3 Decision gates (decide the rules before you look) . . . 6

1.4 Statistical power: why the numbers of animals matter . . . 6

1.5 Evidence ladder and realistic targets . . . 6

1.6 Novelty check before you invest (2 hours) . . . 6

1.7 Timeline . . . 7

**Part 2. Setup: bundle, environment, smoke test, GitHub, preregistration 8**

2.1 Install the tools . . . 8

2.2 Create the environment . . . 8

2.3 Smoke test on the synthetic benchmark (do this first, 20-30 minutes) . . . 9

2.4 Switching to real data . . . 9

2.5 GitHub from day one . . . 9

2.6 Preregister on OSF . . . 10

**Part 3. Datasets: what to download, from where, and why 11**

3.1 Download helper for GEO supplementary files . . . 11

3.2 Sample metadata from GEO (GEOparse) . . . 12

3.3 Mouse cohorts . . . 12

3.4 Human cohort: SEA-AD . . . 12

3.5 Perturbation, screen and proteomic data . . . 13

3.6 GWAS summary statistics . . . 13

**Part 4. The notebooks, cell by cell 14**

Notebook 00 - Synthetic benchmark (smoke test with known ground truth) . . . 14

Notebook 01 - Load real cohorts and attach metadata (skip when running the synthetic smoke test) . . . 17

Notebook 02 - Quality control, doublets, clustering, microglia selection . . . 21

Notebook 03 - Shared state labels across cohorts . . . 27

Notebook 04 - Gene programs and expression-matched null sets . . . 30

Notebook 05 - Primary result: animal-level coupling and the matched null . . . 34

Notebook 06 - Specificity and sensitivity analyses . . . 39

Notebook 07 - State-resolved analysis: composition artefact or intrinsic change? . . . 44

Notebook 08 - Regulators and drivers (replaces the circular "bridge genes") . . . 47

Notebook 09 - Human replication with donor-level analysis (SEA-AD) . . . 53

Notebook 10 - Genetic anchoring with MAGMA (AD and PD) . . . 56

Notebook 11 - Perturbation, CRISPR-screen and proteome concordance . . . 59

Notebook 12 - Evidence integration and network . . . 63

Notebook 13 - Failure-mode gallery: what artefacts look like . . . 67

**Part 5. Reading your results: real effect or artefact? 71**

5.1 Decision table . . . 71

**Part 6. Optional but decisive: a minimal wet-lab validation module 73**

**Part 7. Figures that look like a top-journal paper 74**

**Part 8. Releasing everything publicly on GitHub (and getting a DOI) 75**

8.1 Repository contents . . . 75

8.2 Zenodo DOI and release . . . 75

8.3 Preprint . . . 75

**Part 9. Writing the paper and choosing the journal 77**

9.1 Titles to use (pick by what your data finally show) . . . 77

9.2 Angle that editors respond to . . . 77

9.3 Manuscript skeleton (about 5,000-6,500 words plus Methods) . . . 77

9.4 Journal strategy . . . 78

9.5 Cover letter: five sentences that matter . . . 78

9.6 Reviewer questions to answer in advance . . . 78

**Part 10. Troubleshooting and what to do with each outcome 79**

What each possible outcome becomes . . . 79

**Appendix A. src/mgcoupling/utils.py (complete, tested listing) 80**

**Appendix B. src/mgcoupling/sim.py (synthetic generator) 84**

**Appendix C. environment.yml and repository files 86**

**Appendix D. Test report: what was actually run 87**

**Appendix E. Templates and final checklist 88**

A. README.md template . . . 88

B. CITATION.cff template . . . 88

C. Final pre-submission checklist . . . 88

# Part 1. Strategy: the paper you are now building

## 1.1 What was wrong with the first design, and what replaces it

| Weakness in Guide 1 | Why reviewers reject it | Fix in this guide (and where) |
|---|---|---|
| One pooled sample per age; age = batch | Any age effect could be a library effect | 3+ independent mouse cohorts with several animals per age, plus human donors |
| Cell-level p-values | Thousands of cells from one animal are not independent (pseudoreplication) | Animal/donor is the unit; mixed models, permutation, random-effects meta-analysis |
| Hand-picked, overlapping gene lists | Circular; mitophagy and lysosome lists share genes with DAM and inflammation | Database-derived modules, overlap-excluded, DAM taken from an independent dataset |
| No null model | Any two gene sets correlate to some degree | Expression-matched random gene-set null (200+ draws) |
| "Bridge genes" ranked from the same lists | Circular; STRING enrichment guaranteed | Regulators from TF activity, replicated pseudobulk DE, GWAS and perturbation evidence in a transparent rubric |
| One weak "validation" | Rank correlation with a different system proves little | Independent cohorts, human replication, preranked GSEA on perturbations, CRISPR-screen overlap |
| No human or genetic link | Mouse-only, descriptive | SEA-AD donors; MAGMA on AD and PD GWAS |
| Not reproducible | Cannot be audited | Preregistration, pinned environment, GitHub + Zenodo DOI |

## 1.2 The central question and the hypotheses

**Question.** In microglia, are the transcriptional programs of PINK1/PARKIN-type mitophagy and of lysosomal biogenesis/degradation co-regulated in the young brain, and does this coupling change (weaken, invert, or compensate) with age and in Alzheimer-type disease states?

| ID | Hypothesis | Evidence that tests it |
|---|---|---|
| H1 (primary) | Within-animal coupling between the mitophagy and lysosome programs differs between young and old microglia. | Animal-level partial Spearman, meta-analysed across mouse cohorts, versus matched random null |
| H2 | The change is reproducible in independent cohorts and in human microglia (donor-level, age and Braak stage). | Per-cohort effects, I-squared, SEA-AD analysis |
| H3 | The change tracks activity of TFEB/TFE3/MITF (CLEAR network) and/or inflammatory (DAM, interferon) state. | decoupler TF activity, mixed models |
| H4 | The programs are enriched for AD/PD genetic risk, and Mendelian PD genes occupy the coupled nodes. | MAGMA competitive gene-set analysis, locus overlay |
| H5 | Perturbing mitochondria or lysosomes in microglia shifts the programs in the predicted direction; screen hits overlap the programs. | Preranked GSEA on perturbation datasets, hypergeometric tests on CRISPR hits |

**Two-sided on purpose**

H1 is preregistered two-sided. Coupling could weaken (failure of quality control) or strengthen (compensation). Either is publishable if it is robust. State your directional prediction, but let the data decide.

## 1.3 Decision gates (decide the rules before you look)

| Gate | Test | Pass | If it fails |
|---|---|---|---|
| G1 Data quality | At least 3 mouse cohorts, each with >= 3 animals per age group (young vs old) | Met | Drop cohort or add one |
| G2 Specificity | Observed old-minus-young change in z(rho) lies outside the 95% range of the matched random null | Empirical p < 0.05 | Report as non-specific; pivot to regulators/state analysis |
| G3 Consistency | Direction matches in >= 75% of cohorts; meta-analytic 95% CI excludes 0 | Met | Report heterogeneity; analyse by region/sex/state |
| G4 Human | SEA-AD donor-level association (age or Braak) in the same direction | Same sign, p < 0.05 | Present as mouse-specific; say so |
| G5 Mechanism | TFEB/TFE3/MITF or interferon activity associates with coupling | FDR < 0.05 | Drop mechanism claims; keep descriptive |

A negative gate is not a failed project. A well-controlled "coupling does not change specifically with age" is a legitimate, citable result and prevents you from publishing an artefact.

## 1.4 Statistical power: why the numbers of animals matter

Animal is the unit. With 3 young vs 3 old animals there are only C(6,3) = 20 label permutations, so the smallest possible two-sided exact p is 0.10 - you cannot reach p < 0.05 inside one such cohort. With 4 vs 4 the minimum is 2/70 = 0.029. This is why the design leans on (a) random-effects meta-analysis across cohorts, (b) mixed models that use all cells while modelling animal, and (c) human donors (SEA-AD has dozens). Always report effect sizes with confidence intervals, not just p-values.

## 1.5 Evidence ladder and realistic targets

| Evidence you have | What it supports | Realistic journal tier |
|---|---|---|
| Computational only, multi-cohort, human + genetic | Solid hypothesis-generating study | Aging Cell, GeroScience, Molecular Neurodegeneration (computational track), Genome Medicine, Neurobiology of Disease, Journal of Neuroinflammation, Acta Neuropathologica Communications |
| + in-silico perturbation and screen concordance | Mechanistic prioritisation | Upper end of the above; Nature Communications is a stretch |
| + small wet-lab validation (Part 6) | Functional support in vivo or in primary cells | Nature Communications, Cell Reports, EMBO Journal/EMBO Molecular Medicine, Nature Aging (competitive) |
| + perturbation experiments with causal rescue | Mechanism | Neuron, Nature Neuroscience, Cell (needs a lab team) |

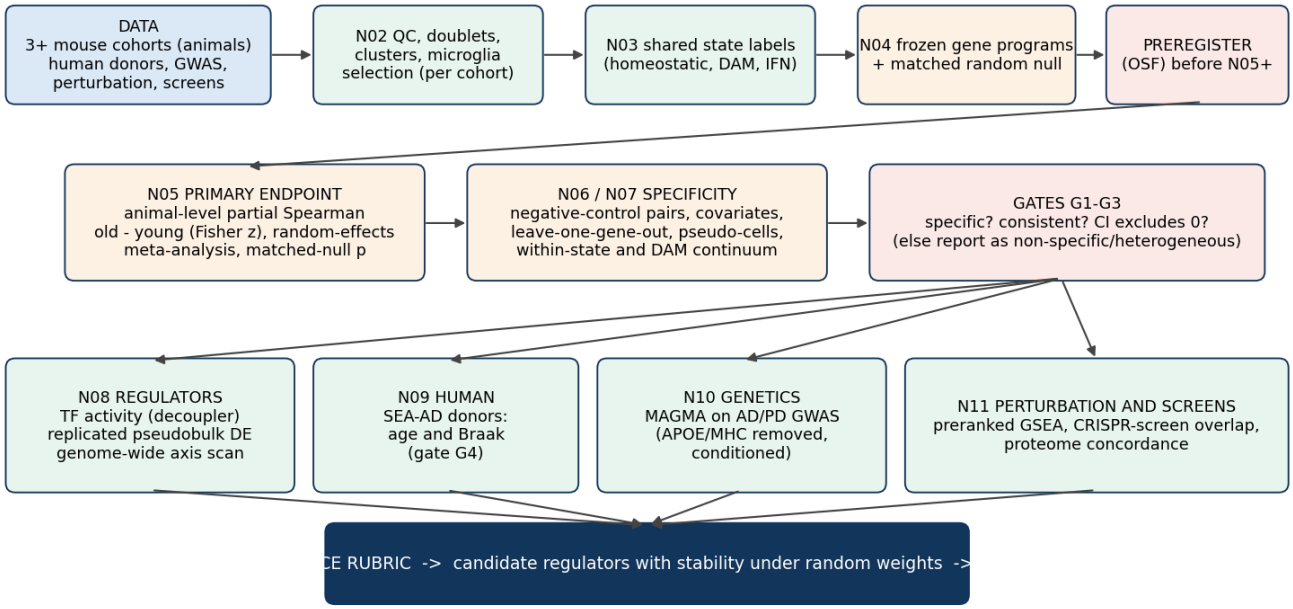
## 1.6 Novelty check before you invest (2 hours)

1. Open https://pubmed.ncbi.nlm.nih.gov and run each query, sorted by Best match and by Most recent: `microglia AND mitophagy AND lysosom* AND (aging OR ageing)` ; `microglia AND "single-cell" AND mitophagy` ; `TFEB AND microglia AND aging` ; `PINK1 AND microglia AND lysosom*`.
2. Open https://www.biorxiv.org and https://www.medrxiv.org and search the same terms for the last 12 months (scoop check).
3. In a spreadsheet record for each close paper: design, species, sample size, what it claims about coupling. Your novelty statement must be a sentence of the form "Prior work showed X in Y; we test Z, which has not been tested with animal-level replication across cohorts."

4 If a paper already shows the same result, reposition: coupling as a quantitative framework, human replication, genetic anchoring, or regulator identification.

## 1.7 Timeline

| Weeks | Work | Output |
|---|---|---|
| 1 | Setup, GitHub, preregistration, novelty check | Repo, OSF registration |
| 2-3 | Download and harmonise all datasets (Notebooks 01-03) | Clean microglia AnnData per cohort |
| 4 | Gene programs and null sets (04) | modules.csv, null sets |
| 5-6 | Coupling statistics and sensitivity (05-06) | Figs 2-3 |
| 7 | States, regulators, replicated DE (07-08) | Figs 4-5 |
| 8 | Human SEA-AD (09) | Fig 6 |
| 9 | MAGMA genetics (10) | Fig 7a |
| 10 | Perturbation, screens, evidence rubric, network (11-12) | Fig 7b-c |
| 11-14 | Writing, figures, GitHub release, preprint; optional wet-lab runs in parallel | Manuscript, DOI |




***Pipeline overview.*** *Orange = decision steps and the primary endpoint; green = data processing and orthogonal evidence; red = preregistration and decision gates.*

# Part 2. Setup: bundle, environment, smoke test, GitHub, preregistration

## 2.1 Install the tools

1. Install Anaconda (https://www.anaconda.com/download) with default options, as in Guide 1.
2. Install Git (https://git-scm.com/downloads) and create a GitHub account (https://github.com/signup) and an ORCID iD (https://orcid.org/register).
3. Unzip the project bundle delivered with this guide (`microglia-coupling.zip`). It contains the folder structure below, the helper library, the simulator, all 14 notebooks as .ipynb files, `environment.yml`, `.gitignore`, a licence, `CITATION.cff` and doc templates. You never need to create folders by hand.

| Path in the bundle | Content |
|---|---|
| notebooks/00_synthetic_benchmark.ipynb ... 13_failure_modes.ipynb | All code from Part 4, one notebook per section, run in numerical order |
| src/mgcoupling/utils.py | Helper library used by every notebook (Appendix A) |
| src/mgcoupling/sim.py | Synthetic generator for Notebooks 00 and 13 (Appendix B) |
| environment.yml, .gitignore, LICENSE, CITATION.cff, README.md | Environment, repository files |
| docs/ | data_sources.md and preregistration_template.md |
| data/, results/, gene_sets/ | Empty folders that the notebooks fill |

## 2.2 Create the environment

Open Anaconda Prompt (Windows) or Terminal. Move into the unzipped folder, then:

```
conda env create -f environment.yml
conda activate mgcoupling
python -m ipykernel install --user --name mgcoupling --display-name "Python (mgcoupling)"
jupyter lab
```

In Jupyter open any notebook and choose Kernel > Change Kernel > Python (mgcoupling). The environment file (Appendix C) was tested with the versions listed in Appendix D. At the very end of the project freeze what worked: `conda env export --from-history > environment_final.yml` and `pip freeze > requirements_final.txt`.

| If you see... | What it means and what to do |
|---|---|
| `CondaSolveError` or hanging solver | Use mamba (`conda install -n base -c conda-forge mamba`, then `mamba env create -f environment.yml`) or create the environment in pieces: `conda create -n mgcoupling python=3.12`, then `pip install` the pip block of environment.yml. |
| `ImportError: Numba needs NumPy 2.0 or less` or NumPy/pandas conflicts | A package pinned an older NumPy. Run `pip install -U numba` inside the environment. If `decoupler` downgrades pandas, install it last, or use a separate environment for Notebook 08 Cells 1-2. |
| `brace expansion` or `mkdir -p {a,b}` does nothing | Plain `sh` does not support braces. This guide never relies on them; if you create folders by hand use one `mkdir` per folder. |
| Jupyter does not list the kernel | Re-run the `ipykernel install` line inside the activated environment and restart Jupyter. |

## 2.3 Smoke test on the synthetic benchmark (do this first, 20-30 minutes)

Run Notebook 00, then Notebooks 02, 03, 04, 05, 06, 07, 08, 09, 11, 12 (and optionally 13). Skip 01 and 10 (they need real data and the MAGMA binary). Every number you see should match the green boxes of Part 4 closely (small differences from random seeds and library versions are normal). This proves that your installation works and shows what a correct run looks like. The table below lists what the pipeline recovered from the planted truth in our test:

| Planted in the simulation | Recovered by the pipeline (this guide's run) | Where |
|---|---|---|
| Within-animal coupling drops from about 0.5 to 0.2 in old animals (all cohorts) | Pooled change -0.124 (95% CI -0.186 to -0.063), same sign in 3 of 3 cohorts | N05 Cell 3 |
| The change is specific to MITOPHAGY-LYSOSOME | Matched-null p = 0.005 (minimum possible with 200 sets); controls within -0.01 to +0.02 | N05 Cell 3, N06 Cell 2 |
| Effect not due to composition | Persists in homeostatic (-0.132) and DAM (-0.129) cells | N07 Cell 2 |
| Four regulator genes track the coupling axis in young animals | Found at ranks 1-4 of 2,948 genes in the genome-wide scan | N08 Cell 5 |
| Coupling declines with Braak stage in donors | -0.031 per Braak stage (CI -0.042 to -0.019); control pair flat | N09 Cell 4 |
| Lysosome programs respond to CCCP, are screen hits and rise in the proteome | z = 6.1, hypergeometric p = 2.6e-08, z = 2.6 | N11 |
| Contaminants (BAM, neurons) and damaged cells exist | Two contaminant clusters dropped; DAM cluster correctly kept; 71% of damaged cells flagged | N02 |

**What the benchmark is, and is not**

It is a TEST HARNESS: a simulator with a known truth, used to verify that the code runs and recovers planted effects. It is NOT evidence about biology, and its effect sizes are deliberately cleaner than real data. Real effects are usually smaller, noisier and more heterogeneous. Never quote any number from the benchmark in a paper.

## 2.4 Switching to real data

1. Delete the synthetic files so nothing is mixed: in the project folder run `rm -rf data/processed/* data/raw/seaad data/raw/screens data/raw/proteomics results/tables/* results/figures/* docs/cluster_decisions_*` (on Windows delete those folders' contents in Explorer). Also delete `data/metadata/animals.csv` and `data/metadata/dam_signature.csv`, `data/raw/GSE179611_*` and `data/processed/mouse_human_orthologs.csv`.
2. Follow Part 3 to obtain each real dataset, fill `data/metadata/animals.csv`, then run Notebook 01 instead of 00. Everything from Notebook 02 on is unchanged except the points flagged "EDIT" and the cohort names.
3. Cells marked "SIMULATION ONLY" print comparisons with the planted truth; they do nothing on real data (the `if` guard skips them).

## 2.5 GitHub from day one

1. On GitHub click New repository, name it `microglia-mitophagy-lysosome-coupling`, public, with no README/licence (the bundle has them).
2. In the project folder: `git init`, then `git branch -M main`, then `git add .`, then `git commit -m "Initial project from bundle"`.
3. Connect and push: `git remote add origin https://github.com/YOUR_USERNAME/microglia-mitophagy-lysosome-coupling.git`, then `git push -u origin main`. When asked for a password use a Personal Access Token (GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic) > scope repo).

4. After each finished notebook: `git add . && git commit -m "Notebook 05 done" && git push`. Small commits are your audit trail.

## 2.6 Preregister on OSF

1. Go to https://osf.io, create a project, then Registrations > New registration > OSF Preregistration.
2. Fill hypotheses H1-H5, the age-group rule, QC rules, the primary endpoint (meta-analytic old-minus-young change in Fisher z of animal-level partial Spearman between MITOPHAGY and LYSOSOME), the matched-null criterion, the decision gates (Section 1.3), and upload `gene_sets/modules.csv` and `results/tables/11_predictions.csv`.
3. Register BEFORE running Notebooks 05-12 on the replication cohorts. You may explore one discovery cohort first; say which. Put the registration DOI in the README and manuscript.

# Part 3. Datasets: what to download, from where, and why

Rule of thumb for a top journal: every claim must be supported by at least one dataset that was **not** used to generate it, and the unit of replication must be the animal or donor. The table lists what you will use.

| Code | Dataset (accession) | Role | Unit of replication | Access |
|---|---|---|---|---|
| D1 | GSE179358: hippocampal microglia, 6/12/18/24 mo (your Guide 1 data) | Exploratory only; shown as supplement | One pooled library per age - NOT replicate-level | GEO, free |
| D2 | GSE121654 [verify]: Hammond et al. 2019, microglia across lifespan and injury | Mouse cohort 1 (young adult vs 18 mo) | Individual mice | GEO, free |
| D3 | GSE129788 [verify]: Ximerakis et al. 2019, young vs old whole-brain scRNA-seq | Mouse cohort 2 | Pooled samples (check sample table) | GEO, free |
| D4 | Tabula Muris Senis, brain myeloid [verify]; CELLxGENE collection, GEO GSE132042 [verify] | Mouse cohort 3 (3 vs 18/21 mo) | Individual mice | CELLxGENE, free |
| D5 | GSE98969 [verify] (Keren-Shaul 2017, 5xFAD DAM) and GSE140511 [verify] (Zhou 2020 snRNA TREM2) | Independent DAM signature; disease-state check | Mice | GEO, free |
| D6 | SEA-AD middle temporal gyrus snRNA-seq (Allen Institute), microglia | Human replication (donor level) | Donors | CELLxGENE / Allen portal, free |
| D7 | AD GWAS (Bellenguez 2022; GWAS Catalog GCST90027158 [verify]) and PD GWAS (Nalls 2019 [verify]) | Genetic anchoring | Summary statistics | GWAS Catalog / IPDGC, free |
| D8 | GSE179611: CCCP +/- LPS microglia bulk RNA-seq (Guide 1) plus others found by search (Section 3.5) | Perturbation concordance | Samples per condition | GEO, free |
| D9 | Published CRISPR screens in microglia (e.g. Drager et al. 2022 Nat Neurosci [verify]) and aging microglia proteomics (PRIDE) | Functional and protein-level support | Screen hits; proteomes | Paper supplements, PRIDE |

**How to verify an accession in 2 minutes**

Open https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSExxxxxx . Check: (1) the title and organism, (2) the "Samples" table - how many samples per age, are mice pooled, (3) the supplementary files list. Write the answers into docs/data_sources.md. If the number of animals per age is below 3, move that cohort to the supplement.

## 3.1 Download helper for GEO supplementary files

Add this function to a notebook cell (or to utils.py). It builds the GEO FTP-over-https path and downloads everything, or only files matching a regular expression.

```
import re, os, requests

def geo_suppl_url(gse):
    return 'https://ftp.ncbi.nlm.nih.gov/geo/series/' + gse[:-3] + 'nnn/' + gse + '/suppl/'

def download_geo_suppl(gse, outdir, pattern=None):
    os.makedirs(outdir, exist_ok=True)
    url = geo_suppl_url(gse)
    html = requests.get(url, timeout=60).text
    for f in re.findall(r'href="([^"?/][^"]*)"', html):
        if pattern and not re.search(pattern, f):
            continue
        dest = os.path.join(outdir, f)
        if os.path.exists(dest):
            continue
        print('downloading', f)
        with requests.get(url + f, stream=True, timeout=600) as r:
            r.raise_for_status()
            with open(dest, 'wb') as fh:
                for chunk in r.iter_content(1 << 20):
                    fh.write(chunk)

download_geo_suppl('GSE121654', '../data/raw/hammond')   # example; check size on the GEO page first
```

## 3.2 Sample metadata from GEO (GEOparse)

```
import GEOparse, pandas as pd
gse = GEOparse.get_GEO(geo='GSE121654', destdir='../data/raw/geo_meta', silent=True)
ph = gse.phenotype_data            # one row per GSM; columns hold title, age, sex, strain ...
pd.set_option('display.max_colwidth', 120)
print(ph.columns.tolist())
ph.head(20)
```

Read the printed columns, then copy sample ID, age, sex, tissue, genotype into `data/metadata/animals.csv` by hand (for 10-60 samples this is faster and safer than automating). Assign `age_group` with the preregistered rule. Write one line per cohort in docs/data_sources.md about how animals map to samples.

## 3.3 Mouse cohorts

1. **D1 (Guide 1 data).** Reuse the GSE179358 files from Guide 1. Treat as exploratory.
2. **D2 Hammond.** Open the GEO page, download the supplementary archive(s) with `download_geo_suppl`. Inspect files with `zcat file.txt.gz | head -c 600` (Mac/Linux/Git Bash) to see if the matrix is genes x cells or cells x genes. Fill animals.csv (age: P100 -> 3.3 months; P540 -> 17.8 months; keep only WT, uninjured).
3. **D3 Ximerakis.** Download the supplementary files from GEO. Use the authors' cell-type annotation file if provided to select microglia, but still re-check markers (Notebook 02). Each GSM may pool mice: record `pooled_n`.
4. **D4 Tabula Muris Senis.** Go to https://cellxgene.cziscience.com/collections, search "Tabula Muris Senis", open the brain dataset (droplet/10x), and download the .h5ad via the download icon. If it offers a myeloid subset, take it; otherwise load the whole file and subset to microglia/brain myeloid using its cell-type annotation column.
5. **D5 AD-model mouse data.** Download GSE98969 and GSE140511 supplementary files. You only need them to define an independent DAM gene list and to check the disease-state direction; they are not part of the age meta-analysis.

## 3.4 Human cohort: SEA-AD

1. Go to https://cellxgene.cziscience.com/collections and search "Seattle Alzheimer's Disease Brain Cell Atlas" (SEA-AD). Open the middle temporal gyrus (MTG) dataset collection.
2. Prefer the **Microglia-PVM** subset file if listed (small); otherwise download the full MTG .h5ad (large, needs about 32 GB RAM; then load with `backed="r"` and subset by cell type before loading).

3. Also open https://portal.brain-map.org/explore/seattle-alzheimers-disease and download the donor metadata table (age at death, sex, Braak stage, CERAD, continuous pseudo-progression score) if it is not already in the .h5ad `obs`.
4. Save as `data/raw/seaad/seaad_mtg_microglia.h5ad`. Nuclei (snRNA-seq) capture fewer mitochondrial and lysosomal transcripts than whole cells; this is a limitation you will state.

## 3.5 Perturbation, screen and proteomic data

1. GEO search 1: go to https://www.ncbi.nlm.nih.gov/gds and search `microglia AND (mitophagy OR CCCP OR "Pink1" OR "Parkin" OR "TFEB" OR "bafilomycin" OR "chloroquine") AND "Mus musculus"[orgn]`. Filter: Study type = Expression profiling by high throughput sequencing. For each hit record: system (primary microglia, BV2, iPSC-microglia), perturbation, n per group, and whether a differential table or counts are provided.
2. GEO search 2: `microglia AND (Trem2 OR Tfeb OR Mitf) AND knockout`. Keep only datasets with >= 3 replicates per group.
3. Keep 2-4 best perturbation datasets (plus GSE179611). Download counts; if only raw counts are given you will run DESeq2-style analysis in Notebook 11.
4. CRISPR screens: PubMed `CRISPR screen microglia iPSC` ; download the supplementary tables with gene-level hits (phenotype, p-value/FDR). Save as CSV in `data/raw/screens/` with columns `gene,score,fdr`.
5. Proteomics: https://www.ebi.ac.uk/pride/ search `microglia aging proteome`; or take supplementary tables from aging brain/microglia proteomic papers. Save gene-level log fold change (old vs young) and p-value.

## 3.6 GWAS summary statistics

1. Open https://www.ebi.ac.uk/gwas/ and search the study titles (AD: Bellenguez et al. 2022 "New insights into the genetic etiology of Alzheimer's disease"; PD: Nalls et al. 2019 "Identification of novel risk loci, causal insights, and heritable risk for Parkinson's disease"). Open the Summary statistics tab and download the harmonised file.
2. Open the header: `zcat file.tsv.gz | head -3`. Identify columns for rsID, p-value and sample size (names differ by study). Record them in docs/data_sources.md.
3. Download MAGMA from https://cncr.nl/research/magma/ (binary for your OS), plus the reference panel `g1000_eur` and the gene location file `NCBI37.3.gene.loc` listed on the same page.

> **Citations and data-use**
>
> All data above are public. Cite every dataset paper and accession in the manuscript and in README. For controlled-access data (e.g. ROSMAP via Synapse) do not redistribute; SEA-AD, GEO and GWAS Catalog files are open.

# Part 4. The notebooks, cell by cell

Every code cell is shown, followed by (i) an expected-output box, (ii) the expected figure where there is one, (iii) a "how to read it" box on the key cells, and (iv) a table of specific problems and fixes. Green boxes are the REAL output of this exact code on the synthetic benchmark. Orange boxes are format examples for steps that cannot run without real data or the internet. All code is also delivered as ready-to-run .ipynb files in the project bundle, so you do not have to copy from this PDF.

## Notebook 00 - Synthetic benchmark (smoke test with known ground truth)

| Purpose | Creates three synthetic mouse cohorts, 60 synthetic human donors and stand-in auxiliary files, with a PLANTED, known effect. Use it to test your installation and to see what correct output looks like before touching real data. |
|---|---|
| Needs | none |
| Produces | data/processed/*_raw.h5ad, data/raw/seaad/..., data/metadata/animals.csv, auxiliary files |
| Time | 3-5 min |

### Cell 1 - Import the generator and inspect its gene universe

`src/mgcoupling/sim.py` (listed in Appendix B) contains a generative model of microglial scRNA-seq: latent mitophagy and lysosome activities whose within-animal correlation is planted (lower in old animals), plus depth variation, DAM and interferon states, contaminant clusters (BAM, neurons), low-quality cells, doublets and a "regulator" gene set that tracks the coupling in young animals. Nothing in it depends on real data.

*Code*

```
import sys; sys.path.append('../src')
import numpy as np, pandas as pd, scanpy as sc, anndata as an, os
import matplotlib.pyplot as plt
from mgcoupling.sim import *
print(G, 'genes;', len(named), 'named,', G - len(named), 'anonymous background')
print({'MITOPHAGY': len(mitophagy), 'LYSOSOME': len(lysosome), 'PROTEASOME': len(proteasome),
       'RIBOSOME': len(ribosome), 'OXPHOS': len(oxphos), 'planted regulators': regulators})
print('expected UMIs per 3,500-UMI cell: mitophagy mean = %.2f, lysosome mean = %.2f' %
      (3500 * base[idx(mitophagy)].mean(), 3500 * base[idx(lysosome)].mean()))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
3200 genes; 214 named, 2986 anonymous background
{'MITOPHAGY': 18, 'LYSOSOME': 33, 'PROTEASOME': 31, 'RIBOSOME': 45, 'OXPHOS': 18, 'planted regulators': ['Tfe...
expected UMIs per 3,500-UMI cell: mitophagy mean = 0.14, lysosome mean = 0.49
```

| If you see... | What it means and what to do |
|---|---|
| ModuleNotFoundError: scanpy | You are not in the project environment. Run `conda activate mgcoupling` and select the "Python (mgcoupling)" kernel in Jupyter (Kernel > Change Kernel). |
| ModuleNotFoundError: mgcoupling.sim | `src/mgcoupling/sim.py` is missing (copy it from the repository bundle or Appendix B) or the notebook is not running inside `notebooks/`. |

### Cell 2 - Build three mouse cohorts with a planted effect

Three cohorts mimic the real design: hammond (4 young + 4 old WT, plus 2 5xFAD), ximerakis (3 vs 3 pooled samples, weaker effect) and tms (4 young, 4 old, 2 middle-aged). Planted truth: latent rho falls from 0.50 to 0.20 (hammond), 0.45 to 0.30 (ximerakis), 0.55 to 0.25 (tms). Raw files are written exactly as Notebook 01 would write real ones.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
wt = lambda ages, g, pooled=1: [(a, g, 'WT', s, pooled) for a, s in zip(ages, 'MFMFMF')]
designs = {
  'hammond':   (wt([3.3] * 4, 'young') + wt([17.8] * 4, 'old') + [(17.8, 'old', '5xFAD', 'M', 1)] * 2,
  » 0.50, 0.20, 700, 11),
  'ximerakis': (wt([2.5] * 3, 'young', 2) + wt([22] * 3, 'old', 2), 0.45, 0.30, 600, 12),
  'tms':       (wt([3] * 4, 'young') + wt([18] * 4, 'old') + [(12, 'mid', 'WT', 'M', 1)] * 2, 0.55,
  » 0.25, 700, 13),
}
all_meta = []
for name, (design, ry, ro, n, seed) in designs.items():
    ad, meta = simulate_cohort(name, design, ry, ro, n, seed)
    ad.layers['counts'] = ad.X.copy()                    # same layout Notebook 01 produces
    ad.write(f'../data/processed/{name}_raw.h5ad'); all_meta.append(meta)
    print(name, ad.shape, 'animals:', ad.obs.animal_id.nunique())
pd.concat(all_meta).to_csv('../data/metadata/animals.csv', index=False)
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
hammond (7000, 3200) animals: 10
ximerakis (3600, 3200) animals: 6
tms (7000, 3200) animals: 10
```

| If you see... | What it means and what to do |
|---|---|
| Writing .h5ad fails with a TypeError about a column of mixed types | Make sure every obs column holds one type (all strings or all numbers). Convert with `.astype(str)`. |
| MemoryError | Reduce the number of cells per animal (the 700 / 600 / 700 values) to 400. The simulation needs about 1.5 GB. |

### Cell 3 - Simulate human donors (SEA-AD-like) and auxiliary files

60 donors with age, sex and Braak stage in the same layout as a CELLxGENE download (Ensembl IDs as index, `feature_name` symbols, counts in `.raw`); coupling declines with Braak stage. The cell also writes small stand-ins for the DAM signature, the CCCP perturbation table, a CRISPR-screen table, an aging proteome and an ortholog table. All of these are replaced by real files in the real analysis.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
rng = np.random.default_rng(99)
roman = ['0', 'I', 'II', 'III', 'IV', 'V', 'VI']
parts = []
for d in range(60):
    braak = int(rng.choice(range(7), p=[.08, .12, .15, .2, .2, .15, .1]))
    age = int(rng.integers(62, 96)); rho = float(np.clip(0.50 - 0.05 * braak + rng.normal(0, .05), .02,
    » .9))
    X, obs = simulate_animal(rng, 150, rho, 0.5, [0.88, 0.06, 0.04, 0.015, 0.005], reg=0.9 - 0.1 * braak)
    obs['donor_id'] = f'D{d + 1:03d}'; obs['development_stage'] = f'{age}-year-old human stage'
    obs['sex'] = rng.choice(['male', 'female']); obs['Braak stage'] = 'Braak stage ' + roman[braak]
    obs['Continuous Pseudo-progression Score'] = float(np.clip(braak / 6 + rng.normal(0, .08), 0, 1))
    obs['true_rho_latent'] = rho
    obs.index = [f'D{d + 1:03d}_n{i:04d}' for i in range(150)]
    parts.append(an.AnnData(X=X, obs=obs, var=pd.DataFrame(index=genes)))
h = an.concat(parts, join='outer', index_unique=None)
h.var = pd.DataFrame({'feature_name': [g.upper() for g in genes]},
» index=['ENSG%011d' % i for i in range(G)])
h.raw = h.copy()                                    # raw counts kept in .raw, as in CELLxGENE files
sc.pp.normalize_total(h, target_sum=1e4); sc.pp.log1p(h)
os.makedirs('../data/raw/seaad', exist_ok=True); os.makedirs('../data/raw/screens', exist_ok=True)
os.makedirs('../data/raw/proteomics', exist_ok=True)
h.write('../data/raw/seaad/seaad_mtg_microglia.h5ad'); print('donors:', h.obs.donor_id.nunique(), h.shape)

pd.DataFrame({'gene': dam + ['Fth1', 'Ctss']}).to_csv('../data/metadata/dam_signature.csv', index=False)
pd.DataFrame({'mouse': genes, 'human': [g.upper() for g in genes]}).to_csv(
    '../data/processed/mouse_human_orthologs.csv', index=False)
from scipy.stats import norm
r = np.random.default_rng(5); lfc = r.normal(0, .35, G)
lfc[idx(lysosome)] += 0.45; lfc[idx(mitophagy)] += 0.25
pv = 2 * norm.sf(np.abs(lfc / 0.18 + r.normal(0, 0.5, G)))
pd.DataFrame({'Gene': genes, 'logFC': lfc, 'PValue': pv}).to_csv(
    '../data/raw/GSE179611_DMSOLPS_vs_CCCPLPS-ExpDiff.csv.gz', index=False)
is_lys = np.isin(np.arange(G), idx(lysosome))
hit = r.random(G) < np.where(is_lys, 0.35, 0.05)
pd.DataFrame({'gene': [g.upper() for g in genes], 'score': r.normal(size=G),
              'fdr': np.where(hit, 0.02, 0.6)}).to_csv('../data/raw/screens/screen_hits.csv', index=False)
pl = r.normal(0, .3, G); pl[idx(lysosome)] += 0.2
pd.DataFrame({'gene': genes, 'log2fc': pl,
» 'p': 0.05}).to_csv('../data/raw/proteomics/aged_microglia_proteome.csv', index=False)
print('auxiliary files written')
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
donors: 60 (9000, 3200)
auxiliary files written
```

| If you see... | What it means and what to do |
|---|---|
| KeyError: feature_name later in Notebook 09 | The donor file has no `feature_name` column; re-run this cell, or (for real SEA-AD data) check `h.var.columns`. |

## Cell 4 - Ground truth: what we planted (Figure 0)

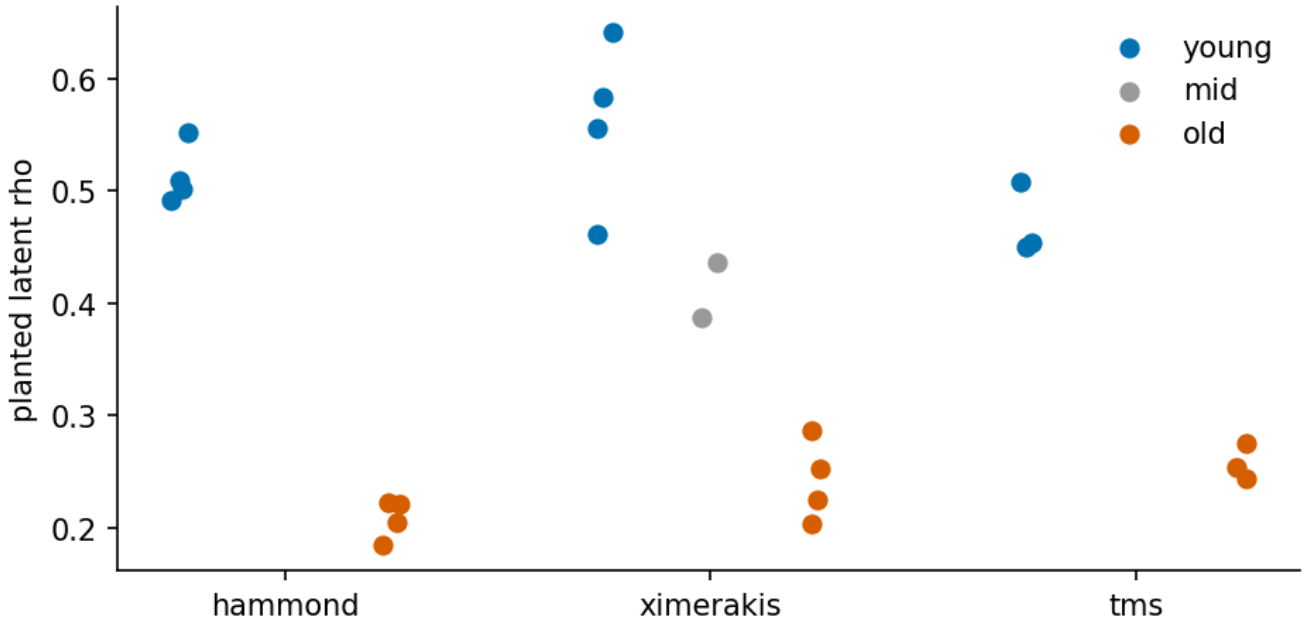
The planted latent correlation for every animal. After Notebooks 02-12 you should recover the same pattern from the counts alone, with attenuation (observed correlations are lower than latent ones because of sparsity).

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
tru = pd.concat([sc.read_h5ad(f'../data/processed/{n}_raw.h5ad').obs
                 .drop_duplicates('animal_id')[['dataset', 'animal_id', 'age_group', 'condition',
                » 'true_rho_latent']]
                 for n in designs])
tru.to_csv('../results/tables/00_planted_truth.csv', index=False)
col = {'young': '#0072B2', 'mid': '#999999', 'old': '#D55E00'}
fig, ax = plt.subplots(figsize=(6.4, 3.2))
for i, (n, g) in enumerate(tru.groupby('dataset')):
    for j, grp in enumerate(['young', 'mid', 'old']):
        v = g[(g.age_group == grp) & (g.condition == 'WT')].true_rho_latent
        ax.scatter(np.full(len(v), i * 4 + j) + rng.uniform(-.1, .1, len(v)), v, c=col[grp], s=34,
        » label=grp if i == 0 else None)
ax.set_xticks([i * 4 + 1 for i in range(3)]); ax.set_xticklabels(list(designs))
ax.set_ylabel('planted latent rho'); ax.legend(frameon=False); ax.spines[['top',
» 'right']].set_visible(False)
plt.tight_layout(); plt.show()
print(tru[tru.condition == 'WT'].groupby(['dataset',
» 'age_group']).true_rho_latent.mean().round(2).unstack().to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
age_group   mid   old  young
dataset
hammond     NaN  0.21   0.51
tms        0.41  0.24   0.56
ximerakis   NaN  0.26   0.47
```




***Expected figure*** *(00_4): real output of this code on the synthetic benchmark.*

**How to read it**

The synthetic world has planted decoupling: young animals around 0.45-0.58 and old around 0.16-0.27 (latent scale). The weaker ximerakis effect (0.45 to 0.30) is deliberate: real cohorts differ in strength.

## Notebook 01 - Load real cohorts and attach metadata (skip when running the synthetic smoke test)

| Purpose | Downloads the real cohorts, attaches animal metadata and writes the same *_raw.h5ad files that Notebook 00 writes. Skip it when running the synthetic smoke test. |
|---|---|
| Needs | GEO / CELLxGENE downloads |
| Produces | data/processed/<cohort>_raw.h5ad, data/metadata/animals.csv |
| Time | hours (downloads), 10-30 min compute |

### Cell 1 - Imports and a GEO download helper

Builds the GEO FTP-over-https path for a series and downloads its supplementary files (all, or only those matching a regular expression). Check file sizes on the GEO page before downloading.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import re, os, requests, pandas as pd, numpy as np, scanpy as sc

def geo_suppl_url(gse):
    return 'https://ftp.ncbi.nlm.nih.gov/geo/series/' + gse[:-3] + 'nnn/' + gse + '/suppl/'

def download_geo_suppl(gse, outdir, pattern=None):
    os.makedirs(outdir, exist_ok=True)
    url = geo_suppl_url(gse)
    html = requests.get(url, timeout=60).text
    for f in re.findall(r'href="([^"?/][^"]*)"', html):
        if pattern and not re.search(pattern, f):
            continue
        dest = os.path.join(outdir, f)
        if os.path.exists(dest):
            continue
        print('downloading', f)
        with requests.get(url + f, stream=True, timeout=600) as r:
            r.raise_for_status()
            with open(dest, 'wb') as fh:
                for chunk in r.iter_content(1 << 20):
                    fh.write(chunk)
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
(no output)
```

| If you see... | What it means and what to do |
|---|---|
| requests.exceptions.ConnectionError | Network or proxy problem. Open the GEO page in a browser, click the (http) link of each supplementary file, and save into `data/raw/<cohort>/` by hand; the rest of the guide is unchanged. |
| HTTP 404 for a series | Series numbers are grouped by thousands: GSE121654 lives in `GSE121nnn`. The helper does this; check the accession for typos. |

## Cell 2 - Download one cohort (example: GSE121654)

Download only what you need. Always check the sample table and file list on the GEO page first (Part 3).

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
download_geo_suppl('GSE121654', '../data/raw/hammond')
print(sorted(os.listdir('../data/raw/hammond'))[:10])
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
downloading GSE121654_RAW.tar
['GSE121654_RAW.tar']
(format example - file names and sizes differ by series; extract archives with `tar -xvf` or your file manager)
```

| If you see... | What it means and what to do |
|---|---|
| Download interrupted | Delete the partial file and run again; the helper skips only complete-looking files that already exist, so remove the partial one first. |

## Cell 3 - Sample metadata from GEO (GEOparse)

GEO sample tables list age, sex, tissue and genotype in free-text columns. Print them and copy the relevant values into `data/metadata/animals.csv` by hand: for 10-60 samples this is faster and safer than automating.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
import GEOparse
gse = GEOparse.get_GEO(geo='GSE121654', destdir='../data/raw/geo_meta', silent=True)
ph = gse.phenotype_data
pd.set_option('display.max_colwidth', 80)
print(ph.columns.tolist())
print(ph[['title', 'source_name_ch1']].head(8).to_string())
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
['title', 'geo_accession', 'status', ..., 'source_name_ch1', 'characteristics_ch1.0.strain', ...]
   title                source_name_ch1
 GSM...  P100_male_rep1       microglia
(format example - the real table lists each mouse's age and sex in free text)
```

| If you see... | What it means and what to do |
|---|---|
| GEOparse fails or is slow | Open the series page in the browser and read the Samples table; copy it into a spreadsheet. The analysis only needs the final animals.csv. |
| Ages given as P-days | Convert to months (P100 = 3.3 months; P540 = 17.8 months). Write the conversion in docs/data_sources.md. |

### Cell 4 - Metadata template: animals.csv

One row per animal (or per pooled sample when the study pooled mice). `age_group` follows the preregistered rule: young <= 6 months, old >= 18 months, everything between is "mid" (kept for continuous analyses, excluded from the binary test).

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
template = pd.DataFrame(columns=['dataset', 'sample_id', 'animal_id', 'age_months', 'age_group',
                                 'sex', 'region', 'condition', 'pooled_n'])
template.loc[0] = ['hammond', 'GSM_XXXX', 'hammond_M1', 3.3, 'young', 'M', 'whole_brain', 'WT', 1]
template.to_csv('../data/metadata/animals_template.csv', index=False)
# Fill the real rows in a spreadsheet, save as ../data/metadata/animals.csv, then:
meta = pd.read_csv('../data/metadata/animals.csv')
print(meta.groupby(['dataset', 'age_group', 'condition']).animal_id.nunique().unstack().to_string())
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
age_group              mid  old  young
dataset   condition
hammond   WT           NaN    4      4
          5xFAD        NaN    2    NaN
(format example)
Check: every cohort used in the primary test needs >= 3 animals in both young and old.
```

| If you see... | What it means and what to do |
|---|---|
| A cohort has fewer than 3 animals per group | Move it to the supplement or find another cohort; do not report p-values from n = 2 (Section 1.4). |

### Cell 5 - Load one cohort of per-sample text matrices (example: Hammond)

Use `zcat file.txt.gz | head -c 600` (Mac/Linux/Git Bash) to see whether genes are rows or columns, and set `genes_in_rows` accordingly. Cells are renamed with the sample ID so barcodes cannot collide.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
meta = pd.read_csv('../data/metadata/animals.csv')
files = {s: f'../data/raw/hammond/{s}_dge.txt.gz' for s in meta[meta.dataset == 'hammond'].sample_id}
parts = [load_dge_txt(p, s, genes_in_rows=True) for s, p in files.items()]
ad = sc.concat(parts, join='outer', index_unique=None)
ad = attach_meta(ad, meta)
ad.layers['counts'] = ad.X.copy()
print(ad.shape, '| counts-like:', looks_like_counts(ad.X))
print(ad.obs.groupby('animal_id', observed=True).size().describe().round(0).to_string())
ad.write('../data/processed/hammond_raw.h5ad')
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
(14522, 24118) | counts-like: True
count     8
mean   1815
min     903
max    2764
cells without metadata: 0
(format example with invented numbers)
```

| If you see... | What it means and what to do |
|---|---|
| cells without metadata > 0 | The `sample_id` values in animals.csv do not match the file stems. Print `ad.obs.sample_id.unique()[:5]` and `meta.sample_id[:5]`. |

| If you see... | What it means and what to do |
|---|---|
| counts-like: False | The file holds normalised values. Look for a raw-count file on GEO; QC and DE need integers. |
| MemoryError | Dense text matrices are large. Read and convert one sample at a time (as here) and delete `df` after each; or use a 16 GB machine. |

## Cell 6 - Load a CELLxGENE cohort (example: Tabula Muris Senis brain)

CELLxGENE files hold raw counts in `.raw` and gene symbols in `var["feature_name"]`; `load_cellxgene` handles both. Print the cell-type and mouse-ID columns first, then select the microglia/myeloid cells and set `sample_id` to the mouse identifier.

*Code (not run in the benchmark: needs real data, the network or a terminal) [» marks a printed continuation; the notebook holds the line unwrapped]*

```
ad = load_cellxgene('../data/raw/tms/tms_brain.h5ad')
print(ad.obs.columns.tolist())
print(ad.obs['cell_type'].value_counts().head(8).to_string())
ad = ad[ad.obs['cell_type'].astype(str).str.contains('microglia|myeloid', case=False)].copy()
ad.obs['sample_id'] = ad.obs['donor_id'].astype(str)          # EDIT: the mouse identifier column of
» your file
ad = attach_meta(ad, pd.read_csv('../data/metadata/animals.csv'))
ad.layers['counts'] = ad.X.copy()
ad.write('../data/processed/tms_raw.h5ad')
print(ad.shape, ad.obs.groupby(['age_group']).animal_id.nunique().to_string())
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
counts-like: True
['cell_type', 'donor_id', 'age', 'sex', 'tissue', ...]
microglial cell    11234
macrophage          1987
...
(8421, 21989)  age_group
old      4
young    4
(format example)
```

| If you see... | What it means and what to do |
|---|---|
| KeyError: cell_type | The column is named differently (`cell_ontology_class`, `free_annotation`). Print the columns and use the one holding cell-type names. |
| After subsetting, very few cells | The string filter missed the label (e.g. "brain myeloid cell"). Print `value_counts()` and adjust the pattern. |

## Cell 7 - Your Guide 1 data (GSE179358) as an exploratory cohort

Same loader as in Guide 1, kept as a supplementary cohort only: one pooled library per age means age and batch are confounded and it cannot enter the primary test.

*Code (not run in the benchmark: needs real data, the network or a terminal) [» marks a printed continuation; the notebook holds the line unwrapped]*

```
d = '../data/raw/GSE179358/GSE179358_RAW'
samples = [('GSM5416205_6mth', 6), ('GSM5416206_12mth', 12), ('GSM5416207_18mth', 18),
» ('GSM5416208_24mth', 24)]
parts = [load_10x_mtx(d, p, p) for p, _ in samples]
ad = sc.concat(parts, join='outer', index_unique=None)
ad.obs['age_months'] = ad.obs['sample_id'].map(dict(samples))
ad.obs['animal_id'] = ad.obs['sample_id']; ad.obs['dataset'] = 'gse179358_pooled'
ad.layers['counts'] = ad.X.copy(); ad.write('../data/processed/gse179358_raw.h5ad'); print(ad)
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
AnnData object with n_obs x n_vars = 9876 x 27998
    obs: 'sample_id', 'age_months', 'animal_id', 'dataset'
    layers: 'counts'
(format example)
```

| If you see... | What it means and what to do |
|---|---|
| FileNotFoundError | Extract GSE179358_RAW.tar so the .gz files sit directly in the folder named in `d`. |

# Notebook 02 - Quality control, doublets, clustering, microglia selection

| Purpose | Per-cohort quality control with MAD rules, doublet detection, normalisation, clustering and cluster-level microglia selection. The same rules for every cohort. |
|---|---|
| Needs | <cohort>_raw.h5ad |
| Produces | <cohort>_microglia.h5ad, docs/cluster_decisions_<cohort>.csv |
| Time | 5 min per cohort (synthetic); 10-30 min (real) |

## Cell 1 - Imports and the MAD helper

MAD (median absolute deviation) rules are data-driven: a cell is an outlier if it lies more than k MADs from its own sample's median. This adapts to each cohort and sample instead of using one fixed cut-off for everything.

*Code*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import scanpy as sc, numpy as np, pandas as pd, matplotlib.pyplot as plt
from scipy import stats
import scrublet as scr

def mad_flag(v, nmads, side='both'):
    med, mad = np.median(v), stats.median_abs_deviation(v)
    lo, hi = med - nmads * mad, med + nmads * mad
    return (v > hi) if side == 'high' else ((v < lo) | (v > hi))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

| If you see... | What it means and what to do |
|---|---|
| ModuleNotFoundError: No module named mgcoupling | The notebook is not running from `notebooks/`, or `src/mgcoupling/__init__.py` is missing. Check `import os; print(os.getcwd())`; it must end in `notebooks`. |
| ModuleNotFoundError: scrublet | Run `pip install scrublet` inside the activated environment, then restart the kernel. |

## Cell 2 - QC flagging function

Flags (does not yet delete) low-quality cells per sample: total counts and detected genes outside 5 MADs, mitochondrial percentage above 5 MADs AND above 10%, or fewer than 200 genes. The mitochondrial rule is switchable because single-nucleus data have almost no mitochondrial reads.

*Code*

```
def qc_flag(ad, use_mt=True):
    ad.var['mt'] = ad.var_names.str.lower().str.startswith('mt-')
    sc.pp.calculate_qc_metrics(ad, qc_vars=['mt'], inplace=True, percent_top=None)
    ad.obs['log_counts'] = np.log1p(ad.obs['total_counts'])
    ad.obs['n_genes'] = ad.obs['n_genes_by_counts']
    ok = pd.Series(True, index=ad.obs_names)
    for s in ad.obs['sample_id'].unique():
        o = ad.obs[ad.obs['sample_id'] == s]
        bad = mad_flag(o['log_counts'].values, 5) | mad_flag(np.log1p(o['n_genes'].values), 5)
        if use_mt:
            bad |= mad_flag(o['pct_counts_mt'].values, 5, 'high') & (o['pct_counts_mt'].values > 10)
        bad |= o['n_genes'].values < 200
        ok[o.index[bad]] = False
    ad.obs['qc_pass'] = ok.values
    return ad
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

| If you see... | What it means and what to do |
|---|---|
| KeyError: sample_id | Metadata were not attached in Notebook 01 (`attach_meta`). Check `ad.obs.columns`. |

| If you see... | What it means and what to do |
|---|---|
| Everything is flagged as failing | Gene symbols may be Ensembl IDs, so `mt-` genes are not found, or counts are not raw. Print `ad.var_names[:5]` and `ad.X.max()`. |

### Cell 3 - Doublet detection function (per sample)

Scrublet simulates doublets from your own cells and scores each real cell by similarity to them. It must be run per sample (per library), never on pooled samples.

*Code*

```
def run_scrublet(ad):
    ad.obs['doublet'] = False
    for s in ad.obs['sample_id'].unique():
        m = (ad.obs['sample_id'] == s).values
        try:
            scrub = scr.Scrublet(ad.layers['counts'][m], expected_doublet_rate=0.06)
            _, pred = scrub.scrub_doublets(verbose=False)
            ad.obs.loc[ad.obs.index[m], 'doublet'] = np.asarray(pred, dtype=bool)
        except Exception as e:
            print('scrublet failed for', s, '->', e)
    return ad
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

| If you see... | What it means and what to do |
|---|---|
| Scrublet: "Failed to find a threshold" / hangs | Common with very small samples (< 300 cells). Skip doublet removal for that sample and say so in the Methods, or lower `expected_doublet_rate`. |
| AttributeError: layers counts | Create the layer first: `ad.layers["counts"] = ad.X.copy()` (Notebook 01 does this). |

### Cell 4 - Load one cohort and flag low-quality cells

Change `name` to run each cohort (hammond, ximerakis, tms). Everything from this cell to the end of the notebook is repeated per cohort.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
name = 'hammond'                      # repeat the notebook for every cohort
ad = sc.read_h5ad(f'../data/processed/{name}_raw.h5ad')
ad = qc_flag(ad, use_mt=True)         # use_mt=False for single-nucleus data
print(ad.obs.groupby('animal_id', observed=True).qc_pass.agg(['size', 'mean']).round(3).head(6))
print('overall pass rate: %.3f' % ad.obs.qc_pass.mean())
if 'lowq_true' in ad.obs:             # SIMULATION ONLY: compare with the truth we planted
    print(pd.crosstab(ad.obs.lowq_true, ad.obs.qc_pass, rownames=['truly low quality'],
    » colnames=['QC pass']))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
            size   mean
animal_id
hammond_01   700  0.967
hammond_02   700  0.969
hammond_03   700  0.956
hammond_04   700  0.967
hammond_05   700  0.973
hammond_06   700  0.961
overall pass rate: 0.968
QC pass            False  True
truly low quality
False                 14   6685
True                 213     88
```

**How to read it**

Pass rate 96.8%. Because the truth is known here, you can see that 213 of 301 truly damaged cells were caught (71%) and 14 good cells were lost (0.2%). In real data there is no truth table: look for a pass rate of 90-98% that is similar across animals; one animal with a much lower rate is a sample-quality problem to report.

| If you see... | What it means and what to do |
|---|---|
| FileNotFoundError: ..._raw.h5ad | Notebook 01 (real data) or Notebook 00 (synthetic) has not been run for this cohort. |

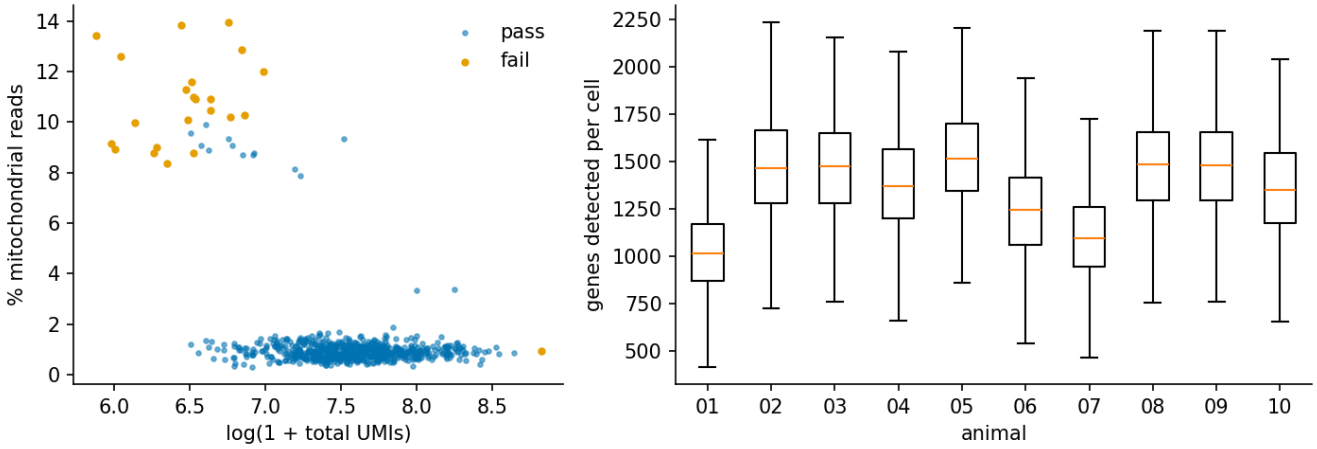
## Cell 5 - Figure: QC before filtering

Always look at the distributions before trusting the thresholds. Left: depth against mitochondrial percentage for one animal, with failing cells in orange. Right: genes per cell for every animal.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
fig, ax = plt.subplots(1, 2, figsize=(9.5, 3.4), gridspec_kw={'width_ratios': [1, 1.3]})
one = ad.obs[ad.obs.animal_id == ad.obs.animal_id.iloc[0]]
ax[0].scatter(one.log_counts[one.qc_pass], one.pct_counts_mt[one.qc_pass], s=5, c='#0072B2', alpha=.5,
» label='pass')
ax[0].scatter(one.log_counts[~one.qc_pass], one.pct_counts_mt[~one.qc_pass], s=9, c='#E69F00',
» label='fail')
ax[0].set_xlabel('log(1 + total UMIs)'); ax[0].set_ylabel('% mitochondrial reads');
» ax[0].legend(frameon=False)
animals = list(ad.obs.animal_id.unique())
ax[1].boxplot([ad.obs.loc[ad.obs.animal_id == a, 'n_genes'] for a in animals], showfliers=False)
ax[1].set_xticks(range(1, len(animals) + 1)); ax[1].set_xticklabels([a.split('_')[-1] for a in animals])
ax[1].set_xlabel('animal'); ax[1].set_ylabel('genes detected per cell')
for a_ in ax: a_.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(02_5): real output of this code on the synthetic benchmark.*

| If you see... | What it means and what to do |
|---|---|
| Figure not shown in Jupyter | Add `%matplotlib inline` in the first cell, or restart the kernel. |

## Cell 6 - Remove failing cells, then detect doublets

Subset to passing cells, run Scrublet per sample, and remove predicted doublets.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
ad = ad[ad.obs['qc_pass'].values].copy()
ad = run_scrublet(ad)
print('predicted doublets:', int(ad.obs.doublet.sum()), 'of', ad.n_obs, 'cells')
if 'doublet_true' in ad.obs:          # SIMULATION ONLY
    print(pd.crosstab(ad.obs.doublet_true, ad.obs.doublet, rownames=['true doublet'],
    » colnames=['predicted']))
ad = ad[~ad.obs['doublet'].values].copy()
print('cells kept:', ad.n_obs)
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
predicted doublets: 151 of 6773 cells
predicted     False  True
true doublet
False          6493    139
True            129     12
cells kept: 6622
```

**How to read it**

Scrublet flagged 151 cells but caught only 12 of 141 true doublets: it detects doublets of two DIFFERENT cell types, not two cells of the same type. Real data behave the same, so report what was removed and state that homotypic doublets remain. This matters little for per-animal correlations over hundreds of cells.

| If you see... | What it means and what to do |
|---|---|
| Scrublet removes far more than 6-10% of cells | Check that the matrix is raw counts, and that you did not pass pooled samples. If the cohort is a sorted single cell type, high predicted rates can be genuine heterogeneity; inspect the threshold histogram (`scrub.plot_histogram()`). |

## Cell 7 - Normalise, find variable genes, reduce dimensions, cluster

Counts are scaled to 10,000 per cell and log-transformed (this log-normalised matrix is what every score in the project uses). Highly variable genes are chosen per sample on raw counts (seurat_v3), PCA/UMAP are computed on a scaled copy, and cells are clustered with Leiden. Mitochondrial percentage is NOT regressed out.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
sc.pp.normalize_total(ad, target_sum=1e4)
sc.pp.log1p(ad)
sc.pp.highly_variable_genes(ad, n_top_genes=2000, flavor='seurat_v3', layer='counts',
» batch_key='sample_id')
adh = ad[:, ad.var.highly_variable].copy()
sc.pp.scale(adh, max_value=10)
sc.tl.pca(adh, n_comps=30)
sc.pp.neighbors(adh, n_neighbors=15, n_pcs=30)
sc.tl.umap(adh)
sc.tl.leiden(adh, resolution=0.8, flavor='igraph', n_iterations=2, directed=False)
ad.obsm['X_umap'] = adh.obsm['X_umap']
ad.obs['leiden'] = adh.obs['leiden'].values
print(ad.obs['leiden'].value_counts().sort_index().to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
leiden
0    1502
1    1266
2    1040
3    1420
4     729
5     147
6     429
7      89
```

| If you see... | What it means and what to do |
|---|---|
| ImportError: scikit-misc | seurat_v3 needs scikit-misc: `pip install scikit-misc`. Fallback: `flavor="seurat"` (log-normalised data, no layer argument). |
| TypeError: leiden() got an unexpected keyword argument flavor | Your scanpy is older than 1.10. Upgrade (`pip install -U scanpy`) or drop `flavor`, `n_iterations`, `directed` and install `leidenalg`. |
| Kernel dies during PCA/UMAP | Out of memory. Use fewer HVGs (1500), or process one cohort at a time with other notebooks closed. |

## Cell 8 - Cluster-level marker table (the basis for keeping or dropping clusters)

Scores pan-myeloid markers, homeostatic markers and non-myeloid lineage markers, then averages them per cluster. Decisions are made per cluster, not per cell, because disease-associated microglia legitimately lose homeostatic markers.

*Code*

```
pan_myeloid = ['Csf1r', 'C1qa', 'C1qb', 'Tyrobp', 'Hexb', 'Cx3cr1', 'Aif1']
homeo = ['P2ry12', 'Tmem119', 'Siglech', 'Selplg', 'Sall1', 'Fcrls']
non_my = {'neuron': ['Snap25', 'Rbfox3', 'Syt1'], 'astro': ['Gfap', 'Aqp4', 'Slc1a3'],
          'oligo': ['Plp1', 'Mbp', 'Mog'], 'endo': ['Cldn5', 'Pecam1'], 'tcell': ['Cd3e', 'Cd3g'],
          'bam': ['Mrc1', 'Lyve1', 'Pf4', 'Cd163'], 'mono': ['Ly6c2', 'Ccr2']}
groups = {'pan_myeloid': pan_myeloid, 'homeostatic': homeo, **non_my}
sc_df = module_scores(ad, groups, min_genes=2)
sc_df['leiden'] = ad.obs['leiden'].values
tab = sc_df.groupby('leiden', observed=True).mean().round(2)
tab['n_cells'] = ad.obs['leiden'].value_counts()
print(tab[['pan_myeloid', 'homeostatic', 'neuron', 'bam', 'mono', 'n_cells']].to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
WARNING astro only 0 genes found
WARNING oligo only 0 genes found
WARNING endo only 0 genes found
WARNING tcell only 0 genes found
WARNING mono only 0 genes found
        pan_myeloid  homeostatic  neuron   bam  mono  n_cells
leiden
0              0.13         0.25   -0.11 -0.14   NaN     1502
1             -0.08         0.04   -0.12 -0.15   NaN     1266
2              0.12         0.23   -0.10 -0.14   NaN     1040
3              0.03         0.16   -0.11 -0.14   NaN     1420
4             -0.36        -1.19   -0.12 -0.16   NaN      729
5             -0.04        -0.83   -0.11  6.41   NaN      147
6              0.04         0.20   -0.12 -0.14   NaN      429
7              0.05         0.18    8.29 -0.14   NaN       89
```

### How to read it

Read the table by columns. Cluster 4 has low homeostatic (-1.19) and low pan-myeloid (-0.36) scores but no lineage signal: it is the DAM cluster and MUST be kept. Cluster 5 has a BAM score of 6.4 and cluster 7 a neuron score of 8.3: contaminants. Contaminant signals are an order of magnitude larger than anything in genuine microglia.

| If you see... | What it means and what to do |
|---|---|
| WARNING ... only 0 genes found | Symbols not matching (case or species). Check `ad.var_names[:10]`. For human data use upper-case symbols. |
| Table is empty / all zeros | The matrix in `ad.X` is not log-normalised. Re-run Cell 7. |

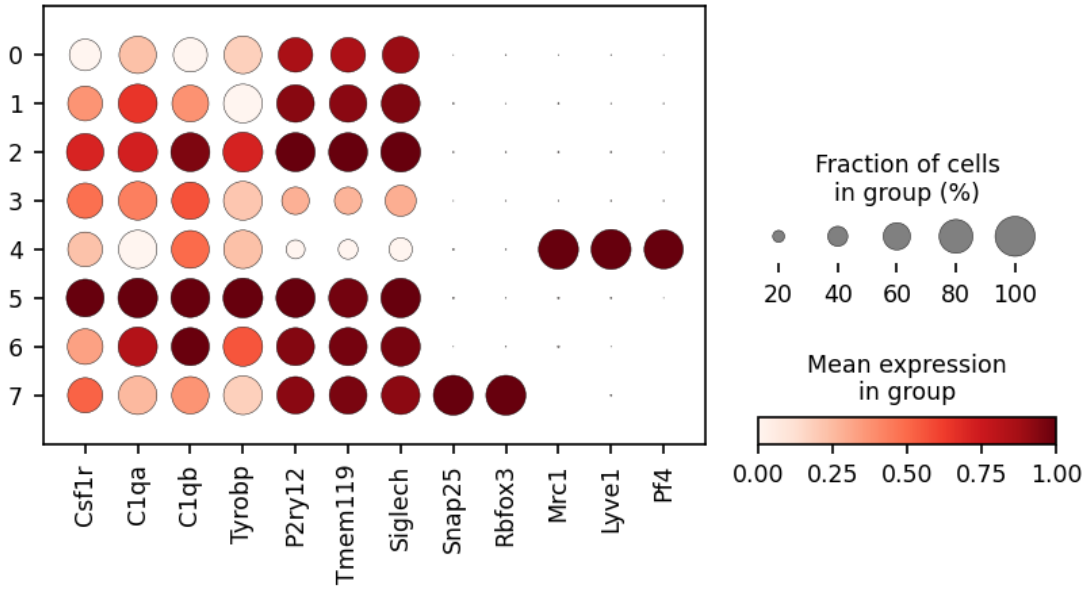
## Cell 9 - Dot plot of the markers by cluster

A visual check of the table: a contaminant cluster shows strong signal for neuronal or BAM markers while keeping low homeostatic markers.

*Code*

```
panel = pan_myeloid[:4] + homeo[:3] + ['Snap25', 'Rbfox3', 'Mrc1', 'Lyve1', 'Pf4']
sc.pl.dotplot(ad, [g for g in panel if g in ad.var_names], groupby='leiden',
              standard_scale='var', show=True)
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(02_9): real output of this code on the synthetic benchmark.*

| If you see... | What it means and what to do |
|---|---|
| dotplot raises "ValueError: ... not in var_names" | Filter the gene list to those present, as in this cell. |

### Cell 10 - Record the decision per cluster and save

An automatic suggestion (strong non-myeloid LINEAGE signal) is shown first; YOU review it against the table and dot plot and edit `drop` if needed. The decision table is saved for the supplement. Do not add a rule such as "low pan-myeloid score": disease-associated microglia lose Hexb and Cx3cr1 and a rule like that silently deletes the very cells you want to study (in our first test it removed the DAM cluster).

*Code*

```
lineage = ['neuron', 'astro', 'oligo', 'endo', 'tcell', 'bam', 'mono']
suggest = tab.index[tab[lineage].max(axis=1) > 0.5]      # lineage markers only (see warning below)
drop = {c: 'non-microglial lineage signal (auto-suggested; reviewed)' for c in suggest}
# edit here if your review disagrees, e.g. drop = {'7': 'BAM (Mrc1, Lyve1, Pf4 high)'}
dec = pd.DataFrame({'cluster': tab.index, 'dataset': name})
dec['decision'] = ['drop' if c in drop else 'keep' for c in dec.cluster]
dec['reason'] = [drop.get(c, 'pan-myeloid high, lineage markers low') for c in dec.cluster]
dec.to_csv(f'../docs/cluster_decisions_{name}.csv', index=False)
print(dec[dec.decision == 'drop'].to_string(index=False))
ad = ad[~ad.obs['leiden'].isin(list(drop))].copy()
if 'state_true' in ad.obs:            # SIMULATION ONLY
    print(ad.obs.state_true.value_counts().to_string())
print(ad.obs.groupby(['age_group', 'animal_id'], observed=True).size().unstack(0).to_string())
ad.write(f'../data/processed/{name}_microglia.h5ad')
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
cluster dataset decision                                                  reason
      5 hammond     drop non-microglial lineage signal (auto-suggested; reviewed)
      7 hammond     drop non-microglial lineage signal (auto-suggested; reviewed)
state_true
homeostatic    5160
DAM             746
IFN             480
age_group     old  young
animal_id
hammond_01    NaN  644.0
hammond_02    NaN  645.0
hammond_03    NaN  635.0
hammond_04    NaN  638.0
hammond_05  626.0    NaN
hammond_06  636.0    NaN
hammond_07  634.0    NaN
hammond_08  654.0    NaN
hammond_09  643.0    NaN
hammond_10  631.0    NaN
```

**How to read it**

Only clusters 5 and 7 are dropped; DAM (746) and IFN (480) cells survive. Remember the trap in the box above: dropping clusters with low pan-myeloid scores would have deleted the DAM cluster. 626-654 cells remain per animal, well above the 100-cell minimum.

| If you see... | What it means and what to do |
|---|---|
| Real data: no cluster is flagged but you can see a contaminant in the dot plot | Lower the thresholds or set `drop` by hand. The automatic rule is only a first guess. |
| An animal ends with < 100 cells | It will be excluded from coupling statistics automatically (min_cells=100). If it is a whole age group, the cohort cannot be used for the primary test. |

## Notebook 03 - Shared state labels across cohorts

| Purpose | Gives all cohorts the same state labels (homeostatic, DAM, interferon...) through an integrated embedding. Labels are used for stratification only. |
|---|---|
| Needs | three <cohort>_microglia.h5ad |
| Produces | state column written into each cohort file |
| Time | 2-5 min |

### Cell 1 - Load all cohorts and combine on shared genes

Integration here is ONLY for giving every cohort the same state labels. Coupling statistics later use each cohort's own, uncorrected log-normalised values.

*Code*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import scanpy as sc, anndata as an, pandas as pd, numpy as np, matplotlib.pyplot as plt
names = ['hammond', 'ximerakis', 'tms']
ads = [sc.read_h5ad(f'../data/processed/{n}_microglia.h5ad') for n in names]
common = sorted(set.intersection(*[set(a.var_names) for a in ads]))
cat = an.concat([a[:, common].copy() for a in ads], join='inner', index_unique=None)
print(cat.shape, '| shared genes:', len(common))
print(cat.obs.groupby('dataset', observed=True).animal_id.nunique().to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
(16120, 3200) | shared genes: 3200
dataset
hammond      10
tms          10
ximerakis     6
```

| If you see... | What it means and what to do |
|---|---|
| Very few shared genes (< 12,000 for mouse) | Cohorts use different gene annotations (symbols vs Ensembl IDs, or old and new symbols). Harmonise symbols first, e.g. map aliases with `mygene` or a symbol-history table. |
| MemoryError at concat | Concatenate only the HVG-selected genes: subset each cohort to 3,000 shared HVGs before `an.concat`. |

### Cell 2 - Variable genes, PCA, Harmony integration, UMAP, clusters

Harmony corrects the PCA embedding for cohort differences (fast, CPU-friendly). We call harmonypy directly because scanpy's wrapper broke with some harmonypy versions. The scVI alternative (Cell 3) is optional.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
sc.pp.highly_variable_genes(cat, n_top_genes=2000, flavor='seurat_v3', layer='counts',
» batch_key='dataset')
adh = cat[:, cat.var.highly_variable].copy()
sc.pp.scale(adh, max_value=10)
sc.tl.pca(adh, n_comps=30)
import harmonypy as hm
ho = hm.run_harmony(adh.obsm['X_pca'], adh.obs, ['dataset'])
Z = np.asarray(ho.Z_corr)                       # orientation differs between harmonypy versions
adh.obsm['X_pca_harmony'] = Z if Z.shape[0] == adh.n_obs else Z.T
sc.pp.neighbors(adh, use_rep='X_pca_harmony', n_neighbors=15)
sc.tl.umap(adh)
sc.tl.leiden(adh, resolution=0.6, flavor='igraph', n_iterations=2, directed=False)
cat.obsm['X_umap'] = adh.obsm['X_umap']; cat.obs['leiden'] = adh.obs['leiden'].values
print(cat.obs['leiden'].value_counts().sort_index().to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
leiden
0    5698
1    3683
2    3866
3    1676
4     173
5    1024
```

| If you see... | What it means and what to do |
|---|---|
| ModuleNotFoundError: harmonypy | `pip install harmonypy`, restart the kernel. |
| ValueError: Value passed for key X_pca_harmony is of incorrect shape | This is the scanpy wrapper `sc.external.pp.harmony_integrate` failing with a newer harmonypy (output transposed). Use the direct `hm.run_harmony` call shown here, which handles both orientations. |
| Harmony runs but cohorts still separate on the UMAP | Cohorts differ in technology (droplet vs plate, cells vs nuclei). Increase `theta` (e.g. `theta=3`) or use scVI. Separation is acceptable for state labelling if each cluster contains cells from all cohorts. |

### Cell 3 (optional) - scVI instead of Harmony

Deep generative integration. Needs `scvi-tools` (PyTorch). On CPU it is slow for more than 100,000 cells; a GPU (or Google Colab) is much faster. Replace Cell 2's Harmony step with this if cohorts do not mix.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
import scvi
adv = cat[:, cat.var.highly_variable].copy()
scvi.model.SCVI.setup_anndata(adv, layer='counts', batch_key='dataset')
model = scvi.model.SCVI(adv, n_latent=30)
model.train(max_epochs=100)
cat.obsm['X_scVI'] = model.get_latent_representation()
sc.pp.neighbors(cat, use_rep='X_scVI'); sc.tl.umap(cat)
sc.tl.leiden(cat, resolution=0.6, flavor='igraph', n_iterations=2, directed=False)
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
Epoch 100/100: 100%|##########| 100/100 [04:12<00:00, 2.5s/it, v_num=1, train_loss_step=1.54e+03]
`Trainer.fit` stopped: `max_epochs=100` reached.
(format example - timing and loss depend on your data and hardware)
```

| If you see... | What it means and what to do |
|---|---|
| RuntimeError / CUDA errors | Force CPU with `model.train(max_epochs=100, accelerator="cpu")`. |
| Counts layer rejected: "non-integer values" | scVI needs raw integer counts. Use `layer="counts"`, never the log-normalised X. |

### Cell 4 - Score state markers, label clusters, check them

Each cluster takes the label of the state whose marker score is highest. Review the result against the dot plot of markers; relabel by hand where the argmax is wrong, and keep a note in the supplement.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
state_markers = {
  'homeostatic': ['P2ry12', 'Tmem119', 'Cx3cr1', 'Hexb', 'Selplg', 'Sall1', 'Siglech'],
  'DAM': ['Trem2', 'Apoe', 'Lpl', 'Cst7', 'Clec7a', 'Itgax', 'Spp1', 'Axl', 'Cd9'],
  'IFN': ['Ifit1', 'Ifit2', 'Ifit3', 'Irf7', 'Isg15', 'Usp18', 'Oasl2'],
  'proliferating': ['Mki67', 'Top2a', 'Stmn1'],
  'antigen_pres': ['H2-Aa', 'H2-Ab1', 'Cd74', 'H2-Eb1'],
  'acute_inflam': ['Il1b', 'Tnf', 'Ccl3', 'Ccl4', 'Nfkbia'],
}
ss = module_scores(cat, state_markers, min_genes=2)
ss['leiden'] = cat.obs['leiden'].values
cl = ss.groupby('leiden', observed=True).mean()
print(cl.round(2).to_string())
lab = cl.idxmax(axis=1)
cat.obs['state'] = cat.obs['leiden'].map(lab).astype(str)
print(cat.obs['state'].value_counts().to_string())
if 'state_true' in cat.obs:           # SIMULATION ONLY: how well did the annotation recover the truth?
    print(pd.crosstab(cat.obs.state_true, cat.obs.state).to_string())
    keep = cat.obs.state_true.isin(['homeostatic', 'DAM', 'IFN'])
    print('accuracy (excluding contaminants): %.3f' % (cat.obs.state_true[keep] ==
    » cat.obs.state[keep]).mean())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
        homeostatic   DAM   IFN  proliferating  antigen_pres  acute_inflam
leiden
0              0.23 -0.15 -0.14           0.04          0.10          0.06
1              0.13 -0.19 -0.17          -0.01          0.01          0.00
2              0.03 -0.26 -0.20          -0.05         -0.13         -0.09
3             -1.21  1.64 -0.17           0.01         -0.02          0.01
4             -0.23 -0.25 -0.19           0.01         -0.40         -0.17
5              0.15 -0.19  2.43           0.01          0.04          0.02
state
homeostatic      13247
DAM               1676
IFN               1024
proliferating      173
state         DAM   IFN  homeostatic  proliferating
state_true
DAM          1665     1           30             10
IFN             0  1010           75              7
homeostatic    11    13        13142            156
accuracy (excluding contaminants): 0.981
```

**How to read it**

Against the planted truth, annotation accuracy is 98.1% (excluding contaminants). One weakness is visible: 156 homeostatic cells were labelled "proliferating". Real data have no truth table, so always check labels against the marker dot plot.

| If you see... | What it means and what to do |
|---|---|
| Two clusters get the same label | Normal (e.g. several homeostatic clusters). Merge by label rather than by cluster. |
| A cluster is labelled "proliferating" but has no Mki67 signal | Argmax picks the highest of low scores. Add a minimum score (e.g. label as "unassigned" if max < 0.3) and inspect manually. |

### Cell 5 - Write labels back to each cohort and draw the UMAP

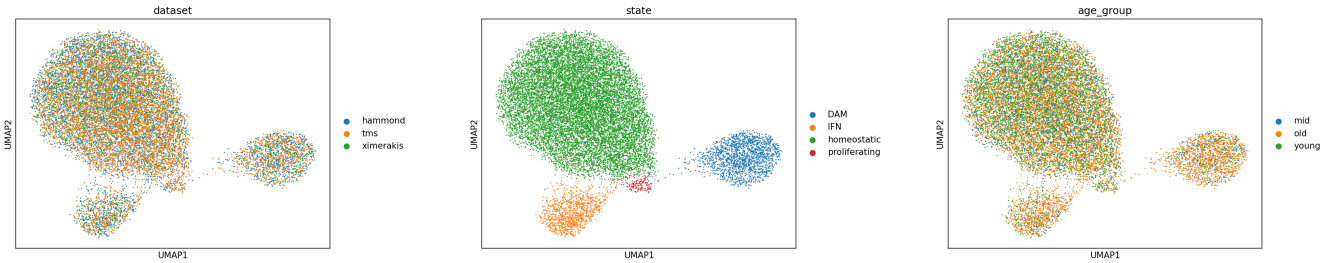
States are saved into each cohort file so Notebook 07 can stratify by state. The three UMAP panels show cohort mixing, states and age groups.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
for a, n in zip(ads, names):
    a.obs['state'] = cat.obs.loc[a.obs_names, 'state'].values
    a.write(f'../data/processed/{n}_microglia.h5ad')
print(cat.obs.groupby(['dataset', 'age_group', 'state'],
» observed=True).size().unstack(fill_value=0).to_string())
sc.pl.umap(cat, color=['dataset', 'state', 'age_group'], ncols=3, wspace=0.35, show=True)
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
state                DAM  IFN  homeostatic  proliferating
dataset   age_group
hammond   old        597  366         2822             39
          young      136   84         2313             29
tms       mid        133   80         1077             12
          old        355  232         1956             28
          young      119   67         2371             36
ximerakis old        261  148         1192             19
          young       75   47         1516             10
```




***Expected figure*** *(03_5): real output of this code on the synthetic benchmark.*

**How to read it**

Left panel: cohorts should overlap within each state. Middle: DAM and interferon-responsive cells form separate islands. Right: old animals contribute many more cells to those islands (see the table: hammond old 597 DAM cells vs 136 young).

| If you see... | What it means and what to do |
|---|---|
| KeyError when assigning states | Cell names must be unique across cohorts (they are, because Notebook 01 prefixes the sample ID). If duplicates exist, `an.concat(..., index_unique="-")` and map back accordingly. |

# Notebook 04 - Gene programs and expression-matched null sets

| Purpose | Builds the two programs of interest and the control programs with provenance, exclusivity, DAM/identity exclusion and a detectability filter; draws expression-matched random gene sets. Freeze and preregister before Notebook 05. |
|---|---|
| Needs | <cohort>_microglia.h5ad, dam_signature.csv |
| Produces | gene_sets/modules.csv, data/processed/null_sets.pkl |
| Time | 2 min |

## Cell 1 - List the pathway libraries available offline-free (needs internet)

gseapy downloads curated pathway libraries from Enrichr. Print which exist in your installed version and pick names from that list, because names change between releases.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import gseapy as gp, pandas as pd, numpy as np, itertools, re, pickle
import scanpy as sc, matplotlib.pyplot as plt
names = gp.get_library_name(organism='Mouse')
print([n for n in names if any(k in n for k in ['GO_', 'KEGG', 'Reactome', 'WikiPathway'])])
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
['GO_Biological_Process_2021', 'GO_Cellular_Component_2021', 'GO_Molecular_Function_2021',
 'KEGG_2019_Mouse', 'WikiPathways_2019_Mouse', ...]
(format example - the exact names differ by gseapy release; use whichever your list shows)
```

| If you see... | What it means and what to do |
|---|---|
| ConnectionError / timeout | Enrichr is temporarily unavailable or your network blocks it. Retry later; or download the gene sets once from https://www.gsea-msigdb.org/gsea/msigdb and read the .gmt file with `gp.read_gmt(path)`. |
| Mouse library missing a term you need | Use the human library and map to mouse symbols with the ortholog table (Notebook 09, Cell 1). |

## Cell 2 - Find relevant terms and their sizes

Decide, and write down in docs/, which database terms feed each program. A term with 600 genes is a catalogue, not a program; prefer specific terms or intersect large terms with curated seeds.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
lib_go = gp.get_library('GO_Biological_Process_2021', organism='Mouse')
lib_cc = gp.get_library('GO_Cellular_Component_2021', organism='Mouse')
for k in lib_go:
    if 'mitophagy' in k.lower():
        print(k, len(lib_go[k]))
for k in lib_cc:
    if 'lysosom' in k.lower():
        print(k, len(lib_cc[k]))
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
mitophagy (GO:0000423)                          29
regulation of mitophagy (GO:1901524)             18
positive regulation of mitophagy (GO:1901526)    12
lysosome (GO:0005764)                           421
lysosomal lumen (GO:0043202)                     93
lysosomal membrane (GO:0005765)                 201
(format example - sizes are illustrative; yours will differ)
```

| If you see... | What it means and what to do |
|---|---|
| KeyError: library not found | Name does not match the list printed in Cell 1 (a year suffix differs). Copy it exactly. |

## Cell 3 - Seed lists and context/negative-control programs

Hand-curated seeds for each program from the literature, extended later with database terms. The RIBOSOME control is built by pattern from the data. Note that Ulk1 is deliberately in both MITOPHAGY and AUTOPHAGY_GEN, and Ctsd/Lamp1/Ctsz will turn up in the DAM signature: Cell 4 handles both.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, itertools, re, pickle, scanpy as sc, matplotlib.pyplot as plt
seed = {
 'MITOPHAGY': ['Pink1','Prkn','Bnip3','Bnip3l','Fundc1','Optn','Calcoco2','Tbk1','Usp30','Mul1',
               'Fkbp8','Bcl2l13','Phb2','Stx17','Ulk1','Park7','Vps13c','Mfn2'],
 'LYSOSOME': ['Mcoln1','Ctsd','Ctsf','Gla','Gba','Gba1','Hexa','Neu1','Tpp1','Npc1','Npc2','Lamp1',
              'Lamp2','Atp6v1h','Atp6v0e1','Atp6v1c1','Sgsh','Gns','Galc','Naglu','Asah1','Lipa',
              'Ppt1','Cln3','Cln5','Gusb','Idua','Ids','Man2b1','Fuca1','Ctsa','Ctsz','Smpd1','Gaa'],
 'OXPHOS': ['Ndufa1','Ndufa9','Ndufb8','Ndufs1','Sdha','Sdhb','Uqcrc1','Uqcrc2','Cox4i1','Cox5a',
            'Cox6a1','Atp5f1a','Atp5f1b','Ndufb5','Ndufs3','Uqcrb','Cox7c','Atp5mc1'],
 'AUTOPHAGY_GEN': ['Atg5','Atg7','Atg12','Atg16l1','Becn1','Map1lc3b','Gabarap',
                   'Gabarapl1','Atg3','Atg4b','Ulk1'],
 'PROTEASOME': ['Psma%d' % i for i in range(1, 8)] + ['Psmb%d' % i for i in range(1, 8)] +
               ['Psmc%d' % i for i in range(1, 7)] + ['Psmd%d' % i for i in (1, 2, 3, 4, 6, 7, 8, 11,
               » 12, 13, 14)],
}
ref = sc.read_h5ad('../data/processed/hammond_microglia.h5ad')
seed['RIBOSOME'] = [g for g in ref.var_names if re.match(r'^Rp[ls]\d+[a]?$', g)]
identity = ['Hexb','Cx3cr1','P2ry12','Tmem119','Csf1r','Sall1','Fcrls',
            'Selplg','Siglech','Aif1','Tyrobp']
print({k: len(v) for k, v in seed.items()})
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
{'MITOPHAGY': 18, 'LYSOSOME': 34, 'OXPHOS': 18, 'AUTOPHAGY_GEN': 11, 'PROTEASOME': 31, 'RIBOSOME': 45}
```

| If you see... | What it means and what to do |
|---|---|
| Reference cohort file not found | Notebook 02 must have finished for `hammond` (any cohort works as the reference for the ribosome pattern). |
| RIBOSOME list is empty | Your symbols are not mouse-style (Rpl3). For human data use `^RP[LS]\d+A?$`. |

### Cell 4 - Enforce exclusivity; remove identity genes and DAM-overlap genes

A gene may belong to only one program. Microglial identity genes and genes in an independently derived DAM signature are removed from the two primary programs, so the correlation cannot simply reflect "disease state" or "microglia-ness".

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
dam = pd.read_csv('../data/metadata/dam_signature.csv')['gene'].tolist()   # from Keren-Shaul 2017
» supplement
dam_set = set(dam)
mods = {k: set(v) for k, v in seed.items()}
for a, b in itertools.combinations(mods, 2):
    ov = mods[a] & mods[b]
    if ov:
        print('overlap', a, '&', b, ':', sorted(ov))
multi = {g for g in set().union(*mods.values()) if sum(g in m for m in mods.values()) > 1}
for k in mods:
    mods[k] -= multi
    mods[k] -= set(identity)
for k in ['MITOPHAGY', 'LYSOSOME']:
    removed = sorted(mods[k] & dam_set)
    print(k, 'removed as DAM-overlap:', removed)
    mods[k] -= dam_set
print({k: len(v) for k, v in mods.items()})
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
overlap MITOPHAGY & AUTOPHAGY_GEN : ['Ulk1']
MITOPHAGY removed as DAM-overlap: []
LYSOSOME removed as DAM-overlap: ['Ctsd', 'Ctsz', 'Lamp1']
{'MITOPHAGY': 17, 'LYSOSOME': 31, 'OXPHOS': 18, 'AUTOPHAGY_GEN': 10, 'PROTEASOME': 31, 'RIBOSOME': 45}
```

#### How to read it

Ulk1 sits in two programs, so it is removed from both. Ctsd, Ctsz and Lamp1 are removed from LYSOSOME because they are DAM signature genes: otherwise the lysosome score would partly measure DAM state.

| If you see... | What it means and what to do |
|---|---|
| FileNotFoundError: dam_signature.csv | Create it from the Keren-Shaul 2017 supplementary table (DAM vs homeostatic genes, top 50-100 up-regulated): a CSV with one column named `gene`. |
| Too many genes removed from LYSOSOME | That is expected: many lysosomal genes are DAM genes. If fewer than 12 remain, broaden the lysosomal program with GO/KEGG terms rather than dropping the DAM rule. |

### Cell 5 - Detectability filter and freeze modules.csv

A gene is kept only if it is detected in at least 1% of cells in at least 80% of cohorts. This stops technology (sparse vs deep data) from deciding which genes enter the score. The resulting file is frozen and registered BEFORE age effects are examined.

*Code*

```
cohorts = ['hammond', 'ximerakis', 'tms']
det = {}
for c in cohorts:
    a = sc.read_h5ad(f'../data/processed/{c}_microglia.h5ad')
    det[c] = pd.Series(np.asarray((a.layers['counts'] > 0).mean(axis=0)).ravel(), index=a.var_names)
keep = {k: [] for k in mods}
for k, genes in mods.items():
    for g in sorted(genes):
        n_ok = sum((g in det[c].index and det[c][g] >= 0.01) for c in cohorts)
        if n_ok >= 0.8 * len(cohorts):
            keep[k].append(g)
    print('%-14s %3d candidate genes -> %3d detectable' % (k, len(genes), len(keep[k])))
rows = [(k, g) for k, v in keep.items() for g in v]
pd.DataFrame(rows, columns=['module', 'gene_mouse']).to_csv('../gene_sets/modules.csv', index=False)
print('median detection (%% of cells), MITOPHAGY: %.1f  LYSOSOME: %.1f' % (
    100 * np.median([det['hammond'][g] for g in keep['MITOPHAGY']]),
    100 * np.median([det['hammond'][g] for g in keep['LYSOSOME']])))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
MITOPHAGY       17 candidate genes ->  17 detectable
LYSOSOME        31 candidate genes ->  30 detectable
OXPHOS          18 candidate genes ->  18 detectable
AUTOPHAGY_GEN   10 candidate genes ->  10 detectable
PROTEASOME      31 candidate genes ->  31 detectable
RIBOSOME        45 candidate genes ->  45 detectable
median detection (% of cells), MITOPHAGY: 12.6  LYSOSOME: 44.6
```

**How to read it**

Median detection: MITOPHAGY genes in 12.6% of cells, LYSOSOME genes in 44.6%. The mitophagy score rests on sparse data, which is why the pseudo-cell and covariate checks in Notebook 06 matter. `Gba` was dropped (absent), `Gba1` kept.

| If you see... | What it means and what to do |
|---|---|
| MITOPHAGY ends with fewer than 12 genes | Do not lower the 1% rule. Add database mitophagy terms (GO:0000423, Reactome/KEGG mitophagy) and re-run Cells 3-5. |
| Symbol present in the list but missing in one cohort ("Gba") | Old vs new symbol (Gba / Gba1). Keep both in the seed; the detectability filter drops whichever does not exist. |

## Cell 6 - Expression-matched random gene sets (the null) and a sanity figure

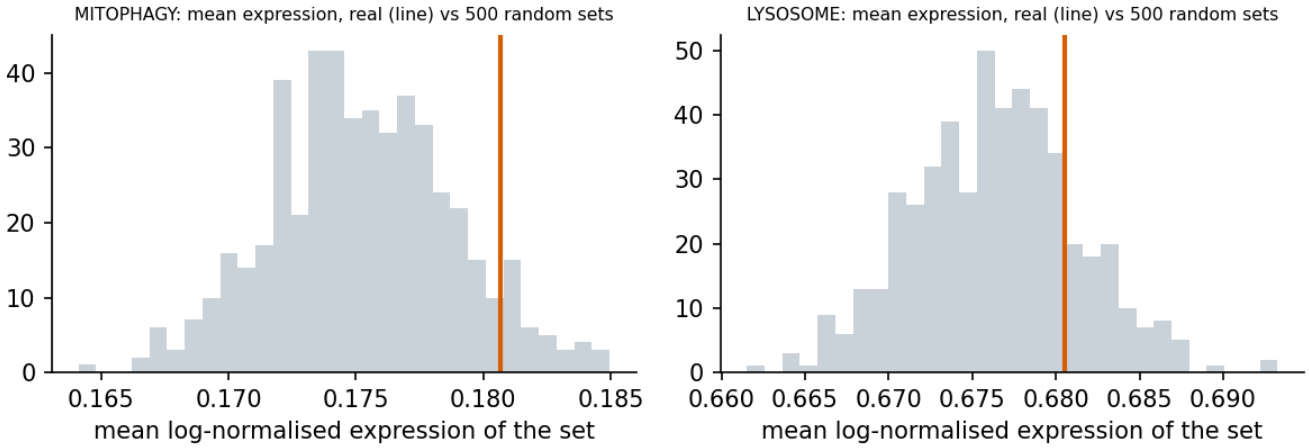
For each real program, 500 random gene sets of the same size are drawn from genes with matching average expression (20 expression bins), excluding every program and DAM/identity gene. They define what "any two gene sets" would do. The figure checks that the matching worked.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
allmod = set(mods_df.gene_mouse) | set(identity) | dam_set
null_mito = matched_random_sets(ref, modules['MITOPHAGY'], n_sets=500, seed=1, exclude=allmod)
null_lyso = matched_random_sets(ref, modules['LYSOSOME'], n_sets=500, seed=2, exclude=allmod)
pickle.dump({'mito': null_mito, 'lyso': null_lyso}, open('../data/processed/null_sets.pkl', 'wb'))
me = pd.Series(np.asarray(ref.X.mean(axis=0)).ravel(), index=ref.var_names)
fig, ax = plt.subplots(1, 2, figsize=(8, 2.9))
for a_, (nm, real, nul) in zip(ax, [('MITOPHAGY', modules['MITOPHAGY'], null_mito),
                                    ('LYSOSOME', modules['LYSOSOME'], null_lyso)]):
    a_.hist([me[s].mean() for s in nul], bins=30, color='#c9d1d9')
    a_.axvline(me[real].mean(), color='#D55E00', lw=2)
    a_.set_title(nm + ': mean expression, real (line) vs 500 random sets', fontsize=7.5)
    a_.set_xlabel('mean log-normalised expression of the set'); a_.spines[['top',
    » 'right']].set_visible(False)
plt.tight_layout(); plt.show()
print(len(null_mito), 'null sets per program; example random set:', null_mito[0][:6])
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
500 null sets per program; example random set: ['Gene1365', 'Gene1277', 'Gene2144', 'Gene2837', 'Gene0039', '...
```




***Expected figure*** *(04_6): real output of this code on the synthetic benchmark.*

**How to read it**

The red line (real set) should fall inside the grey histogram (500 random sets). If it lies far outside, matching failed and the null is not comparable.

| If you see... | What it means and what to do |
|---|---|
| Real set mean lies far outside the random distribution | Matching failed (too few genes per bin or very skewed expression). Lower `n_bins` (e.g. 10) or check for genes with zero expression in the reference cohort. |
| KeyError in matched_random_sets | A seed gene is absent from `ref.var_names`; the function skips absent genes, but check that the module file is not empty. |

## Notebook 05 - Primary result: animal-level coupling and the matched null

| Purpose | The primary endpoint: animal-level coupling, old versus young, per cohort and pooled, with the matched-null test. Produces Figure 2. |
|---|---|
| Needs | modules.csv, null_sets.pkl, cohorts |
| Produces | tables/05_*, figures/fig2_primary_result.pdf |
| Time | 3-6 min |

### Cell 1 - Imports, frozen programs, null sets

Loads the frozen gene programs and the 500 expression-matched random sets saved in Notebook 04. Nothing is re-chosen here: the endpoint was fixed in the preregistration.

*Code*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, scanpy as sc, pickle, matplotlib.pyplot as plt
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
null = pickle.load(open('../data/processed/null_sets.pkl', 'rb'))
cohorts = ['hammond', 'ximerakis', 'tms']
print({k: len(v) for k, v in modules.items()})
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
{'AUTOPHAGY_GEN': 10, 'LYSOSOME': 30, 'MITOPHAGY': 17, 'OXPHOS': 18, 'PROTEASOME': 31, 'RIBOSOME': 45}
```

| If you see... | What it means and what to do |
|---|---|
| FileNotFoundError: null_sets.pkl | Notebook 04 Cell 6 has not been run. |

### Cell 2 - Animal-level coupling and permutation test, per cohort

For each cohort: score every cell, compute each animal's partial Spearman correlation between the MITOPHAGY and LYSOSOME scores (adjusting for depth), compare old with young at the animal level (Fisher z), run an exact-style permutation test over animal labels, and compute the same old-minus-young statistic for 200 random gene-set pairs.

*Code*

```
res, acs, nulls = [], [], {}
for c in cohorts:
    a = load_wt(c)
    ac, eff = cohort_effect(a, modules['MITOPHAGY'], modules['LYSOSOME'])
    ac['dataset'] = c; acs.append(ac)
    if eff is None:
        print(c, 'skipped: fewer than 2 animals in a group'); continue
    diff, se, n_o, n_y = eff
    _, p_perm = perm_diff(ac.loc[ac.age_group == 'old', 'z'], ac.loc[ac.age_group == 'young', 'z'])
    res.append({'dataset': c, 'diff_z': diff, 'se': se, 'n_old': n_o, 'n_young': n_y, 'p_perm': p_perm})
    groups = ac[['animal_id', 'age_group']]
    nulls[c] = null_delta(a, a.obs[['animal_id', 'log_counts', 'n_genes']],
                          null['mito'][:200], null['lyso'][:200], groups)
res = pd.DataFrame(res); ac_all = pd.concat(acs, ignore_index=True)
ac_all.to_csv('../results/tables/05_animal_coupling.csv', index=False)
print(res.round(3).to_string(index=False))
print(ac_all.groupby(['dataset', 'age_group']).rho.agg(['mean', 'min', 'max']).round(3).to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
  dataset  diff_z    se  n_old  n_young  p_perm
  hammond  -0.167 0.027      4        4   0.030
ximerakis  -0.050 0.061      3        3   0.598
      tms  -0.109 0.038      4        4   0.057
                      mean    min    max
dataset   age_group
hammond   old        0.140  0.127  0.152
          young      0.298  0.230  0.333
tms       old        0.193  0.138  0.244
          young      0.295  0.230  0.358
ximerakis old        0.192  0.123  0.243
          young      0.240  0.184  0.328
```

#### How to read it

Same sign in all three cohorts. Hammond (-0.167) has p_perm 0.030, the smallest possible with 4 v 4 animals (0.029); ximerakis is the weakest (-0.050, SE 0.061) because its planted effect was smaller and it has only 3 v 3 samples. Observed animal-level rho (young 0.23-0.36, old 0.12-0.24) is lower than the planted latent values because sparsity attenuates correlations; compare groups, not absolute values.

| If you see... | What it means and what to do |
|---|---|
| Runs for a long time | The matched null (200 sets x cohorts) dominates. Use `[:50]` while testing; use 200-500 for the final analysis. |
| `eff is None` for a cohort | Fewer than 2 animals per group passed `min_cells=100`. Check cells per animal (Notebook 02, Cell 10). |
| p_perm is exactly 0.1 (or 0.0286) | That is the smallest p possible with 3v3 (0.1) or 4v4 (0.029) animals; it is a limit of the design, not a bug (Section 1.4). |
| KeyError: sex or state | The columns were not carried through Notebook 02/03. Re-run those notebooks. |

### Cell 3 - Random-effects meta-analysis and the matched-null test

Cohort effects are combined with DerSimonian-Laird random effects (the "random effect" row). The matched-null test asks how often random gene-set pairs, analysed identically, produce an old-minus-young change as large as the real one (empirical p, gate G2).

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
m = meta_random(res.diff_z, res.se, res.dataset.tolist())
sf = m.summary_frame()
print(sf.loc[res.dataset.tolist() + ['random effect'], ['eff', 'sd_eff', 'ci_low',
» 'ci_upp']].round(3).to_string())
print('I2 = %.1f%%   tau2 = %.4f' % (100 * float(np.ravel(m.i2)[0]) if np.ndim(m.i2) else 100 * m.i2,
» float(np.ravel(m.tau2)[0]) if np.ndim(m.tau2) else m.tau2))
sf.to_csv('../results/tables/05_meta_summary.csv')

w = 1.0 / res.se.values ** 2
obs = float((w * res.diff_z.values).sum() / w.sum())
nm = np.array([nulls[c] for c in res.dataset])                  # cohorts x null sets
null_pooled = (w[:, None] * nm).sum(0) / w.sum()
p_null = (np.sum(np.abs(null_pooled) >= abs(obs)) + 1) / (len(null_pooled) + 1)
print('pooled observed change in z = %.3f' % obs)
print('null: mean %.3f, 2.5th-97.5th percentile [%.3f,
» %.3f]' % ((null_pooled.mean(),) + tuple(np.percentile(null_pooled, [2.5, 97.5]))))
print('empirical p vs matched random pairs = %.4f   (rho scale: %.3f)' % (p_null, np.tanh(obs)))
np.save('../results/tables/05_null_pooled.npy', null_pooled)
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
                eff  sd_eff  ci_low  ci_upp
hammond      -0.167   0.027  -0.220  -0.114
ximerakis    -0.050   0.061  -0.169   0.069
tms          -0.109   0.038  -0.184  -0.034
random effect -0.124   0.031  -0.186  -0.063
I2 = 47.6%   tau2 = 0.0014
pooled observed change in z = -0.136
null: mean -0.001, 2.5th-97.5th percentile [-0.039, 0.036]
empirical p vs matched random pairs = 0.0050   (rho scale: -0.135)
```

### How to read it

Random-effects pooled change -0.124 (95% CI -0.186 to -0.063); heterogeneity I-squared 47.6%. Observed pooled change (-0.136) lies far outside the random-pair range (-0.039 to +0.036): empirical p = 0.005. With 200 null sets the smallest possible p is 1/201 = 0.005, so use 1,000+ sets for the final analysis. Gates: G2 passes; G3 passes (3 of 3 cohorts negative, CI excludes 0).

| If you see... | What it means and what to do |
|---|---|
| KeyError in `sf.loc[...]` | Row labels differ between statsmodels versions. Print `sf.index` and select by what you see. |
| p_null is large although the meta-analysis is "significant" | Meaning: the change is not specific. Random gene sets decouple just as much, so depth/state effects are the likely explanation (Notebook 06 will confirm). |
| I2 very high (> 75%) | Cohorts disagree. Look at which cohort differs (platform, region, sex) before pooling; report heterogeneity. |

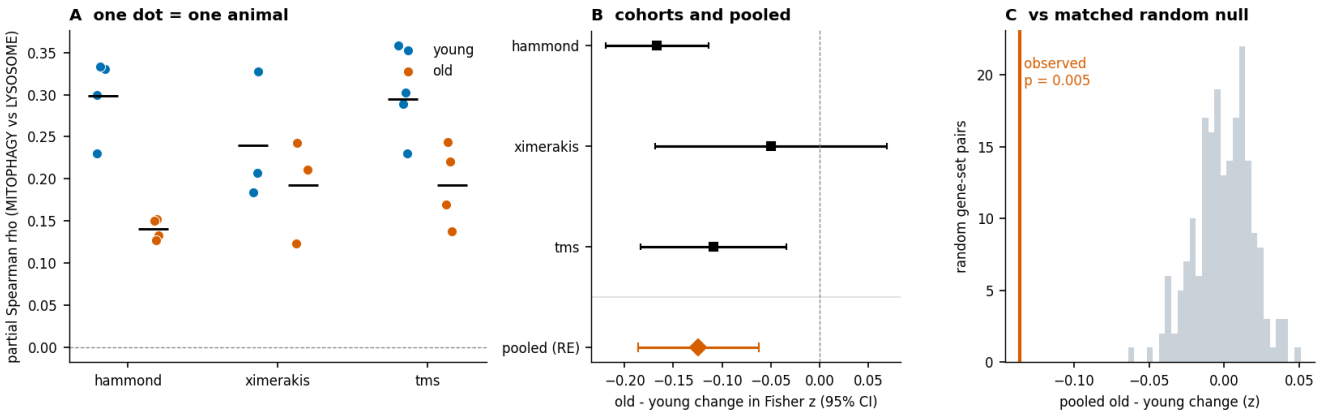
## Cell 4 - Figure 2: the primary result

Panel A: every animal is a dot (n = animals, not cells). Panel B: forest plot of the old-minus-young change with 95% CI per cohort and pooled. Panel C: the pooled change against the null distribution from random expression-matched gene-set pairs.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
okabe = {'young': '#0072B2', 'old': '#D55E00'}
plt.rcParams.update({'font.size': 8, 'axes.linewidth': .7, 'pdf.fonttype': 42})
fig, ax = plt.subplots(1, 3, figsize=(11, 3.5), gridspec_kw={'width_ratios': [1.35, 1, 1]})
rng_ = np.random.default_rng(0)
for i, c in enumerate(cohorts):
    for j, g in enumerate(['young', 'old']):
        v = ac_all[(ac_all.dataset == c) & (ac_all.age_group == g)].rho
        ax[0].scatter(i * 3 + j + rng_.uniform(-.13, .13, len(v)), v, s=34, color=okabe[g],
                      edgecolor='white', lw=.5, label=g if i == 0 else None, zorder=3)
        ax[0].hlines(v.mean(), i * 3 + j - .3, i * 3 + j + .3, color='black', lw=1.4, zorder=4)
ax[0].set_xticks([i * 3 + .5 for i in range(3)]); ax[0].set_xticklabels(cohorts)
ax[0].axhline(0, color='grey', lw=.6, ls='--'); ax[0].legend(frameon=False, loc='upper right')
ax[0].set_ylabel('partial Spearman rho (MITOPHAGY vs LYSOSOME)'); ax[0].set_title('A  one dot = one
» animal', loc='left', fontweight='bold')
pool = sf.loc['random effect']
rows = [(c, r.diff_z, r.se) for c, r in zip(res.dataset, res.itertuples())]
for k, (c, e, s) in enumerate(rows):
    ax[1].errorbar(e, 3 - k, xerr=1.96 * s, fmt='s', color='black', capsize=2, ms=5)
ax[1].errorbar(pool['eff'], 0, xerr=[[pool['eff'] - pool['ci_low']], [pool['ci_upp'] - pool['eff']]],
               fmt='D', color='#D55E00', capsize=3, ms=7)
ax[1].axhline(0.5, color='#dddddd', lw=.8)
ax[1].set_yticks([3, 2, 1, 0]); ax[1].set_yticklabels(list(res.dataset) + ['pooled (RE)'])
ax[1].axvline(0, color='grey', lw=.6, ls='--'); ax[1].set_xlabel('old - young change in Fisher z (95%
» CI)')
ax[1].set_title('B  cohorts and pooled', loc='left', fontweight='bold')
ax[2].hist(null_pooled, bins=28, color='#c9d1d9'); ax[2].axvline(obs, color='#D55E00', lw=2)
ax[2].set_xlabel('pooled old - young change (z)'); ax[2].set_ylabel('random gene-set pairs')
ax[2].text(obs, ax[2].get_ylim()[1] * .92, ' observed\n p = %.3f' % p_null, color='#D55E00', ha='left',
» va='top')
ax[2].set_title('C  vs matched random null', loc='left', fontweight='bold')
for a_ in ax: a_.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig2_primary_result.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(05_4): real output of this code on the synthetic benchmark.*

## How to read it

Panel A: young animals sit above old in every cohort; the weak ximerakis cohort overlaps. Panel B: all three intervals lean negative; ximerakis crosses zero but the pooled interval does not. Panel C: the observed change is far outside the null distribution: the change is specific to this pair of programs, not a generic property of gene-set pairs.

| If you see... | What it means and what to do |
|---|---|
| Panel B shows only one cohort | Cohorts with fewer than 2 animals per group were skipped in Cell 2. |
| Font warnings about Arial | Harmless; Matplotlib falls back to DejaVu Sans. Install Arial or Helvetica for journal-style figures. |

## Cell 5 - Mixed model on all cells (robustness check, animal as a random effect)

Uses every cell but models animal as a random effect, with a random intercept and slope: lyso ~ mito * old. The mito:old coefficient is the change in the mitophagy-lysosome slope in old animals. This is supporting evidence, not the primary test.

*Code*

```
rows = []
for c in cohorts:
    a = load_wt(c)
    sco = module_scores(a, {'mito': modules['MITOPHAGY'], 'lyso': modules['LYSOSOME']}, min_genes=1)
    d = a.obs[['animal_id', 'age_group', 'log_counts', 'n_genes']].join(sco)
    d['old'] = (d.age_group == 'old').astype(int)
    d = d.groupby('animal_id', observed=True).filter(lambda g: len(g) >= 100)
    d = d.sample(min(len(d), 30000), random_state=0)
    fit = mixed_interaction(d)
    ci = fit.conf_int().loc['mito:old']
    rows.append({'dataset': c, 'coef_mito:old': fit.params['mito:old'], 'ci_low': ci[0], 'ci_high': ci[1],
                 'p': fit.pvalues['mito:old']})
print(pd.DataFrame(rows).round(4).to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
  dataset  coef_mito:old  ci_low  ci_high      p
  hammond        -0.1933 -0.3660  -0.0206 0.0283
ximerakis        -0.0297 -0.2247   0.1653 0.7653
      tms        -0.1243 -0.2850   0.0364 0.1296
```

### How to read it

Mixed-model interaction (cells used, animal modelled): hammond -0.19 (CI -0.37 to -0.02), tms -0.12 (CI -0.29 to +0.04), ximerakis -0.03 (CI -0.22 to +0.17). Same direction as the primary result, but wide intervals: supporting evidence only.

| If you see... | What it means and what to do |
|---|---|
| ConvergenceWarning / singular matrix | Drop the random slope: `re_formula=None` in `mixed_interaction`, or standardise the scores. With few animals (< 5 per group) the random-slope variance is poorly identified. |
| p-values look extreme (1e-30) | Expected with thousands of cells, and the reason this is not the primary test. Report the coefficient and CI, not the p-value. |

## Cell 6 - Sensitivity: sex and cohort as covariates (animal level)

Checks that the age effect is not an artefact of sex imbalance or cohort differences, using an animal-level regression with robust standard errors.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import statsmodels.formula.api as smf
m1 = smf.ols('z ~ C(age_group, Treatment("young")) + C(sex) + C(dataset)',
» data=ac_all).fit(cov_type='HC3')
tab = pd.DataFrame({'coef': m1.params, 'se': m1.bse, 'p': m1.pvalues}).round(3)
print(tab.to_string())
print('animals:', len(ac_all), '| old-young coefficient (z):', round(m1.params.iloc[1], 3))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
                                     coef     se      p
Intercept                           0.293  0.032  0.000
C(age_group, Treatment("young"))[T.old] -0.114  0.028  0.000
C(sex)[T.M]                        -0.022  0.027  0.427
C(dataset)[T.tms]                   0.025  0.029  0.387
C(dataset)[T.ximerakis]            -0.001  0.039  0.983
animals: 22 | old-young coefficient (z): -0.114
```

### How to read it

Adjusted for sex and cohort the animal-level age effect is -0.114 (p < 0.001, 22 animals); sex is not significant. The estimate matches the pooled meta-analytic value.

| If you see... | What it means and what to do |
|---|---|
| Singular matrix / NaN SEs | A covariate level has a single animal (e.g. one female in a cohort). Merge levels or drop `C(sex)`. |

## Notebook 06 - Specificity and sensitivity analyses

| Purpose | Specificity and sensitivity: negative-control pairs, alternative scoring, covariates, down-sampling, pseudo-cells and leave-one-gene-out. Produces Figure 3. |
|---|---|
| Needs | as Notebook 05 |
| Produces | tables/06_*, figures/fig3*.pdf |
| Time | 5-8 min |

### Cell 1 - Imports and a pooling helper

`pooled_effect` (already in utils.py) returns the inverse-variance pooled old-minus-young change in Fisher z for any pair of gene lists, across cohorts. Every analysis in this notebook is one call to it with a different pair of lists or options.

*Code*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, scanpy as sc, matplotlib.pyplot as plt
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
cohorts = ['hammond', 'ximerakis', 'tms']
e0, s0, parts0 = pooled_effect(cohorts, modules['MITOPHAGY'], modules['LYSOSOME'])
print('baseline pooled change in z = %.3f (SE %.3f)' % (e0, s0))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
baseline pooled change in z = -0.136 (SE 0.021)
```

| If you see... | What it means and what to do |
|---|---|
| Pooled effect here differs from Notebook 05 | Notebook 05 reports the random-effects estimate; this notebook uses the fixed-effect (inverse-variance) pooled value. Both are valid; state which you use. With heterogeneous cohorts they differ. |

### Cell 2 - Specificity: negative-control and context pairs

If coupling is specifically changed between mitophagy and lysosome programs, then pairing the same programs with negative controls (proteasome, ribosome) should show little change. Context pairs (OXPHOS-lysosome, general autophagy-lysosome) show whether the effect is mitochondrial function or autophagy in general.

*Code*

```
pairs = {'MITOPHAGY-LYSOSOME': ('MITOPHAGY', 'LYSOSOME'),
         'MITOPHAGY-PROTEASOME': ('MITOPHAGY', 'PROTEASOME'),
         'MITOPHAGY-RIBOSOME': ('MITOPHAGY', 'RIBOSOME'),
         'PROTEASOME-LYSOSOME': ('PROTEASOME', 'LYSOSOME'),
         'OXPHOS-LYSOSOME': ('OXPHOS', 'LYSOSOME'),
         'AUTOPHAGY_GEN-LYSOSOME': ('AUTOPHAGY_GEN', 'LYSOSOME')}
rows = []
for name, (p, q) in pairs.items():
    e, s, _ = pooled_effect(cohorts, modules[p], modules[q])
    rows.append({'pair': name, 'pooled_delta_z': e, 'se': s})
spec = pd.DataFrame(rows)
spec.to_csv('../results/tables/06_specificity_pairs.csv', index=False)
print(spec.round(3).to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
                 pair  pooled_delta_z    se
    MITOPHAGY-LYSOSOME          -0.136 0.021
  MITOPHAGY-PROTEASOME          -0.012 0.017
    MITOPHAGY-RIBOSOME          -0.010 0.011
   PROTEASOME-LYSOSOME           0.020 0.011
       OXPHOS-LYSOSOME           0.050 0.016
AUTOPHAGY_GEN-LYSOSOME          -0.003 0.017
```

**How to read it**

The pair of interest moved by -0.136; the controls MITOPHAGY-PROTEASOME (-0.012), MITOPHAGY-RIBOSOME (-0.010), AUTOPHAGY_GEN-LYSOSOME (-0.003) hardly moved. OXPHOS-LYSOSOME rose slightly (+0.050, SE 0.016): not everything is flat and you should report that too.

| If you see... | What it means and what to do |
|---|---|
| KeyError: PROTEASOME | The program was dropped by the detectability filter or never defined (Notebook 04). Check `modules.keys()`. |
| Control pairs move as much as the main pair | Then the change is not specific. Report it as a global coordination change (Part 10) rather than a pathway-specific one. |

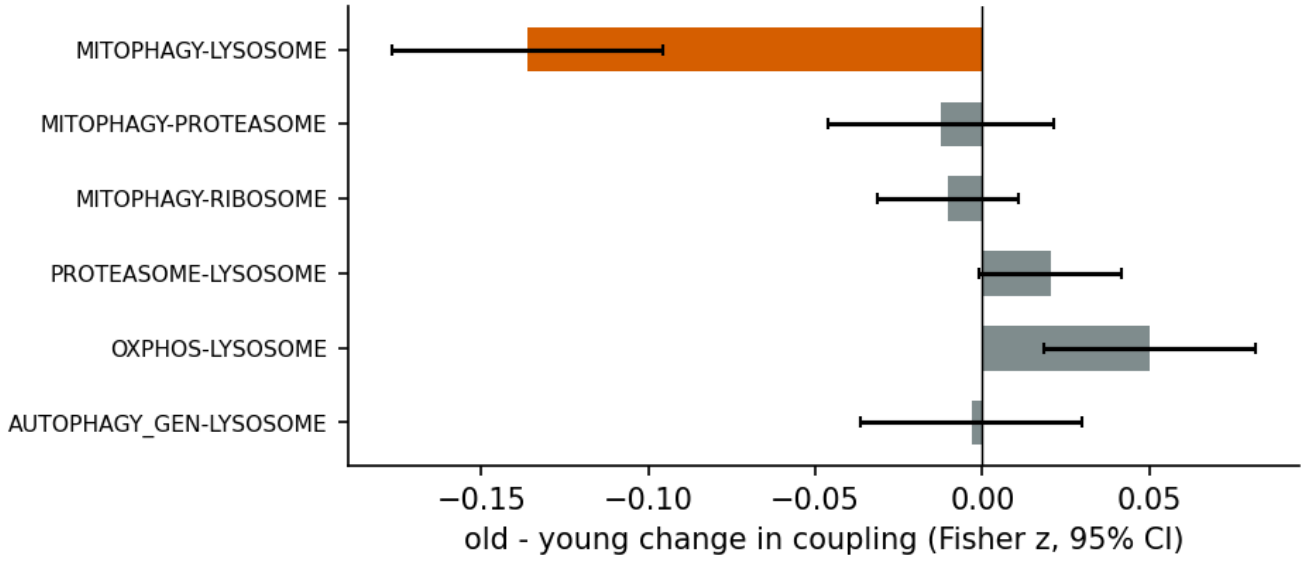
### Cell 3 - Figure: specificity

Pooled old-minus-young change (z) with 95% CI for each pair. The highlighted bar is the pair of interest; the others are controls.

*Code*

```
fig, ax = plt.subplots(figsize=(6.4, 2.9))
y = np.arange(len(spec))[::-1]
cols = ['#D55E00' if n == 'MITOPHAGY-LYSOSOME' else '#7f8c8d' for n in spec.pair]
ax.barh(y, spec.pooled_delta_z, xerr=1.96 * spec.se, color=cols, height=.6, capsize=2)
ax.set_yticks(y); ax.set_yticklabels(spec.pair, fontsize=7.5)
ax.axvline(0, color='black', lw=.7); ax.set_xlabel('old - young change in coupling (Fisher z, 95% CI)')
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig3a_specificity.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(06_3): real output of this code on the synthetic benchmark.*

**How to read it**

The orange bar is the pair of interest; its interval is far from zero and from the controls. A pass looks like this: one bar standing out, the rest around zero.

## Cell 4 - Alternative scoring, covariate, down-sampling

Three robustness checks: (a) scanpy's `score_genes` instead of mean-z; (b) mitochondrial read fraction added as a covariate; (c) every animal down-sampled to 400 cells so cell number cannot drive the result.

*Code*

```
def eff_scanpy_scoring(c):
    a = load_wt(c).copy()
    sc.tl.score_genes(a, modules['MITOPHAGY'], score_name='mito', ctrl_size=50, random_state=0)
    sc.tl.score_genes(a, modules['LYSOSOME'], score_name='lyso', ctrl_size=50, random_state=0)
    d = a.obs[['animal_id', 'age_group', 'log_counts', 'n_genes', 'mito', 'lyso']]
    ac = animal_coupling(d, 'mito', 'lyso', min_cells=100).merge(
        d[['animal_id', 'age_group']].drop_duplicates(), on='animal_id')
    return group_effect(ac)[:2]

def eff_downsample(c, n=400, seed=0):
    a = load_wt(c)
    keep = a.obs.groupby('animal_id', observed=True).sample(n=n, random_state=seed).index
    ac, eff = cohort_effect(a[keep].copy(), modules['MITOPHAGY'], modules['LYSOSOME'], min_cells=n)
    return eff[:2]

def ivw(effs):
    e = np.array([x[0] for x in effs]); s = np.array([x[1] for x in effs]); w = 1 / s ** 2
    return float((w * e).sum() / w.sum()), float(np.sqrt(1 / w.sum()))

sens = {'baseline (mean-z score)': (e0, s0)}
sens['+ mitochondrial % covariate'] = pooled_effect(cohorts, modules['MITOPHAGY'], modules['LYSOSOME'],
                                                    covars=('log_counts', 'n_genes', 'pct_counts_mt'))[:2]
sens['scanpy score_genes'] = ivw([eff_scanpy_scoring(c) for c in cohorts])
sens['down-sampled to 400 cells'] = ivw([eff_downsample(c) for c in cohorts])
for k, (e, s) in sens.items():
    print('%-30s %7.3f (SE %.3f)' % (k, e, s))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
baseline (mean-z score)         -0.136 (SE 0.021)
+ mitochondrial % covariate     -0.136 (SE 0.021)
scanpy score_genes              -0.126 (SE 0.019)
down-sampled to 400 cells       -0.129 (SE 0.026)
```

### How to read it

All alternatives agree: baseline -0.136, with mitochondrial-percentage covariate -0.136, scanpy scoring -0.126, down-sampled to 400 cells -0.129.

| If you see... | What it means and what to do |
|---|---|
| score_genes: "ValueError: could not convert string to float" | A gene list contains an absent gene in an unexpected way; filter lists to `a.var_names` first. |
| Down-sampling fails with "Cannot take a larger sample" | An animal has fewer than 400 cells; lower `n`. |

## Cell 5 - Pseudo-cell pooling inside one state (is it just dropout noise?)

Random groups of 8 homeostatic cells from the same animal are averaged, which removes much of the dropout noise. IMPORTANT: pooling must happen within a state. In our first attempt we pooled cells of all states together and the result flipped (pooled change -0.006): averaging cells makes the variation between pools dominated by how many DAM cells each pool happens to contain, so composition manufactures coupling and old, DAM-rich animals looked MORE coupled. Pooling within state removes that artefact.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
def pool_scores(d, k=8, seed=0):
    rng = np.random.default_rng(seed); out = []
    for (an_, st), g in d.groupby(['animal_id', 'state'],
    » observed=True):      # pool within animal AND state
        idx = rng.permutation(len(g)); n = (len(g) // k) * k
        for chunk in idx[:n].reshape(-1, k):
            s = g.iloc[chunk]
            out.append({'animal_id': an_, 'age_group': s.age_group.iloc[0],
                        'mito': s.mito.mean(), 'lyso': s.lyso.mean(),
                        'log_counts': s.log_counts.mean(), 'n_genes': s.n_genes.mean()})
    return pd.DataFrame(out)

effs, rho_tab = [], []
for c in cohorts:
    a = load_wt(c)
    sco = module_scores(a, {'mito': modules['MITOPHAGY'], 'lyso': modules['LYSOSOME']}, min_genes=1)
    d = a.obs[['animal_id', 'age_group', 'state', 'log_counts', 'n_genes']].join(sco)
    d = d[d.state == 'homeostatic']
    pc = pool_scores(d)
    ac = animal_coupling(pc, 'mito', 'lyso', min_cells=10).merge(
        pc[['animal_id', 'age_group']].drop_duplicates(), on='animal_id')
    effs.append(group_effect(ac)[:2])
    rho_tab.append(ac.groupby('age_group').rho.mean().rename(c))
sens['pseudo-cells (8 homeostatic cells)'] = ivw(effs)
print(pd.DataFrame(rho_tab).round(3).to_string())
print('pooled change in z: %.3f (SE %.3f)' % sens['pseudo-cells (8 homeostatic cells)'])
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
age_group    old  young
hammond    0.129  0.318
ximerakis  0.200  0.171
tms        0.065  0.354
pooled change in z: -0.208 (SE 0.060)
```

### How to read it

Pooling 8 homeostatic cells gives -0.208 (SE 0.060), same sign and larger than at single-cell level. The box above explains why pooling across ALL states flipped the sign in our first attempt: a composition artefact manufactured by averaging.

| If you see... | What it means and what to do |
|---|---|
| Effect changes sign when pooling across all states | Composition artefact (see the text above). Pool within state, or restrict to one state, and report k as a sensitivity parameter (try 5, 8, 15). |
| Estimates differ a lot between k = 5, 8, 15 | Fewer, larger pools mean fewer data points per animal and noisier correlations. Report the range; do not choose k after seeing the result. |

## Cell 6 - Leave-one-gene-out (is one gene driving it?)

Each gene is removed in turn from MITOPHAGY or LYSOSOME, and the pooled change is recomputed. The shift from baseline is the gene's influence; this table also feeds the evidence rubric in Notebook 12.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
loo = []
for prog in ['MITOPHAGY', 'LYSOSOME']:
    for g in modules[prog]:
        m1 = [x for x in modules['MITOPHAGY'] if not (prog == 'MITOPHAGY' and x == g)]
        l1 = [x for x in modules['LYSOSOME'] if not (prog == 'LYSOSOME' and x == g)]
        e, s, _ = pooled_effect(cohorts, m1, l1)
        loo.append({'program': prog, 'gene_removed': g, 'delta_z': e, 'shift': e - e0})
loo = pd.DataFrame(loo).sort_values('shift')
loo.to_csv('../results/tables/06_leave_one_gene_out.csv', index=False)
print('range of pooled delta_z after removing any single gene: [%.3f, %.3f]' % (loo.delta_z.min(),
» loo.delta_z.max()))
print(loo.head(4).round(3).to_string(index=False)); print(loo.tail(4).round(3).to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
range of pooled delta_z after removing any single gene: [-0.142, -0.121]
  program gene_removed  delta_z  shift
MITOPHAGY         Mul1   -0.142 -0.006
 LYSOSOME         Gba1   -0.141 -0.005
MITOPHAGY        Park7   -0.140 -0.004
 LYSOSOME          Gns   -0.139 -0.003
  program gene_removed  delta_z  shift
MITOPHAGY       Vps13c   -0.127  0.009
MITOPHAGY         Optn   -0.126  0.011
MITOPHAGY       Fundc1   -0.124  0.013
MITOPHAGY       Bnip3l   -0.121  0.015
```

**How to read it**

Removing any single gene leaves the pooled change between -0.142 and -0.121. No single gene drives the result.

| If you see... | What it means and what to do |
|---|---|
| Takes > 10 minutes | Each iteration recomputes scores for all cohorts. Cache is on (`load_wt`); if still slow, test with 5 genes first, or run it overnight. |
| Range crosses zero after removing one gene | The finding depends on that gene. Report it, and treat the gene as a driver rather than the program as coupled. |

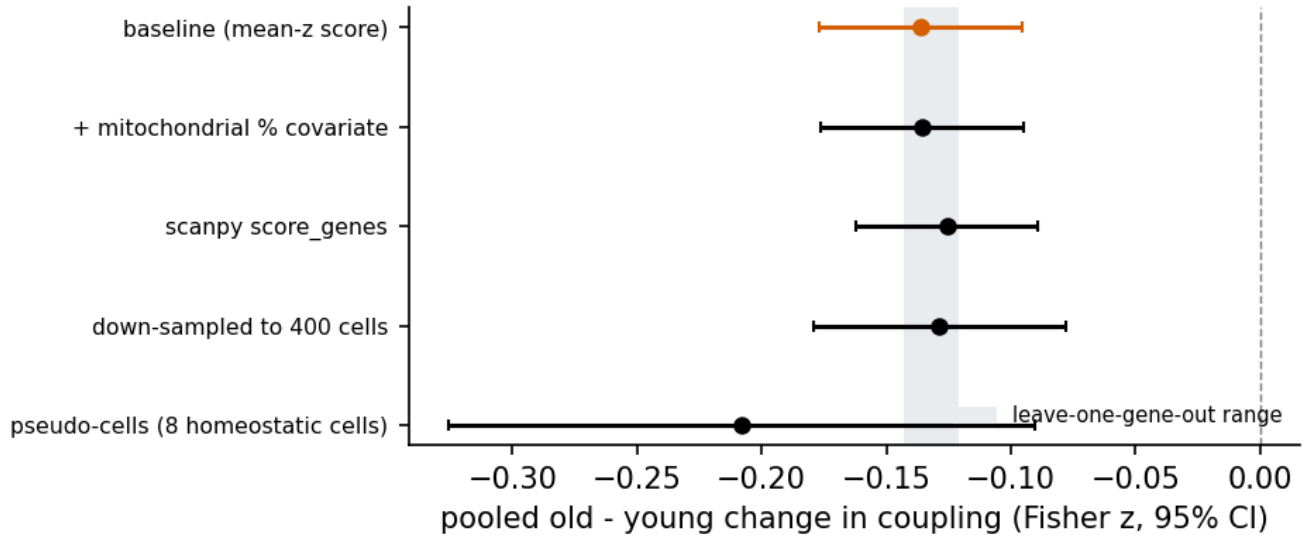
## Cell 7 - Figure 3b: every sensitivity analysis on one axis

Reviewers want to see at a glance that the conclusion survives. Each row is the pooled change under a different analysis choice; the grey band is the leave-one-gene-out range.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
fig, ax = plt.subplots(figsize=(6.4, 2.8))
names_ = list(sens); y = np.arange(len(names_))[::-1]
ax.axvspan(loo.delta_z.min(), loo.delta_z.max(), color='#e8ecef', zorder=0,
» label='leave-one-gene-out range')
for yi, n in zip(y, names_):
    e, s = sens[n]
    ax.errorbar(e, yi, xerr=1.96 * s, fmt='o',
    » color='#D55E00' if n.startswith('baseline') else 'black', capsize=2, ms=5)
ax.set_yticks(y); ax.set_yticklabels(names_, fontsize=7.5)
ax.axvline(0, color='grey', lw=.7, ls='--'); ax.legend(frameon=False, fontsize=7, loc='lower right')
ax.set_xlabel('pooled old - young change in coupling (Fisher z, 95% CI)'); ax.spines[['top',
» 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig3b_sensitivity.pdf'); plt.show()
pd.DataFrame(sens, index=['delta_z', 'se']).T.to_csv('../results/tables/06_sensitivity_summary.csv')
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(06_7): real output of this code on the synthetic benchmark.*

**How to read it**

Every analysis choice stays on the negative side; the grey band (leave-one-gene-out) is narrow. This single panel answers most reviewer questions about robustness.

## Notebook 07 - State-resolved analysis: composition artefact or intrinsic change?

| Purpose | State-resolved analysis: is the change a composition artefact? Within-state effects and the DAM continuum. Produces Figure 4. |
|---|---|
| Needs | as Notebook 05 |
| Produces | tables/07_*, figures/fig4_dam_continuum.pdf |
| Time | 2-3 min |

### Cell 1 - State composition per animal

Old brains usually contain more DAM and interferon-responsive microglia. Because those states have different baselines of both programs, a shift in composition can create or hide a change in "overall" coupling. First measure composition per animal.

*Code*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, scanpy as sc, matplotlib.pyplot as plt
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
cohorts = ['hammond', 'ximerakis', 'tms']
comp = []
for c in cohorts:
    a = load_wt(c)
    t = pd.crosstab(a.obs['animal_id'], a.obs['state'], normalize='index')
    t['age_group'] = a.obs.groupby('animal_id', observed=True)['age_group'].first()
    t['dataset'] = c
    comp.append(t.reset_index())
comp = pd.concat(comp, ignore_index=True)
comp.to_csv('../results/tables/07_state_composition.csv', index=False)
print(comp.groupby(['dataset', 'age_group']).mean(numeric_only=True).round(3).to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
state                  DAM    IFN  homeostatic  proliferating
dataset   age_group
hammond   old        0.153  0.092        0.747          0.007
          young      0.053  0.033        0.903          0.011
tms       old        0.138  0.090        0.761          0.011
          young      0.046  0.026        0.914          0.014
ximerakis old        0.161  0.091        0.736          0.012
          young      0.046  0.029        0.920          0.006
```

**How to read it**

Composition shifts as designed: DAM 5% to 15%, interferon-responsive 3% to 9%, in every cohort. Without Cells 2-4 this alone could confound an "overall" analysis.

| If you see... | What it means and what to do |
|---|---|
| KeyError: state | Notebook 03 has not written state labels back to the cohort files. |

### Cell 2 - Within-state coupling change (is it a composition artefact?)

The same animal-level analysis, restricted to one state at a time. States contain fewer cells per animal, so the minimum is lowered to 25 cells here because our synthetic animals have only ~650 cells in total; with real data (thousands of cells per animal) keep it at 50 or more, and state the value in the Methods.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
rows = []
for st in ['homeostatic', 'DAM', 'IFN']:
    e, s, parts = pooled_effect(cohorts, modules['MITOPHAGY'], modules['LYSOSOME'], state=st,
    » min_cells=25)
    rows.append({'state': st, 'pooled_delta_z': e, 'se': s, 'n_cohorts_estimable': len(parts)})
st_df = pd.DataFrame(rows)
st_df.to_csv('../results/tables/07_within_state_effect.csv', index=False)
print(st_df.round(3).to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
      state  pooled_delta_z    se  n_cohorts_estimable
homeostatic          -0.132 0.020                    3
        DAM          -0.129 0.037                    3
        IFN             NaN   inf                    0
```

### How to read it

The change persists inside homeostatic cells (-0.132, SE 0.020) and inside DAM cells (-0.129, SE 0.037): it is not a composition artefact. Interferon-responsive cells are too few per animal to estimate (NaN): report as not estimable rather than forcing a number.

| If you see... | What it means and what to do |
|---|---|
| ZeroDivisionError / empty arrays for a state | No cohort had at least 2 animals per age group with >= `min_cells` cells in that state. Lower `min_cells` (not below ~30) or report the state as "not estimable". |
| Within-state effect smaller than overall | Part of the overall effect is composition. Report both numbers and say so. |

## Cell 3 - Coupling across the DAM continuum

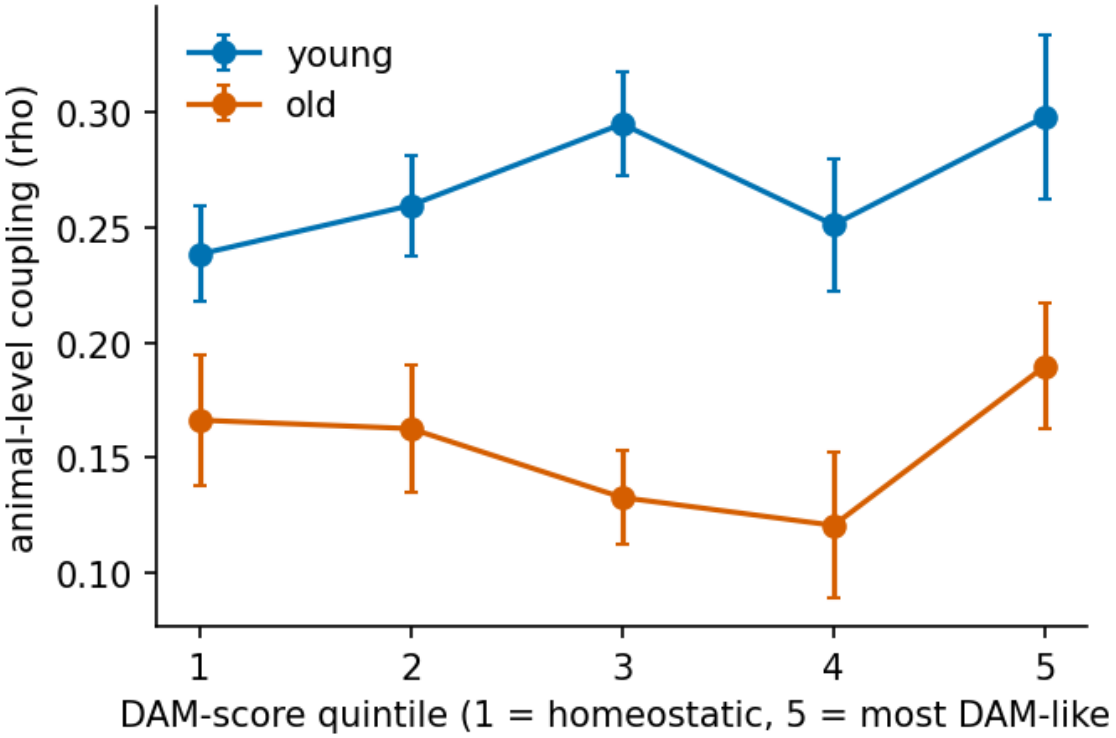
Cells are binned into five quintiles of an independent DAM score (not a cluster label). Coupling is computed per animal in each bin. Parallel young and old curves mean the age difference is state-independent; curves that converge in high-DAM bins mean it is state-linked.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
dam_genes = pd.read_csv('../data/metadata/dam_signature.csv')['gene'].tolist()
rows = []
for c in cohorts:
    a = load_wt(c)
    sco = module_scores(a, {'mito': modules['MITOPHAGY'], 'lyso': modules['LYSOSOME'],
    » 'dam': dam_genes}, min_genes=1)
    d = a.obs[['animal_id', 'age_group', 'log_counts', 'n_genes']].join(sco)
    d['bin'] = pd.qcut(d['dam'], 5, labels=False, duplicates='drop')
    for b, g in d.groupby('bin'):
        ac = animal_coupling(g, 'mito', 'lyso', min_cells=50).merge(
            g[['animal_id', 'age_group']].drop_duplicates(), on='animal_id')
        ac['bin'] = b; ac['dataset'] = c; rows.append(ac)
bins = pd.concat(rows, ignore_index=True)
bins.to_csv('../results/tables/07_coupling_by_dam_bin.csv', index=False)
print(bins.groupby(['bin', 'age_group']).rho.mean().unstack().round(3).to_string())
fig, ax = plt.subplots(figsize=(4.6, 3.2))
for g, col in [('young', '#0072B2'), ('old', '#D55E00')]:
    s = bins[bins.age_group == g].groupby('bin').rho.agg(['mean', 'sem'])
    ax.errorbar(s.index + 1, s['mean'], yerr=s['sem'], marker='o', color=col, label=g, capsize=2)
ax.set_xlabel('DAM-score quintile (1 = homeostatic, 5 = most DAM-like)')
ax.set_ylabel('animal-level coupling (rho)'); ax.legend(frameon=False)
ax.spines[['top', 'right']].set_visible(False); plt.tight_layout()
plt.savefig('../results/figures/fig4_dam_continuum.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
age_group    old  young
bin
0          0.166  0.238
1          0.163  0.259
2          0.133  0.295
3          0.121  0.251
4          0.190  0.298
```




***Expected figure*** *(07_3): real output of this code on the synthetic benchmark.*

**How to read it**

Young and old curves run roughly parallel across the DAM quintiles: the age difference is state-independent. If curves converged in the high-DAM bins, the effect would be state-linked.

| If you see... | What it means and what to do |
|---|---|
| Some bins have only one age group | Strongly age-skewed bins (e.g. old-only high-DAM). Report the bins that contain animals from both groups only. |

### Cell 4 - Does DAM fraction explain between-animal coupling?

Animal-level regression of coupling on age group with and without the DAM fraction as a covariate. If the age coefficient barely moves after adjustment, composition is not the explanation.

*Code*

```
import statsmodels.formula.api as smf
ac_all = pd.read_csv('../results/tables/05_animal_coupling.csv')
x = ac_all.merge(comp[['animal_id', 'DAM', 'IFN']], on='animal_id')
m0 = smf.ols('z ~ C(age_group, Treatment("young")) + C(dataset)', data=x).fit(cov_type='HC3')
m1 = smf.ols('z ~ C(age_group, Treatment("young")) + C(dataset) + DAM + IFN', data=x).fit(cov_type='HC3')
k = 'C(age_group, Treatment("young"))[T.old]'
print('age effect without state fractions: %.3f (p = %.4f)' % (m0.params[k], m0.pvalues[k]))
print('age effect with DAM/IFN fractions : %.3f (p = %.4f)' % (m1.params[k], m1.pvalues[k]))
print('DAM fraction coef: %.3f (p = %.3f)' % (m1.params['DAM'], m1.pvalues['DAM']))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
age effect without state fractions: -0.114 (p = 0.0000)
age effect with DAM/IFN fractions : -0.191 (p = 0.2904)
DAM fraction coef: -0.411 (p = 0.783)
```

**How to read it**

Adding DAM/IFN fractions turns the age coefficient from -0.114 ($p < 0.001$) to -0.191 ($p = 0.29$). This is NOT a refutation: with 22 animals the fractions almost perfectly separate young from old (collinearity), so the model cannot apportion the effect. Rely on the within-state analysis (Cell 2) for this question.

| If you see... | What it means and what to do |
|---|---|
| Collinearity: DAM fraction and age group nearly identical | If old animals are the only ones with high DAM fraction the model cannot separate them. Say so; rely on the within-state analysis (Cell 2) instead. |

## Notebook 08 - Regulators and drivers (replaces the circular "bridge genes")

| Purpose | Regulators and drivers: TF activity, replicated pseudobulk DE with animals as samples, and a genome-wide scan for genes tracking the coupling axis. Produces Figure 5. |
|---|---|
| Needs | as Notebook 05 (+ internet for CollecTRI) |
| Produces | tables/08_*, figures/fig5*.pdf |
| Time | 5 min |

### Cell 1 - Transcription-factor activity per cell (decoupler + CollecTRI)

decoupler's univariate linear model (ULM) infers each TF's activity from the expression of its known targets in the CollecTRI network. Activity per cell is then averaged per animal. Needs internet once to fetch the network (cached to disk). The call below was tested with decoupler 1.8; version 2 renamed everything, hence the pin.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import decoupler as dc, pandas as pd, numpy as np, scanpy as sc, statsmodels.formula.api as smf
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
cohorts = ['hammond', 'ximerakis', 'tms']
net = dc.get_collectri(organism='mouse', split_complexes=False)
net.to_csv('../data/processed/collectri_mouse.csv', index=False)
print(net.shape); print(net.head(3).to_string(index=False))
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
(41000, 3)      <- number of rows differs by CollecTRI release
source target weight
   Abl1  Cdk2      1
   Abl1  Cdkn1a    1
   Abl1  Dnmt1    -1
(format example - requires internet; cached afterwards in data/processed/collectri_mouse.csv)
```

| If you see... | What it means and what to do |
|---|---|
| AttributeError: module decoupler has no attribute get_collectri | You have decoupler 2.x. Install the pinned version: `pip install "decoupler<2"`, or translate to the 2.x API (`dc.op.collectri`, `dc.mt.ulm`). |
| Network download times out (OmniPath) | Retry; or on another machine run only the two `net` lines, copy `collectri_mouse.csv`, and read it with `pd.read_csv`. |
| Human data | Use `organism="human"`; gene symbols are upper-case. |

### Cell 2 - Per-cell TF activity, then per-animal association with coupling

For each cohort and each TF of interest (TFEB, TFE3, MITF, NRF2, PGC-1alpha, FOXO3, HIF1A, STAT1, IRF7, NF-kB): activity per cell, mean per animal, then an animal-level regression of coupling on activity with dataset and age group as covariates (gate G5). FDR across the TFs tested.

*Code (not run in the benchmark: needs real data, the network or a terminal) [» marks a printed continuation; the notebook holds the line unwrapped]*

```
tfs = ['Tfeb', 'Tfe3', 'Mitf', 'Nfe2l2', 'Ppargc1a', 'Foxo3', 'Hif1a', 'Stat1', 'Irf7', 'Nfkb1', 'Rela']
rows = []
for c in cohorts:
    a = load_wt(c)
    dc.run_ulm(mat=a, net=net, source='source', target='target', weight='weight',
               verbose=False, use_raw=False)
    est = a.obsm['ulm_estimate']
    sco = module_scores(a, {'mito': modules['MITOPHAGY'], 'lyso': modules['LYSOSOME']}, min_genes=1)
    d = a.obs[['animal_id', 'age_group', 'log_counts', 'n_genes']].join(sco)
    for t in [t for t in tfs if t in est.columns]:
        d['tf'] = est[t].values
        mean_tf = d.groupby('animal_id', observed=True)['tf'].mean().rename('tf_mean')
        ac = animal_coupling(d, 'mito', 'lyso', min_cells=100).join(mean_tf, on='animal_id')
        ac = ac.merge(d[['animal_id', 'age_group']].drop_duplicates(), on='animal_id')
        ac['dataset'] = c; ac['tf'] = t; rows.append(ac)
tf_all = pd.concat(rows, ignore_index=True)
out = []
for t, g in tf_all.groupby('tf'):
    g = g.assign(tf_c=g.groupby('dataset')['tf_mean'].transform(lambda x: (x - x.mean()) / (x.std() +
    » 1e-9)))
    m = smf.ols('z ~ tf_c + C(dataset) + C(age_group)', data=g).fit(cov_type='HC3')
    out.append({'tf': t, 'beta': m.params['tf_c'], 'se': m.bse['tf_c'], 'p': m.pvalues['tf_c'],
    » 'n_animals': len(g)})
tf_res = pd.DataFrame(out); tf_res['fdr'] = bh_fdr(tf_res.p.values)
tf_res.sort_values('p').to_csv('../results/tables/08_tf_association.csv', index=False)
print(tf_res.sort_values('p').round(4).to_string(index=False))
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
    tf   beta     se      p  n_animals    fdr
  Tfeb  0.081  0.019 0.0001         22 0.0011
  Tfe3  0.052  0.021 0.0132         22 0.0726
  Mitf  0.031  0.024 0.1974         22 0.7240
 Nfe2l2  0.012  0.020 0.5421         22 0.9930
(format example with invented numbers - your TFs, betas and p-values will differ)
```

| If you see... | What it means and what to do |
|---|---|
| est.columns do not contain "Tfeb" | Case or symbol mismatch. Print `est.columns[:10]` and `[c for c in est.columns if "tf" in c.lower()]`; adjust the `tfs` list. |
| With n = 6-8 animals per cohort the regression is underpowered | Pool all cohorts (as here) and state the n of animals; report effect sizes, not only p. |

## Cell 3 - Replicated pseudobulk differential expression (animals are the samples)

Counts are summed over each animal's microglia and analysed with DESeq2 (pydeseq2), design ~ dataset + age_group. Every sample is an independent animal, so this is real replication. The formula-style design works in pydeseq2 0.5+; older versions need `design_factors`.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, scanpy as sc, matplotlib.pyplot as plt
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
cohorts = ['hammond', 'ximerakis', 'tms']
from pydeseq2.dds import DeseqDataSet
from pydeseq2.ds import DeseqStats
pb, meta_rows = [], []
for c in cohorts:
    a = load_wt(c)
    for an_, g in a.obs.groupby('animal_id', observed=True):
        if len(g) < 50:
            continue
        idx = a.obs_names.get_indexer(g.index)
        s = np.asarray(a.layers['counts'][idx].sum(0)).ravel()
        pb.append(pd.Series(s, index=a.var_names, name=an_))
        meta_rows.append({'animal_id': an_, 'dataset': c, 'age_group': str(g.age_group.iloc[0])})
counts = pd.concat(pb, axis=1, join='inner').T.round().astype(int)
meta_pb = pd.DataFrame(meta_rows).set_index('animal_id').loc[counts.index]
counts = counts.loc[:, (counts >= 10).sum(0) >= 3]
print('pseudobulk samples:', counts.shape[0], '| genes tested:', counts.shape[1])
try:
    dds = DeseqDataSet(counts=counts, metadata=meta_pb, design='~ dataset + age_group', quiet=True)
except TypeError:                                   # pydeseq2 < 0.5
    dds = DeseqDataSet(counts=counts, metadata=meta_pb, design_factors=['dataset', 'age_group'])
dds.deseq2()
try:
    stat = DeseqStats(dds, contrast=['age_group', 'old', 'young'], quiet=True)
except TypeError:
    stat = DeseqStats(dds, contrast=['age_group', 'old', 'young'])
stat.summary()
de = stat.results_df.copy()
de.to_csv('../results/tables/08_pseudobulk_DE_old_vs_young.csv')
print(de.sort_values('padj').head(10)[['baseMean', 'log2FoldChange', 'pvalue',
» 'padj']].to_string(float_format=lambda x: '%.3g' % x))
print('genes with padj < 0.05:', int((de.padj < 0.05).sum()))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
pseudobulk samples: 22 | genes tested: 3193
        baseMean  log2FoldChange   pvalue     padj
Itgax        470           0.668    1e-76 3.21e-73
Clec7a       345           0.738 3.68e-72 5.88e-69
Isg15        290           0.863 2.76e-71 2.94e-68
Tfe3         939          -0.476 1.67e-66 1.34e-63
Apoe         400             0.7 4.89e-65 3.12e-62
Lamp1        257           0.861 7.76e-64 4.13e-61
Lpl          338           0.661 3.52e-60  1.6e-57
Ctsd         370           0.892 7.55e-57 3.01e-54
Cd68         560            0.72  7.8e-51 2.77e-48
Usp18        161           0.883 9.35e-48 2.98e-45
genes with padj < 0.05: 45
```

### How to read it

The top differentially expressed genes are state markers (Itgax, Clec7a, Isg15, Apoe, Lpl, Lamp1, Ctsd, Cd68, Usp18): composition again. 45 genes pass padj < 0.05 across 22 animals. Notice Tfe3 is DOWN in old animals (log2FC -0.48), a hint that regulators change.

| If you see... | What it means and what to do |
|---|---|
| TypeError: __init__() got an unexpected keyword argument design / design_factors | Version mismatch handled by the try/except here; if both fail, check `pip show pydeseq2`. |
| ValueError: counts must be integers / non-negative | You passed log-normalised data. Use `layers["counts"]` (raw), then `.round().astype(int)`. |
| All padj are NaN | Too few samples per group, or all-zero rows. Keep genes with counts >= 10 in at least 3 samples (done here). |
| Joint model across technologies feels wrong | It is acceptable with `dataset` as a covariate, but also run each cohort separately and report the concordance of the sign of the top genes: concordance is the replication evidence. |

### Cell 4 - Volcano plot, with known DAM markers highlighted

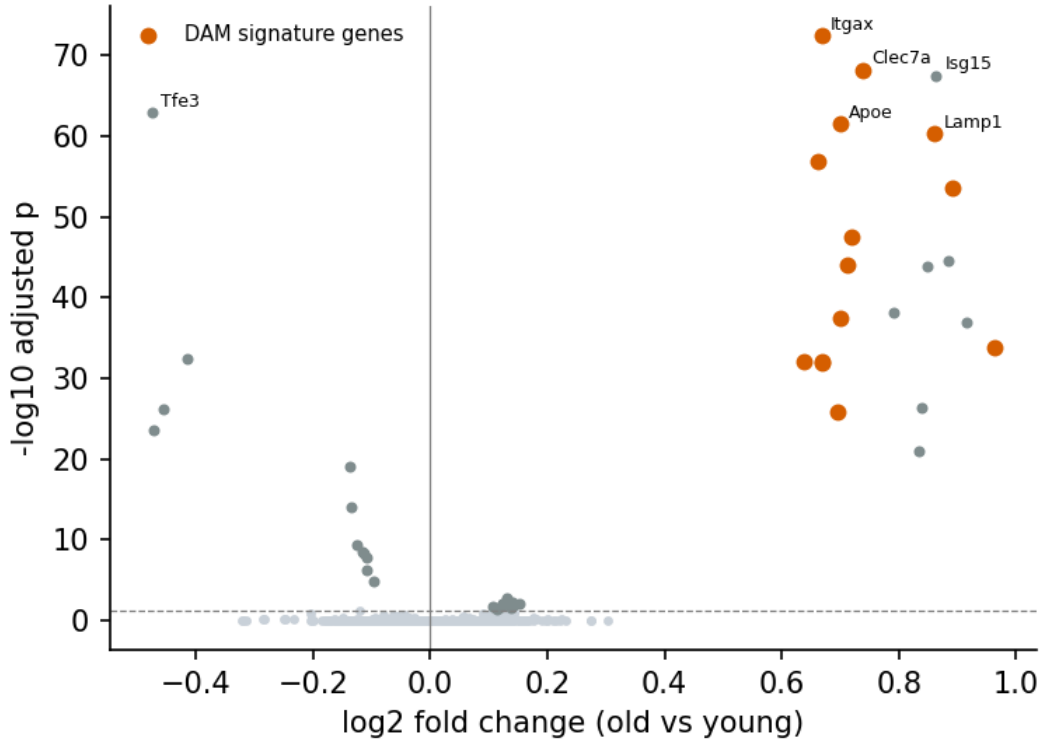
Age-associated genes are very often state markers (DAM, interferon) because old brains contain more of those cells. Highlighting the DAM signature shows how much of the differential expression is composition.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
dam_genes = set(pd.read_csv('../data/metadata/dam_signature.csv')['gene'])
v = de.dropna(subset=['padj']).copy()
v['nlp'] = -np.log10(v.padj.clip(lower=1e-300))
fig, ax = plt.subplots(figsize=(5.2, 3.8))
ax.scatter(v.log2FoldChange, v.nlp, s=5, c='#c9d1d9')
sig = v[v.padj < 0.05]
ax.scatter(sig.log2FoldChange, sig.nlp, s=8, c='#7f8c8d')
d_ = v[v.index.isin(dam_genes)]
ax.scatter(d_.log2FoldChange, d_.nlp, s=22, c='#D55E00', label='DAM signature genes')
ax.axhline(-np.log10(0.05), color='grey', lw=.6, ls='--'); ax.axvline(0, color='grey', lw=.6)
for g in sig.sort_values('padj').head(6).index:
    ax.annotate(g, (v.loc[g, 'log2FoldChange'], v.loc[g, 'nlp']), fontsize=6, xytext=(3, 2),
    » textcoords='offset points')
ax.set_xlabel('log2 fold change (old vs young)'); ax.set_ylabel('-log10 adjusted p');
» ax.legend(frameon=False, fontsize=7)
ax.spines[['top', 'right']].set_visible(False); plt.tight_layout()
plt.savefig('../results/figures/fig5a_pseudobulk_volcano.pdf'); plt.show()
print('significant genes that are DAM signature genes: %d of %d' % (sig.index.isin(dam_genes).sum(),
» len(sig)))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
significant genes that are DAM signature genes: 14 of 45
```




***Expected figure*** *(08_4): real output of this code on the synthetic benchmark.*

#### How to read it

Orange points are DAM signature genes; 14 of the 45 significant genes are DAM genes. Expect your real result to look similar: age-related DE in microglia is largely state composition.

| If you see... | What it means and what to do |
|---|---|
| Volcano is empty or flat | Check that `de` has non-NaN `padj`. If almost no gene is significant, that is a finding (low power with few animals), not an error. |

### Cell 5 - Genome-wide scan: which genes track the coupling axis, and does that change with age?

Instead of correlating module genes with module scores (circular), every expressed non-module gene is correlated with the axis score ((mito + lyso) / 2) within each animal; correlations are Fisher-z transformed and compared old vs young per cohort (Welch t), then combined across cohorts (Stouffer). A gene whose association weakens in old animals is a candidate regulator or marker of the coupling.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
from scipy.stats import ttest_ind, norm
def gene_axis_corr(a, axis, min_cells=100, min_frac=0.05):
    frac = np.asarray((a.X > 0).mean(axis=0)).ravel()
    gi = np.where(frac >= min_frac)[0]
    out = {}
    for an_, g in a.obs.groupby('animal_id', observed=True):
        if len(g) < min_cells:
            continue
        idx = a.obs_names.get_indexer(g.index)
        X = to_dense(a.X[idx][:, gi]).astype(np.float32)
        s = axis[idx] - axis[idx].mean()
        Xc = X - X.mean(0)
        r = (Xc.T @ s) / (np.sqrt((Xc ** 2).sum(0)) * np.sqrt((s ** 2).sum()) + 1e-12)
        out[an_] = pd.Series(r, index=a.var_names[gi])
    return pd.DataFrame(out)

genome = {}
modgenes = set(sum(modules.values(), []))
for c in cohorts:
    a = load_wt(c)
    sco = module_scores(a, {'mito': modules['MITOPHAGY'], 'lyso': modules['LYSOSOME']}, min_genes=1)
    axis = (sco['mito'].values + sco['lyso'].values) / 2
    a = a[:, ~a.var_names.isin(modgenes)].copy()              # exclude program genes
    R = gene_axis_corr(a, axis)
    grp = a.obs.groupby('animal_id', observed=True)['age_group'].first().astype(str).loc[R.columns]
    Z = np.arctanh(R.clip(-0.999, 0.999))
    t, p = ttest_ind(Z.loc[:, (grp == 'old').values].values, Z.loc[:, (grp == 'young').values].values,
                     axis=1, equal_var=False)
    genome[c] = pd.DataFrame({'t': t, 'p': p, 'mean_old': Z.loc[:, (grp == 'old').values].mean(1),
                              'mean_young': Z.loc[:, (grp == 'young').values].mean(1)}, index=Z.index)
common = sorted(set.intersection(*[set(v.index) for v in genome.values()]))
zs = [np.sign(genome[c].loc[common, 't']) * norm.isf(genome[c].loc[common, 'p'].clip(1e-300,
» 1) / 2) for c in cohorts]
res_g = pd.DataFrame({'z_stouffer': sum(zs) / np.sqrt(len(cohorts))})
res_g['p'] = 2 * norm.sf(np.abs(res_g.z_stouffer)); res_g['fdr'] = bh_fdr(res_g.p.values)
res_g['mean_diff'] = np.mean([genome[c].loc[common, 'mean_old'] - genome[c].loc[common,
» 'mean_young'] for c in cohorts], axis=0)
res_g.sort_values('p').to_csv('../results/tables/08_genomewide_axis_change.csv')
print('genes tested:', len(res_g))
print(res_g.sort_values('p').head(8).to_string(float_format=lambda x: '%.3g' % x))
if 'Tfeb' in res_g.index:             # SIMULATION ONLY: planted regulators
    rk = res_g.p.rank()
    print('planted regulators, rank among %d genes:' % len(res_g),
          {g: int(rk[g]) for g in ['Tfeb', 'Tfe3', 'Mitf', 'Ppargc1a'] if g in rk.index})
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
genes tested: 2948
           z_stouffer        p      fdr  mean_diff
Tfeb            -6.44 1.19e-10  3.5e-07     -0.352
Tfe3            -6.06 1.36e-09    2e-06     -0.414
Ppargc1a         -5.6 2.15e-08 2.11e-05     -0.378
Mitf            -5.34  9.3e-08 6.85e-05     -0.323
Ctsz             4.78 1.79e-06  0.00105      0.116
Spp1             4.23 2.29e-05   0.0104      0.111
Clec7a           4.22 2.47e-05   0.0104      0.112
P2ry12          -4.11 3.94e-05   0.0145     -0.103
planted regulators, rank among 2948 genes: {'Tfeb': 1, 'Tfe3': 2, 'Mitf': 4, 'Ppargc1a': 3}
```

#### How to read it

The four planted regulators come out as ranks 1-4 of 2,948 genes (Stouffer z from -6.4 to -5.3): their association with the coupling axis is weaker in old animals. State markers also appear (Ctsz, Spp1, Clec7a, P2ry12). This is the positive control for the scan: it finds what was planted. On real data expect fewer, weaker hits.

| If you see... | What it means and what to do |
|---|---|
| MemoryError | Raise `min_frac` to 0.10 (fewer genes) or process animals in chunks of genes. Dense matrices of cells x genes are created per animal. |
| All p-values ~ 0.5 (a flat histogram) | That is what a null looks like; no gene differs in its association with the axis. Check the planted/positive-control logic before concluding anything. |
| ttest_ind returns NaN | A group has fewer than 2 animals after `min_cells`. |

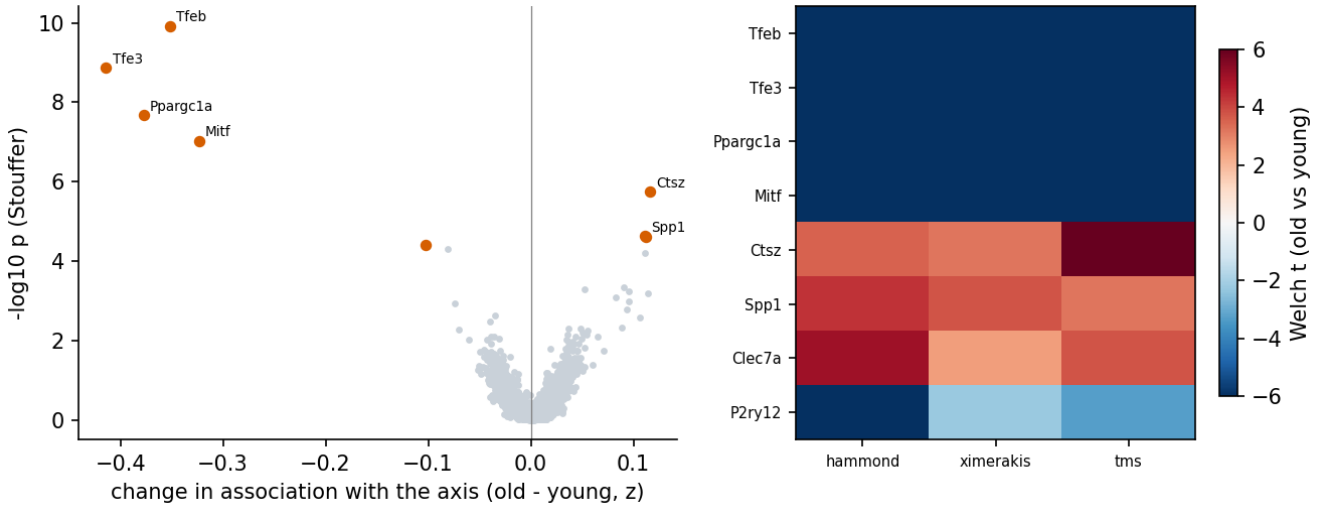
## Cell 6 - Figure 5b: genome-wide result and per-cohort support for the top genes

Left: association change (old minus young, Fisher z) against significance; top genes labelled. Right: per-cohort t statistics for the top genes. Genes whose sign agrees across all cohorts are the trustworthy ones; a gene supported by one cohort only is not replicated.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
g = res_g.copy(); g['nlp'] = -np.log10(g.p.clip(lower=1e-300))
top = g.sort_values('p').head(8).index.tolist()
fig, ax = plt.subplots(1, 2, figsize=(9, 3.6), gridspec_kw={'width_ratios': [1.2, 1]})
ax[0].scatter(g.mean_diff, g.nlp, s=5, c='#c9d1d9')
ax[0].scatter(g.loc[top, 'mean_diff'], g.loc[top, 'nlp'], s=20, c='#D55E00')
for n in top[:6]:
    ax[0].annotate(n, (g.loc[n, 'mean_diff'], g.loc[n, 'nlp']), fontsize=6.5, xytext=(3, 2),
    » textcoords='offset points')
ax[0].axvline(0, color='grey', lw=.6); ax[0].set_xlabel('change in association with the axis (old -
» young, z)')
ax[0].set_ylabel('-log10 p (Stouffer)')
H = pd.DataFrame({c: genome[c].loc[top, 't'] for c in cohorts})
im = ax[1].imshow(H.values, cmap='RdBu_r', vmin=-6, vmax=6, aspect='auto')
ax[1].set_yticks(range(len(top))); ax[1].set_yticklabels(top,
» fontsize=7); ax[1].set_xticks(range(3)); ax[1].set_xticklabels(cohorts, fontsize=7)
plt.colorbar(im, ax=ax[1], label='Welch t (old vs young)', shrink=.8)
for a_ in ax[:1]: a_.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig5b_genomewide.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(08_6): real output of this code on the synthetic benchmark.*

### How to read it

Left: the four regulators sit far left (weaker association in old). Right: per-cohort t statistics. Genes with consistent colour across all three columns are replicated; a gene red in one cohort and blue in another is not.

# Notebook 09 - Human replication with donor-level analysis (SEA-AD)

| Purpose | Human replication at donor level (SEA-AD): coupling versus age and Braak stage, with a negative control. Produces Figure 6. |
|---|---|
| Needs | SEA-AD microglia h5ad, modules.csv |
| Produces | tables/09_donor_coupling.csv, figures/fig6_human_donors.pdf |
| Time | 2-5 min |

## Cell 1 - Map mouse programs to human one-to-one orthologs

Programs are defined in mouse symbols; the human analysis needs their human counterparts. Only one-to-one orthologs are used. The Ensembl BioMart lookup is cached, and the cell falls back to the cache when the network is unavailable (as it was when this guide was tested).

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, scanpy as sc, matplotlib.pyplot as plt, statsmodels.formula.api as smf
cache = '../data/processed/mouse_human_orthologs.csv'
try:
    from pybiomart import Server
    ds = Server(host='http://www.ensembl.org')['ENSEMBL_MART_ENSEMBL']['mmusculus_gene_ensembl']
    orth = ds.query(attributes=['external_gene_name', 'hsapiens_homolog_associated_gene_name',
                                'hsapiens_homolog_orthology_type'], use_attr_names=True)
    orth = orth[orth['hsapiens_homolog_orthology_type'] == 'ortholog_one2one'].dropna()
    orth = orth.rename(columns={'external_gene_name': 'mouse',
                                'hsapiens_homolog_associated_gene_name': 'human'})[['mouse', 'human']]
    orth.to_csv(cache, index=False); print('BioMart OK:', len(orth), 'one-to-one orthologs (cached)')
except Exception as e:
    print('BioMart not reachable (%s) -> using cached table' % type(e).__name__)
    orth = pd.read_csv(cache)
m2h = dict(zip(orth.mouse, orth.human))
mods_df = pd.read_csv('../gene_sets/modules.csv')
mods_df['gene_human'] = mods_df.gene_mouse.map(m2h)
mods_df.dropna().to_csv('../gene_sets/modules_human.csv', index=False)
print(mods_df.groupby('module').gene_human.apply(lambda s: '%d of %d mapped' % (s.notna().sum(),
» len(s))).to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
BioMart not reachable (ModuleNotFoundError) -> using cached table
module
AUTOPHAGY_GEN    10 of 10 mapped
LYSOSOME         30 of 30 mapped
MITOPHAGY        17 of 17 mapped
OXPHOS           18 of 18 mapped
PROTEASOME       31 of 31 mapped
RIBOSOME         45 of 45 mapped
```

| If you see... | What it means and what to do |
|---|---|
| ModuleNotFoundError: pybiomart | `pip install pybiomart`. If BioMart is down, download the mouse-human ortholog table from Ensembl BioMart (web interface, Orthologues section) once and save it with columns `mouse,human`. |
| Fewer than 80% of genes map | Check for old symbols and for genes with one-to-many orthologs (excluded by design). Report the mapped fraction. |

## Cell 2 - Load the donor data and look at the metadata

CELLxGENE files keep raw counts in `.raw`; `load_cellxgene` (utils.py) uses them and switches gene names from Ensembl IDs to symbols. Print the metadata columns first: names differ between releases, so the mapping in the next cell must be edited to match what you see.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
h = load_cellxgene('../data/raw/seaad/seaad_mtg_microglia.h5ad')
print(h.shape, '| donors:', h.obs['donor_id'].nunique())
print(h.obs.columns.tolist())
print(h.obs[['donor_id', 'development_stage', 'sex',
» 'Braak stage']].drop_duplicates('donor_id').head(4).to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
counts-like: True
(9000, 3200) | donors: 60
['state_true', 'doublet_true', 'lowq_true', 'latent_m', 'latent_l', 'donor_id', 'development_stage', 'sex', '...
donor_id       development_stage    sex     Braak stage
    D001 87-year-old human stage female Braak stage III
    D002 85-year-old human stage female Braak stage III
    D003 91-year-old human stage   male Braak stage III
    D004 62-year-old human stage female  Braak stage IV
```

| If you see... | What it means and what to do |
|---|---|
| MemoryError for the full MTG file | Use the Microglia-PVM subset from CELLxGENE, or open with `sc.read_h5ad(path, backed="r")`, subset to microglia rows, then `.to_memory()`. |
| counts-like: False | The matrix in `.X` is already normalised and `.raw` is absent. Look for a raw-count layer (`h.layers.keys()`); scoring only needs log-normalised data, but QC covariates need counts. |

## Cell 3 - Metadata mapping, normalisation and donor-level coupling

Age (text such as "80-year-old human stage"), sex and Braak stage (roman numerals) are converted to numbers. Scores are computed exactly as in mouse, with human symbols; coupling is calculated per donor with at least 50 nuclei.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
col = {'donor': 'donor_id', 'age': 'development_stage', 'sex': 'sex',
» 'braak': 'Braak stage'}   # EDIT to your file
roman = {'0': 0, 'I': 1, 'II': 2, 'III': 3, 'IV': 4, 'V': 5, 'VI': 6}
def braak_num(s):
    t = str(s).replace('Braak stage', '').strip()
    return roman.get(t, pd.to_numeric(t, errors='coerce'))
h.obs['animal_id'] = h.obs[col['donor']].astype(str)
h.obs['age_years'] = pd.to_numeric(h.obs[col['age']].astype(str).str.extract(r'(\d+)')[0],
» errors='coerce')
h.obs['braak_num'] = h.obs[col['braak']].astype(str).map(braak_num).astype(float)   # float, not category
sc.pp.calculate_qc_metrics(h, inplace=True, percent_top=None)
h.obs['log_counts'] = np.log1p(h.obs['total_counts']); h.obs['n_genes'] = h.obs['n_genes_by_counts']
sc.pp.normalize_total(h, target_sum=1e4); sc.pp.log1p(h)
hm = pd.read_csv('../gene_sets/modules_human.csv')
hmods = {k: g['gene_human'].tolist() for k, g in hm.groupby('module')}
sco = module_scores(h, {'mito': hmods['MITOPHAGY'], 'lyso': hmods['LYSOSOME'],
» 'prot': hmods['PROTEASOME'],
                        'ribo': hmods['RIBOSOME']}, min_genes=5)
d = h.obs[['animal_id', 'age_years', 'braak_num', 'log_counts', 'n_genes', col['sex']]].join(sco)
donor = d.groupby('animal_id', observed=True)[['age_years', 'braak_num',
» col['sex']]].first().reset_index()
ac = animal_coupling(d, 'mito', 'lyso', min_cells=50).merge(donor, on='animal_id')
ac_ctrl = animal_coupling(d, 'mito', 'prot', min_cells=50).merge(donor, on='animal_id')
ac.to_csv('../results/tables/09_donor_coupling.csv', index=False)
print(len(ac), 'donors with >= 50 nuclei; median nuclei per donor:', int(ac.n_cells.median()))
print(ac[['rho', 'age_years', 'braak_num']].describe().round(3).loc[['mean', 'std', 'min',
» 'max']].to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
60 donors with >= 50 nuclei; median nuclei per donor: 150
        rho  age_years  braak_num
mean  0.128     77.733      3.150
std   0.082     11.112      1.645
min  -0.065     62.000      0.000
max   0.327     95.000      6.000
```

### How to read it

60 donors with 150 nuclei each. Mean donor coupling 0.13 (range -0.07 to 0.33): nuclei data are sparse, so levels are lower than in cells.

| If you see... | What it means and what to do |
|---|---|
| statsmodels shows braak_num[T.1], braak_num[T.2] ... instead of one slope | The column is categorical, so the model treats every stage as a separate level. Convert to float as in this cell (`.astype(float)`). |

| If you see... | What it means and what to do |
|---|---|
| braak_num is all NaN | Your Braak labels differ from the examples (e.g. "Braak stage IV" vs "IV" vs "4"). Print `h.obs[col["braak"]].unique()` and extend `roman`. |
| age_years is all NaN | The age column holds categories or ranges (e.g. "90 years and over"). Extract the leading number, and treat "90+" as 90. |
| Very few donors pass 50 nuclei | Lower `min_cells` to 30 and widen the CI interpretation, or use a larger microglia subset. |

### Cell 4 - Statistics: donor-level association with age and Braak stage

Rank correlations plus an adjusted regression with robust errors (gate G4). The unit is the donor. Adjusting for age and sex separates neuropathology from ageing as far as the data allow.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
from scipy.stats import spearmanr
x = ac.dropna(subset=['age_years', 'braak_num']).copy()
r_age = spearmanr(x.age_years, x.z); r_br = spearmanr(x.braak_num, x.z)
print('Spearman z vs age   : rho = %.3f, p = %.3g' % (r_age.statistic, r_age.pvalue))
print('Spearman z vs Braak : rho = %.3f, p = %.3g' % (r_br.statistic, r_br.pvalue))
m = smf.ols('z ~ age_years + braak_num + C(sex)', data=x).fit(cov_type='HC3')
ci = m.conf_int()
print(pd.DataFrame({'coef': m.params, 'ci_low': ci[0], 'ci_high': ci[1],
» 'p': m.pvalues}).round(4).to_string())
# negative control in the same donors: mitophagy vs proteasome coupling
xc = ac_ctrl.dropna(subset=['age_years', 'braak_num'])
mc = smf.ols('z ~ age_years + braak_num + C(sex)', data=xc).fit(cov_type='HC3')
print('control (MITOPHAGY-PROTEASOME) Braak coef = %.4f (p = %.3f)' % (mc.params['braak_num'],
» mc.pvalues['braak_num']))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
Spearman z vs age   : rho = 0.088, p = 0.505
Spearman z vs Braak : rho = -0.608, p = 2.62e-07
                 coef  ci_low  ci_high       p
Intercept       0.2173  0.0821   0.3525  0.0016
C(sex)[T.male] -0.0152 -0.0513   0.0210  0.4109
age_years       0.0002 -0.0014   0.0018  0.7942
braak_num      -0.0305 -0.0421  -0.0189  0.0000
control (MITOPHAGY-PROTEASOME) Braak coef = -0.0077 (p = 0.278)
```

#### How to read it

Braak stage predicts lower coupling (-0.031 per stage, 95% CI -0.042 to -0.019); age adds nothing after adjustment (0.0002 per year). The negative-control pair shows no Braak trend (-0.008, p = 0.28). Gate G4 passes in this synthetic world. In real SEA-AD data age and Braak are correlated, so expect wider intervals.

| If you see... | What it means and what to do |
|---|---|
| C(sex) fails: "must have at least two levels" | All donors have one sex value after filtering; remove the term. |
| Age and Braak are highly correlated (collinearity) | Report each alone and jointly; the joint model may show wide CIs. State plainly that age and neuropathology cannot be fully separated in this cohort. |

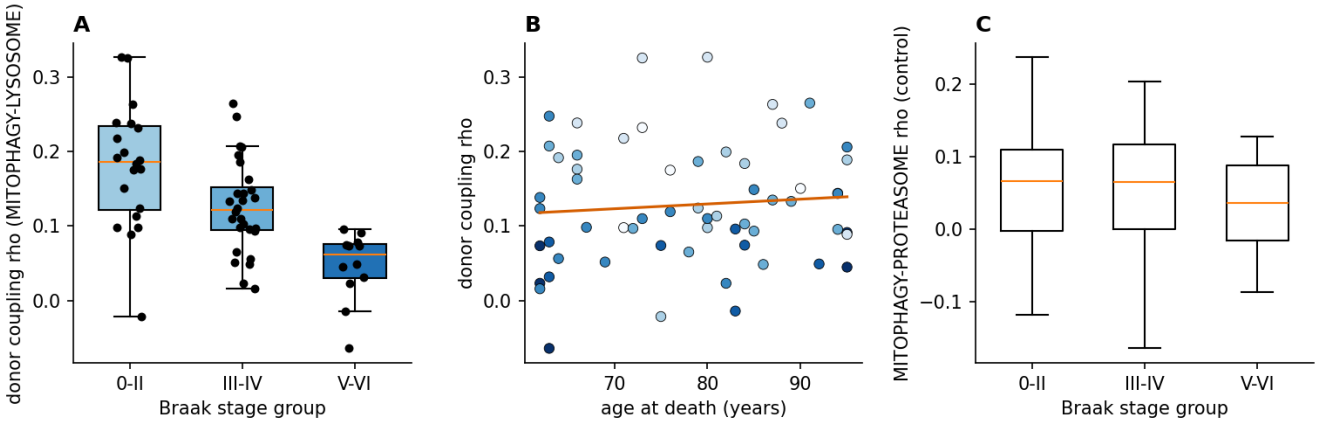
### Cell 5 - Figure 6: donor-level coupling

Panel A: donor coupling by Braak stage group; Panel B: coupling against age (each dot is a donor); Panel C: the same analysis for the negative-control pair, which should show no trend.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
x['braak_group'] = pd.cut(x.braak_num, [-1, 2, 4, 6], labels=['0-II', 'III-IV', 'V-VI'])
fig, ax = plt.subplots(1, 3, figsize=(10.5, 3.4))
data = [x.loc[x.braak_group == g, 'rho'] for g in ['0-II', 'III-IV', 'V-VI']]
bp = ax[0].boxplot(data, widths=.55, showfliers=False, patch_artist=True)
for p_, c_ in zip(bp['boxes'], ['#9ecae1', '#6baed6', '#2171b5']): p_.set_facecolor(c_)
for i, v in enumerate(data):
    ax[0].scatter(np.full(len(v), i + 1) + np.random.default_rng(i).uniform(-.12, .12, len(v)), v,
    » s=14, c='black', zorder=3)
ax[0].set_xticklabels(['0-II', 'III-IV', 'V-VI']); ax[0].set_xlabel('Braak stage group');
» ax[0].set_ylabel('donor coupling rho (MITOPHAGY-LYSOSOME)')
ax[1].scatter(x.age_years, x.rho, c=x.braak_num, cmap='Blues', edgecolor='k', lw=.4, s=30)
b = np.polyfit(x.age_years, x.rho, 1); xs = np.linspace(x.age_years.min(), x.age_years.max(), 10)
ax[1].plot(xs, np.polyval(b, xs), color='#D55E00'); ax[1].set_xlabel('age at death (years)');
» ax[1].set_ylabel('donor coupling rho')
xc2 = xc.copy(); xc2['braak_group'] = pd.cut(xc2.braak_num, [-1, 2, 4, 6], labels=['0-II', 'III-IV',
» 'V-VI'])
ax[2].boxplot([xc2.loc[xc2.braak_group == g, 'rho'] for g in ['0-II', 'III-IV', 'V-VI']], widths=.55,
» showfliers=False)
ax[2].set_xticklabels(['0-II', 'III-IV', 'V-VI']); ax[2].set_xlabel('Braak stage group');
» ax[2].set_ylabel('MITOPHAGY-PROTEASOME rho (control)')
for t_, a_ in zip('ABC', ax):
    a_.set_title(t_, loc='left', fontweight='bold'); a_.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig6_human_donors.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(09_5): real output of this code on the synthetic benchmark.*

**How to read it**

Panel A: coupling falls step by step across Braak groups; B: no obvious age trend; C: the control pair is flat. These three panels together are the human replication.

| If you see... | What it means and what to do |
|---|---|
| Empty box for a Braak group | No donors in that group; merge groups or change the cut points. |

# Notebook 10 - Genetic anchoring with MAGMA (AD and PD)

| Purpose | Genetic anchoring with MAGMA: do the programs carry AD or PD common-variant risk beyond any microglia-expressed gene set? Runs partly in a terminal and needs the MAGMA binary. |
|---|---|
| Needs | GWAS summary statistics, MAGMA reference files |
| Produces | tables/10_magma_*.csv |
| Time | 30-60 min |

## Cell 1 - Reduce the GWAS file to SNP, P, N

MAGMA needs only an rsID, a p-value and a sample size. Column names differ between studies, so print the header first and edit the rename dictionary. Using rsIDs means differences in genome build between the summary statistics and the reference panel do not matter.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np
ss = pd.read_csv('../data/raw/gwas/ad_bellenguez.tsv.gz', sep='\t')
print(ss.columns.tolist()); print(ss.head(2).T)
# EDIT the two source column names after printing (rsID and p-value columns differ by study)
out = ss.rename(columns={'rsid': 'SNP', 'p_value': 'P'})[['SNP', 'P']].dropna()
out = out[out.SNP.astype(str).str.startswith('rs')]
N_EFF = None                      # <-- fill with the effective/total sample size reported by the study
assert N_EFF is not None, 'set N_EFF from the paper'
out['N'] = N_EFF
out.to_csv('../data/raw/magma/ad_sumstats.txt', sep=' ', index=False)
print(len(out), 'SNPs written')
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
['variant_id', 'rsid', 'chromosome', 'base_pair_location', 'effect_allele', 'other_allele', 'beta', 'standard...
(format example - the real header of your GWAS Catalog file will differ)
9423311 SNPs written
```

| If you see... | What it means and what to do |
|---|---|
| KeyError on rename | The header uses other names (`SNP`, `rs_id`, `pval`, `P-value`). Rename by what you printed. |
| Very few SNPs start with "rs" | The file uses chr:pos IDs. Map to rsIDs with the dbSNP map from the MAGMA site or use a harmonised version of the file from GWAS Catalog. |
| p-values written as 0 | Underflow. MAGMA tolerates small p-values; if it complains, clip at 1e-300 (`out.P = out.P.clip(1e-300)`). |

## Cell 2 - Gene sets as Entrez IDs (including the conditioning set)

MAGMA's gene location file uses Entrez IDs. Sets are written in long format (SET ENTREZ). A set of all genes detected in microglia nuclei is added so that program enrichment can be conditioned on "being expressed in microglia".

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
gl = pd.read_csv('../data/raw/magma/NCBI37.3.gene.loc', sep=r'\s+', header=None,
                 names=['entrez', 'chr', 'start', 'stop', 'strand', 'symbol'])
sym2ent = dict(zip(gl.symbol, gl.entrez))
hm = pd.read_csv('../gene_sets/modules_human.csv')
rows = [(k, sym2ent[s]) for k, s in zip(hm.module, hm.gene_human) if s in sym2ent]
import scanpy as sc
h = sc.read_h5ad('../data/raw/seaad/seaad_mtg_microglia.h5ad')
sym = h.var['feature_name'].astype(str).values if 'feature_name' in h.var else h.var_names.values
raw = h.raw.X if h.raw is not None else h.X
frac = np.asarray((raw > 0).mean(axis=0)).ravel()
expressed = [s for s, f in zip(sym, frac) if f >= 0.01 and s in sym2ent]
rows += [('MICROGLIA_EXPRESSED', sym2ent[s]) for s in expressed]
sets_df = pd.DataFrame(rows).drop_duplicates()
sets_df.to_csv('../data/raw/magma/magma_sets.txt', sep=' ', header=False, index=False)
print(sets_df.groupby(0).size().to_string())
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
AUTOPHAGY_GEN            9
LYSOSOME                27
MICROGLIA_EXPRESSED   9654
MITOPHAGY               15
OXPHOS                  17
PROTEASOME              30
RIBOSOME                44
(format example - counts depend on your data)
```

| If you see... | What it means and what to do |
|---|---|
| Few genes map to Entrez | GRCh37 gene.loc uses older symbols (e.g. GBA vs GBA1). Add an alias table, or map with `mygene`. Report the mapped fraction per set. |
| MICROGLIA_EXPRESSED is tiny | The file is a normalised subset without `.raw`; use a counts layer for the detection fraction. |

## Cells 3-4 (terminal) - MAGMA annotation and gene-level analysis

Put the MAGMA binary, `g1000_eur.bed/.bim/.fam`, `NCBI37.3.gene.loc`, the two sumstats files and `magma_sets.txt` in `data/raw/magma/`. Run in a terminal opened there. On Windows write each command on one line and use `magma.exe`.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
awk '{print $2, $1, $4}' g1000_eur.bim > g1000_eur.snploc
magma --annotate window=35,10 --snp-loc g1000_eur.snploc --gene-loc NCBI37.3.gene.loc --out annot
magma --bfile g1000_eur --pval ad_sumstats.txt use=SNP,P ncol=N \
      --gene-annot annot.genes.annot --out ad_gene
magma --bfile g1000_eur --pval pd_sumstats.txt use=SNP,P ncol=N \
      --gene-annot annot.genes.annot --out pd_gene
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
Welcome to MAGMA v1.10 ...
Start of annotation: ... 18,xxx genes ... 7,xxx,xxx SNPs mapped to at least one gene
Start of gene analysis ...
Writing gene analysis results to file ad_gene.genes.out / ad_gene.genes.raw
(format example - messages and counts differ by version and data)
```

| If you see... | What it means and what to do |
|---|---|
| magma: command not found | Give the binary a path (`./magma`) or add its folder to PATH; on Mac you may need `chmod +x magma` and to allow it in Security settings. |
| "no SNPs matched" / empty annotation | rsIDs in your file do not appear in the reference `.bim`. Check `head ad_sumstats.txt` and `head g1000_eur.bim`. |
| awk not available (Windows) | Use Git Bash, or create `g1000_eur.snploc` in pandas: read `.bim` (columns 2, 1, 4) and write with spaces. |

## Cell 5 - Remove APOE and MHC genes from the gene-level results

A single locus can create an enrichment. APOE (chr19 44-46 Mb, hg19) dominates AD signals and the MHC (chr6 25-34 Mb) is extreme in many traits. Genes in these regions are removed before the set analysis.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
def drop_regions(path_in, path_out, regions):
    keep = []
    for line in open(path_in):
        f = line.split()
        if line.startswith('#') or len(f) < 4 or not f[1].isdigit():
            keep.append(line); continue
        ch, st, sp = f[1], int(f[2]), int(f[3])
        if any(ch == str(r[0]) and st <= r[2] and sp >= r[1] for r in regions):
            continue
        keep.append(line)
    open(path_out, 'w').writelines(keep)

drop_regions('../data/raw/magma/ad_gene.genes.raw', '../data/raw/magma/ad_gene_noAPOE.genes.raw',
             [(19, 44000000, 46000000), (6, 25000000, 34000000)])
drop_regions('../data/raw/magma/pd_gene.genes.raw', '../data/raw/magma/pd_gene_noMHC.genes.raw',
             [(6, 25000000, 34000000)])
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
(no output; two new files are written)
Check: `wc -l ad_gene.genes.raw ad_gene_noAPOE.genes.raw` -> the second has a few dozen fewer lines.
```

| If you see... | What it means and what to do |
|---|---|
| Same number of lines before and after | Chromosome in your file is "chr19" or "19"? The code expects plain digits in column 2. Check `head ad_gene.genes.raw` and adapt. |

## Cells 6-7 - Competitive gene-set analysis and results table

The set test is competitive (are program genes more associated than other genes?), corrected for gene size and density by default, and conditioned on MICROGLIA_EXPRESSED. The negative-control programs (proteasome, ribosome) are analysed identically and must be shown beside the programs of interest. Check the MAGMA manual if your version words the conditioning option differently.

*Code (not run in the benchmark: needs real data, the network or a terminal) [» marks a printed continuation; the notebook holds the line unwrapped]*

```
# terminal:
# magma --gene-results ad_gene_noAPOE.genes.raw --set-annot magma_sets.txt gene-col=2 set-col=1 \
#       --model condition-hide=MICROGLIA_EXPRESSED --out ad_sets
# magma --gene-results pd_gene_noMHC.genes.raw --set-annot magma_sets.txt gene-col=2 set-col=1 \
#       --model condition-hide=MICROGLIA_EXPRESSED --out pd_sets
for trait in ['ad', 'pd']:
    r = pd.read_csv(f'../data/raw/magma/{trait}_sets.gsa.out', sep=r'\s+', comment='#')
    r['bonf'] = (r.P * len(r)).clip(upper=1)
    r.to_csv(f'../results/tables/10_magma_{trait}.csv', index=False)
    print(trait.upper()); print(r[['VARIABLE', 'NGENES', 'BETA', 'SE', 'P',
    » 'bonf']].round(4).to_string(index=False))
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
AD
  VARIABLE  NGENES   BETA     SE      P   bonf
  LYSOSOME      27   0.31   0.14 0.0121 0.0606
  MITOPHAGY     15   0.12   0.19 0.2641 1.0000
 PROTEASOME     30  -0.04   0.13 0.6190 1.0000
  RIBOSOME      44   0.02   0.11 0.4290 1.0000
(format example with INVENTED numbers - shows the layout only; your genetics may well be null)
```

| If you see... | What it means and what to do |
|---|---|
| "condition-hide" not recognised | MAGMA versions differ; check `magma --help` or the manual (section on conditional analysis) and use the equivalent option, or drop conditioning and report that you did. |
| Gene-set file read error | Check column order: `set-col=1 gene-col=2` must match `SET ENTREZ` as written in Cell 2. |
| Everything null | A legitimate result. Common-variant risk may not be enriched in these programs; say so and discuss rare variants and non-genetic routes. |

### Cell 8 - Locus overlay: do known risk genes sit in the programs?

A direct, interpretable check that does not depend on MAGMA: which Mendelian PD genes and AD risk genes are present in each program after all filters. Edit the starting lists from GWAS Catalog and PanelApp.

*Code*

```
import pandas as pd
hm = pd.read_csv('../gene_sets/modules_human.csv')
pd_genes = ['SNCA', 'LRRK2', 'PRKN', 'PINK1', 'PARK7', 'VPS35', 'GBA1', 'GBA', 'ATP13A2', 'TMEM175',
            'CTSB', 'GALC', 'SMPD1', 'ASAH1', 'VPS13C', 'DNAJC6', 'SYNJ1', 'FBXO7', 'CHCHD2']
ad_genes = ['TREM2', 'APOE', 'ABCA7', 'SORL1', 'PLCG2', 'INPP5D', 'CD33', 'BIN1',
            'PICALM', 'CLU', 'CR1', 'SPI1']
for name, gl_ in [('PD (starting list - verify against current literature)', pd_genes),
                  ('AD (starting list - verify against current literature)', ad_genes)]:
    print(name)
    for k, g in hm.groupby('module'):
        hit = sorted(set(g.gene_human) & set(gl_))
        if hit: print('  %-14s %s' % (k, hit))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
PD (starting list - verify against current literature)
  LYSOSOME       ['ASAH1', 'GALC', 'GBA1', 'SMPD1']
  MITOPHAGY      ['PARK7', 'PINK1', 'PRKN', 'VPS13C']
AD (starting list - verify against current literature)
```

| If you see... | What it means and what to do |
|---|---|
| Prints nothing for AD | The AD risk genes mostly encode innate-immune/lipid functions, not mitophagy or lysosomal genes, so little overlap is expected. This is informative. |

## Notebook 11 - Perturbation, CRISPR-screen and proteome concordance

| Purpose | Perturbation, CRISPR-screen and proteome concordance with predictions written first. Produces Figure 7a-c. |
|---|---|
| Needs | GSE179611 table, screen hits, proteome table |

| Purpose | Perturbation, CRISPR-screen and proteome concordance with predictions written first. Produces Figure 7a-c. |
|---|---|
| Produces | tables/11_*, figures/fig7abc_orthogonal.pdf |
| Time | 3 min |

### Cell 1 - Write the predictions BEFORE looking at any result

A prediction table is created first and saved; it should also be uploaded to the preregistration. Predictions must come with a one-line rationale. These two rows are examples of the format; write your own after reading the literature.

*Code*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, gseapy as gp, matplotlib.pyplot as plt
pred = pd.DataFrame([
  {'dataset': 'GSE179611 CCCP vs DMSO (LPS)', 'program': 'LYSOSOME', 'predicted': 'up',
   'rationale': 'mitochondrial damage engages TFEB/TFE3 lysosomal programs'},
  {'dataset': 'GSE179611 CCCP vs DMSO (LPS)', 'program': 'PROTEASOME', 'predicted': 'none',
   'rationale': 'negative control'}])
pred.to_csv('../results/tables/11_predictions.csv', index=False)
print(pred.to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
                    dataset    program predicted                                                rationale
GSE179611 CCCP vs DMSO (LPS)   LYSOSOME        up mitochondrial damage engages TFEB/TFE3 lysosomal programs
GSE179611 CCCP vs DMSO (LPS) PROTEASOME      none                                          negative control
```

| If you see... | What it means and what to do |
|---|---|
| Temptation to edit the table after seeing the result | Do not. The file is timestamped by Git and by the OSF registration. Report predictions that fail. |

### Cell 2 - Rank genes and run preranked GSEA for every program

Genes are ranked by sign(log fold change) x -log10 p. GSEA asks whether each program sits at the top or bottom of the ranking. Confirm the contrast direction (which group is numerator) and the species on the GEO page before interpreting the sign.

*Code*

```
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
df = pd.read_csv('../data/raw/GSE179611_DMSOLPS_vs_CCCPLPS-ExpDiff.csv.gz')
print(df.columns.tolist())
df = df.rename(columns={'Gene': 'gene', 'logFC': 'log2FC', 'PValue': 'pvalue'})    # EDIT to your columns
df['rank'] = np.sign(df.log2FC) * -np.log10(df.pvalue.clip(lower=1e-300))
rnk = df.dropna(subset=['gene', 'rank']).drop_duplicates('gene')
rnk = rnk.set_index('gene')['rank'].sort_values(ascending=False)
pre = gp.prerank(rnk=rnk, gene_sets=modules, permutation_num=1000, min_size=5, max_size=1000,
                 seed=0, outdir=None, verbose=False, threads=1)
print(pre.res2d[['Term', 'ES', 'NES', 'NOM p-val', 'FDR q-val']].round(3).to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
['Gene', 'logFC', 'PValue']
         Term     ES    NES  NOM p-val  FDR q-val
     LYSOSOME  0.757  2.109      0.000      0.000
    MITOPHAGY  0.567  1.365      0.106      0.182
       OXPHOS -0.389 -0.997      0.444      0.920
AUTOPHAGY_GEN -0.442 -0.986      0.470      0.478
     RIBOSOME  0.274  0.838      0.746      0.948
   PROTEASOME  0.232  0.652      0.951      0.937
```

#### How to read it

LYSOSOME is strongly enriched among genes up in CCCP (NES 2.11, FDR < 0.001); MITOPHAGY only weakly (NES 1.37, FDR 0.18); controls are null. This matches the planted truth.

| If you see... | What it means and what to do |
|---|---|
| Most genes not found in the gene sets | Species mismatch (human symbols vs mouse sets). Convert with the ortholog table (Notebook 09, Cell 1). |
| Column names do not match | Print `df.columns` and edit the rename dictionary; some files use `log2FoldChange`, `padj`, `FDR`. |
| gseapy raises about duplicated gene names | Drop duplicates before ranking, as done here. |

### Cell 3 - A second, assumption-light test: mean rank metric versus random sets

GSEA depends on its enrichment statistic. As a cross-check, the mean rank metric of each program is compared with 5,000 random gene sets of the same size (z-score and empirical p). Agreement between the two methods is the evidence; disagreement is a reason for caution.

*Code*

```
def set_z(metric, genes, n_perm=5000, seed=0):
    rng = np.random.default_rng(seed)
    g = [x for x in genes if x in metric.index]
    obs = metric.loc[g].mean()
    seeds = rng.integers(0, 10**9, n_perm)
    null = np.array([metric.sample(len(g), random_state=int(s)).mean() for s in seeds])
    dev = np.abs(null - null.mean())
    p = (np.sum(dev >= abs(obs - null.mean())) + 1) / (n_perm + 1)
    return obs, (obs - null.mean()) / null.std(), p

rows = []
for k, genes in modules.items():
    o, z, p = set_z(rnk, genes)
    rows.append({'dataset': 'GSE179611_CCCP', 'program': k, 'mean_rank_metric': o, 'z': z, 'p': p})
pert = pd.DataFrame(rows)
pert.to_csv('../results/tables/11_perturbation_CCCP.csv', index=False)
print(pert.round(4).to_string(index=False))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
       dataset       program  mean_rank_metric       z      p
GSE179611_CCCP AUTOPHAGY_GEN           -0.4406 -0.8707 0.3717
GSE179611_CCCP      LYSOSOME            2.1116  6.0785 0.0002
GSE179611_CCCP     MITOPHAGY            0.7624  1.5680 0.1116
GSE179611_CCCP        OXPHOS           -0.3509 -0.9588 0.3329
GSE179611_CCCP    PROTEASOME           -0.0526 -0.3742 0.6933
GSE179611_CCCP      RIBOSOME            0.1582  0.3158 0.7522
```

#### How to read it

The second method agrees: LYSOSOME z = 6.1 (p = 0.0002), MITOPHAGY z = 1.6 (p = 0.11), controls near 0. Agreement between GSEA and this test is the evidence.

| If you see... | What it means and what to do |
|---|---|
| Slow (minutes) | 5,000 permutations x 6 programs; reduce to 1,000 for testing. |

### Cell 4 - CRISPR-screen overlap (hypergeometric test)

Are program genes over-represented among genes whose knock-out/knock-down changed a microglial phenotype? The background (universe) must be the genes the screen actually tested.

*Code*

```
from scipy.stats import hypergeom
hm = pd.read_csv('../gene_sets/modules_human.csv')
scr = pd.read_csv('../data/raw/screens/screen_hits.csv')            # columns: gene, score, fdr
scr['gene'] = scr.gene.astype(str).str.upper()
hits = set(scr.loc[scr.fdr < 0.1, 'gene']); universe = set(scr.gene)
rows = []
for k, g in hm.groupby('module'):
    s = set(g.gene_human.str.upper()) & universe
    x = len(s & hits)
    p = hypergeom.sf(x - 1, len(universe), len(hits), len(s))
    rows.append({'program': k, 'in_universe': len(s), 'screen_hits': x,
                 'expected': round(len(s) * len(hits) / len(universe), 1), 'p_hypergeom': p})
scr_df = pd.DataFrame(rows)
scr_df.to_csv('../results/tables/11_screen_overlap.csv', index=False)
print('screen: %d genes tested, %d hits (FDR < 0.1)' % (len(universe), len(hits)))
print(scr_df.to_string(index=False, float_format=lambda x: '%.3g' % x))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
screen: 3200 genes tested, 181 hits (FDR < 0.1)
      program  in_universe  screen_hits  expected  p_hypergeom
AUTOPHAGY_GEN           10            0       0.6            1
     LYSOSOME           30           12       1.7     2.64e-08
    MITOPHAGY           17            1         1        0.629
       OXPHOS           18            1         1         0.65
   PROTEASOME           31            0       1.8            1
     RIBOSOME           45            3       2.5        0.473
```

### How to read it

Screen hits: 12 LYSOSOME genes among the hits versus 1.7 expected by chance (p = 2.6e-08); no enrichment for the others.

| If you see... | What it means and what to do |
|---|---|
| p-values of exactly 1 | `s & hits` is empty; check symbol case (screens often use upper-case) and the FDR column name. |
| Hits defined by a different threshold | Use the screen's own recommended threshold and say so; do not tune it to maximise overlap. |

## Cell 5 - Protein-level concordance

Do the programs change in the same direction at protein level in aged microglia? The same random-set z-test is applied to a per-gene protein log2 fold change (old vs young). Genes quantified in proteomics are biased toward abundant proteins, so the random sets are drawn only from quantified proteins.

*Code*

```
prot = pd.read_csv('../data/raw/proteomics/aged_microglia_proteome.csv')    # gene, log2fc, p
lfc = prot.dropna(subset=['gene', 'log2fc']).drop_duplicates('gene').set_index('gene')['log2fc']
rows = []
for k, genes in modules.items():
    o, z, p = set_z(lfc, genes)
    rows.append({'program': k, 'n_quantified': len([x for x in genes if x in lfc.index]),
                 'mean_log2fc_old_vs_young': o, 'z': z, 'p': p})
prot_df = pd.DataFrame(rows); prot_df.to_csv('../results/tables/11_proteome_concordance.csv', index=False)
print(prot_df.to_string(index=False, float_format=lambda x: '%.3g' % x))
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
      program  n_quantified  mean_log2fc_old_vs_young      z     p
AUTOPHAGY_GEN            10                    0.0541  0.573 0.559
     LYSOSOME            30                     0.146   2.62 0.008
    MITOPHAGY            17                   -0.0468 -0.641 0.523
       OXPHOS            18                    0.0599  0.845 0.401
   PROTEASOME            31                    0.0787   1.43  0.16
     RIBOSOME            45                    0.0651   1.43 0.156
```

### How to read it

Proteome: LYSOSOME proteins are up in old (z = 2.6, p = 0.008); other programs are not.

| If you see... | What it means and what to do |
|---|---|
| Only a few program genes quantified | Normal for membrane/low-abundance proteins. Report n_quantified; interpret programs with fewer than 5 quantified genes as not testable. |

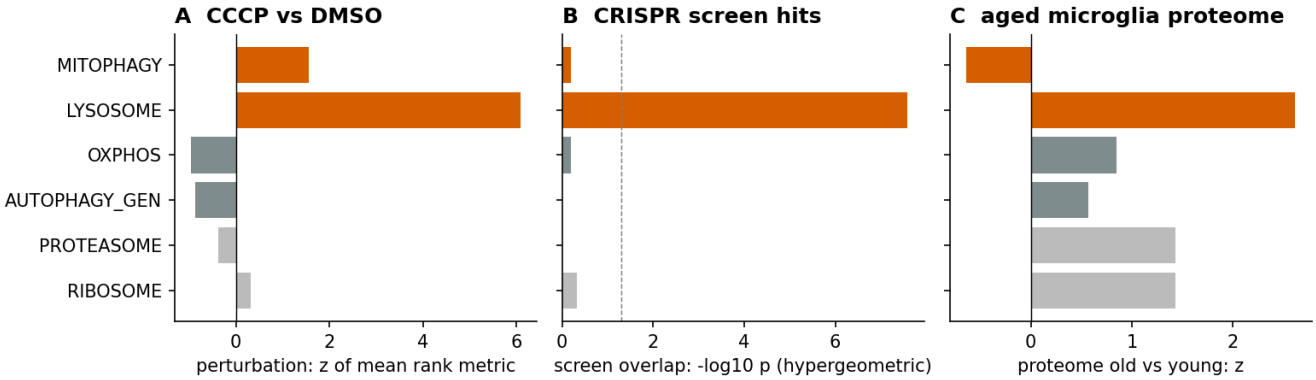
### Cell 6 - Figure 7a-c: perturbation, screen and proteome concordance

One panel per orthogonal line of evidence, each showing the programs of interest next to the negative-control programs.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
order = ['MITOPHAGY', 'LYSOSOME', 'OXPHOS', 'AUTOPHAGY_GEN', 'PROTEASOME', 'RIBOSOME']
colr = ['#D55E00', '#D55E00', '#7f8c8d', '#7f8c8d', '#bbbbbb', '#bbbbbb']
fig, ax = plt.subplots(1, 3, figsize=(10.5, 3.2), sharey=True)
a0 = pert.set_index('program').loc[order]
ax[0].barh(order[::-1], a0.z[::-1], color=colr[::-1]); ax[0].axvline(0, color='black', lw=.7)
ax[0].set_xlabel('perturbation: z of mean rank metric'); ax[0].set_title('A  CCCP vs DMSO', loc='left',
» fontweight='bold')
a1 = scr_df.set_index('program').loc[order]
ax[1].barh(order[::-1], -np.log10(a1.p_hypergeom[::-1]),
» color=colr[::-1]); ax[1].axvline(-np.log10(0.05), color='grey', ls='--', lw=.7)
ax[1].set_xlabel('screen overlap: -log10 p (hypergeometric)'); ax[1].set_title('B  CRISPR screen hits',
» loc='left', fontweight='bold')
a2 = prot_df.set_index('program').loc[order]
ax[2].barh(order[::-1], a2.z[::-1], color=colr[::-1]); ax[2].axvline(0, color='black', lw=.7)
ax[2].set_xlabel('proteome old vs young: z'); ax[2].set_title('C  aged microglia proteome', loc='left',
» fontweight='bold')
for a_ in ax: a_.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig7abc_orthogonal.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(11_6): real output of this code on the synthetic benchmark.*

**How to read it**

One orthogonal line of evidence per panel, programs of interest in orange next to the controls in grey. Only LYSOSOME stands out in all three here; MITOPHAGY stays modest (as planted).

## Notebook 12 - Evidence integration and network

| Purpose | Integrates all lines of evidence into a transparent, weight-robust ranking of candidates; optional STRING network. Produces Figure 7d. |
|---|---|
| Needs | tables from 06, 08, 10, 11 |
| Produces | tables/12_evidence_rubric.csv, figures/fig7d_evidence.pdf |
| Time | 1 min |

## Cell 1 - Assemble the evidence matrix

One row per candidate gene (program genes plus the top genome-wide genes), one column per independent line of evidence, each converted to a 0-1 percentile so that lines are comparable. A gene absent from a table gets NaN for that line (it is not penalised).

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
import pandas as pd, numpy as np, os, matplotlib.pyplot as plt
mods_df = pd.read_csv('../gene_sets/modules.csv')
modules = {k: g['gene_mouse'].tolist() for k, g in mods_df.groupby('module')}
loo = pd.read_csv('../results/tables/06_leave_one_gene_out.csv').set_index('gene_removed')
de = pd.read_csv('../results/tables/08_pseudobulk_DE_old_vs_young.csv', index_col=0)
gw = pd.read_csv('../results/tables/08_genomewide_axis_change.csv', index_col=0)
cccp = pd.read_csv('../data/raw/GSE179611_DMSOLPS_vs_CCCPLPS-ExpDiff.csv.gz').set_index('Gene')
scr = pd.read_csv('../data/raw/screens/screen_hits.csv')
prot = pd.read_csv('../data/raw/proteomics/aged_microglia_proteome.csv').set_index('gene')
orth = pd.read_csv('../data/processed/mouse_human_orthologs.csv')
top_gw = set(gw.sort_values('p').head(100).index)
cand = sorted(set(modules['MITOPHAGY']) | set(modules['LYSOSOME']) | top_gw)
pct = lambda s: s.rank(pct=True)
ev = pd.DataFrame(index=cand)
ev['coupling_influence'] = pct(loo['shift'].abs().reindex(cand))
ev['age_DE'] = pct(de['log2FoldChange'].abs().reindex(cand))
ev['axis_change'] = pct(gw['z_stouffer'].abs().reindex(cand))
ev['cccp_response'] = pct((np.sign(cccp.logFC) *
» -np.log10(cccp.PValue.clip(lower=1e-300))).abs().reindex(cand))
hits = set(scr.loc[scr.fdr < 0.1, 'gene'])
m2h = dict(zip(orth.mouse, orth.human))
ev['screen_hit'] = [float(m2h.get(g, '').upper() in hits) if g in m2h else np.nan for g in cand]
ev['protein_change'] = pct(prot['log2fc'].abs().reindex(cand))
if os.path.exists('../data/processed/collectri_mouse.csv'):         # added when Notebook 08 Cell 1 was
» run
    net = pd.read_csv('../data/processed/collectri_mouse.csv')
    tgt = set(net[net.source.str.lower().isin(['tfeb', 'tfe3', 'mitf'])].target.str.capitalize())
    ev['tfeb_target'] = [1.0 if g in tgt else 0.0 for g in cand]
print(ev.shape, '| evidence lines:', ev.columns.tolist())
print(ev.notna().sum().to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
(147, 6) | evidence lines: ['coupling_influence', 'age_DE', 'axis_change', 'cccp_response', 'screen_hit', 'pr...
coupling_influence     47
age_DE                147
axis_change           100
cccp_response         147
screen_hit            147
protein_change        147
```

| If you see... | What it means and what to do |
|---|---|
| Gene symbols do not align between tables | Tables from different species/sources use different case. Convert all to one symbol system before reindexing (ortholog table in Notebook 09). |
| Few genes have 3 or more lines | Add lines (GWAS gene-level p from MAGMA `*.genes.out`, other perturbation datasets) or report that the integration is under-powered. |

## Cell 2 - Score, require at least three lines, test robustness to weights

The score is the mean of available lines. Because any weighting is arbitrary, 1,000 random weight vectors (Dirichlet) re-rank the genes; the fraction of draws in which a gene stays in the top 20 is reported. Genes that are top-ranked only under one weighting are not trustworthy.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
lines = ev.columns.tolist()
ev['n_lines'] = ev[lines].notna().sum(axis=1)
ev['score'] = ev[lines].mean(axis=1, skipna=True)
ev = ev[ev.n_lines >= 3].sort_values('score', ascending=False)
rng = np.random.default_rng(0); hit = pd.Series(0, index=ev.index)
filled = ev[lines].fillna(ev[lines].mean())
for _ in range(1000):
    w = rng.dirichlet(np.ones(len(lines)))
    order = np.argsort(-(filled.values @ w))[:20]
    hit[ev.index[order]] += 1
ev['top20_frequency'] = hit / 1000
ev.to_csv('../results/tables/12_evidence_rubric.csv')
show = ev[['n_lines', 'score', 'top20_frequency']].head(12)
print(show.round(3).to_string())
if 'Tfeb' in ev.index:                # SIMULATION ONLY
    print('planted regulators:', {g: int(ev.index.get_loc(g)) + 1 for g in ['Tfeb', 'Tfe3', 'Mitf',
    » 'Ppargc1a'] if g in ev.index})
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
          n_lines  score  top20_frequency
Lamp1           5  0.819            0.915
Ids             5  0.781            0.852
Usp30           5  0.757            0.780
Cln5            5  0.745            0.785
Fuca1           5  0.737            0.743
Ppt1            5  0.722            0.695
Gene0212        5  0.710            0.681
Atp6v1c1        5  0.709            0.628
Npc2            5  0.655            0.615
Spp1            5  0.654            0.678
Trem2           5  0.648            0.680
Bcl2l13         5  0.647            0.631
planted regulators: {'Tfeb': 19, 'Tfe3': 29, 'Mitf': 41, 'Ppargc1a': 23}
```

**How to read it**

Lysosome genes (Lamp1, Ids, Cln5...) rank first because they have all five evidence lines. The planted regulators rank 19-41 because only three or four lines cover them: the rubric favours genes with many lines, a known bias. Say so in the paper, and report top20_frequency, not just rank.

| If you see... | What it means and what to do |
|---|---|
| Ranking changes completely with the weights | Then the evidence is weak or inconsistent. Report the stable genes only (top20_frequency > 0.8) and say the rest are not prioritised. |
| LinAlgError / shape errors in the matrix product | `filled` has NaN columns (a line with no data). Drop all-NaN columns first. |

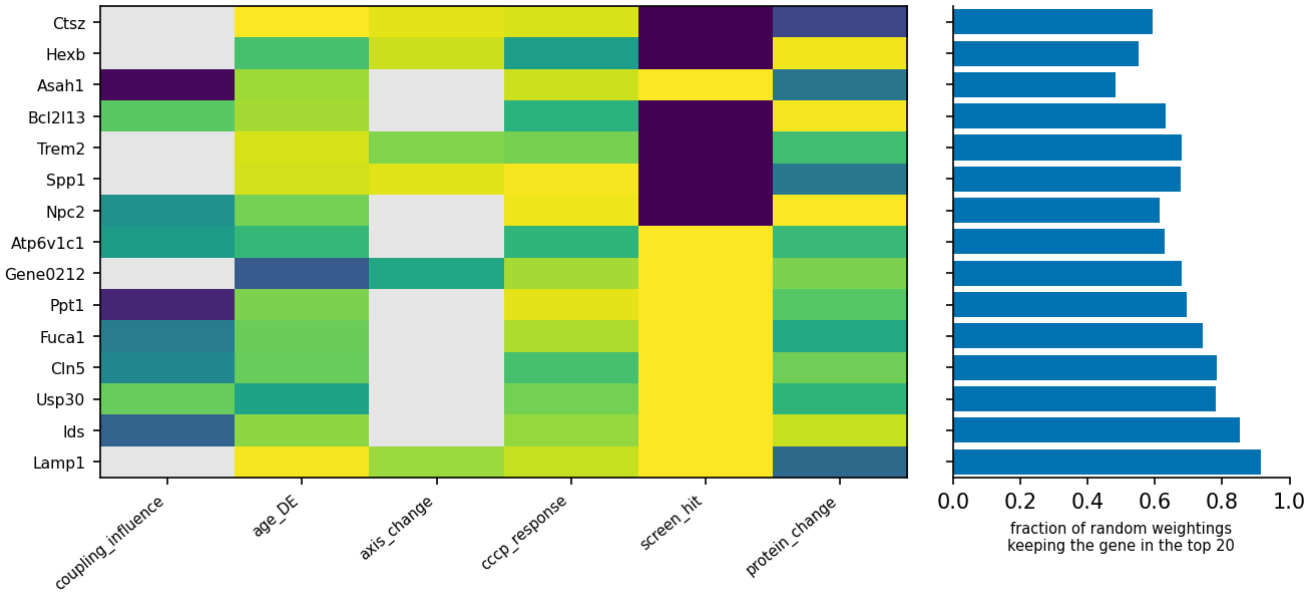
## Cell 3 - Figure 7d: evidence heat map and ranking stability

Rows are the top candidates, columns are evidence lines (percentile); grey = no data for that gene. The right panel shows how often each gene stays in the top 20 under random weights.

*Code*

```
top = ev.head(15)
M = np.ma.masked_invalid(top[lines].values.astype(float))
cmap = plt.get_cmap('viridis').copy(); cmap.set_bad('#e5e5e5')
fig, ax = plt.subplots(1, 2, figsize=(9, 4.2), gridspec_kw={'width_ratios': [2.4, 1]}, sharey=True)
ax[0].imshow(M, aspect='auto', cmap=cmap, vmin=0, vmax=1)
ax[0].set_yticks(range(len(top))); ax[0].set_yticklabels(top.index, fontsize=7.5)
ax[0].set_xticks(range(len(lines))); ax[0].set_xticklabels(lines, rotation=40, ha='right', fontsize=7)
ax[1].barh(range(len(top)), top.top20_frequency, color='#0072B2'); ax[1].set_xlim(0, 1)
ax[1].set_xlabel('fraction of random weightings\nkeeping the gene in the top 20', fontsize=7)
ax[0].invert_yaxis(); ax[1].spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig7d_evidence.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**




***Expected figure*** *(12_3): real output of this code on the synthetic benchmark.*

**How to read it**

Rows are candidates, columns evidence lines; grey cells mean "no data for this gene". Bars on the right show stability under random weights: a bar near 1.0 is a robust candidate.

## Cell 4 - STRING network (descriptive only; needs internet)

The STRING API does exactly what the website does, but reproducibly. Because the genes were chosen from related pathways, dense connectivity is expected by construction; the network is for illustration and for betweenness-based hub labelling, never as evidence.

*Code (not run in the benchmark: needs real data, the network or a terminal)*

```
import requests, io, networkx as nx
top30 = ev.head(30).index.tolist()
r = requests.post('https://string-db.org/api/tsv/network',
                  data={'identifiers': '\r'.join(top30), 'species': 10090, 'required_score': 700,
                        'caller_identity': 'mgcoupling_project'}, timeout=120)
edges = pd.read_csv(io.StringIO(r.text), sep='\t')
G = nx.from_pandas_edgelist(edges, 'preferredName_A', 'preferredName_B', edge_attr='score')
bc = pd.Series(nx.betweenness_centrality(G)).sort_values(ascending=False)
print(len(G.nodes), 'nodes,', len(G.edges), 'edges'); print(bc.head(8).round(3).to_string())
edges.to_csv('../results/tables/12_string_edges.tsv', sep='\t', index=False)
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
26 nodes, 61 edges
Lamp1       0.214
Ctsd        0.168
Tfeb        0.151
Pink1       0.120
...
(format example with invented numbers - requires internet)
```

| If you see... | What it means and what to do |
|---|---|
| Empty response / HTTP 4xx | Species code must be 10090 (mouse); lower `required_score` to 400; or use the website (Multiple proteins > Mus musculus > Exports). |
| Hairball network | Normal for pathway genes. Increase confidence to 900, label only nodes with the top betweenness, and avoid reading edges as evidence. |

## Cell 5 - Functional enrichment of the genome-wide hits with an explicit background

Enrichment of the genome-wide association genes (not the program genes, which would be circular) with a stated background of every gene that could have been tested. Needs Enrichr libraries from the internet in real use, so this

cell is shown as format only.

*Code (not run in the benchmark: needs real data, the network or a terminal) [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import gseapy as gp
hits = gw[gw.fdr < 0.1].index.tolist()
bg = gw.index.tolist()                                    # all genes that were testable
enr = gp.enrichr(gene_list=hits, gene_sets=['GO_Biological_Process_2021', 'KEGG_2019_Mouse'],
                 organism='Mouse', background=bg, outdir=None)
print(enr.results.sort_values('Adjusted P-value').head(8)[['Gene_set', 'Term',
» 'Adjusted P-value']].to_string(index=False))
```

**FORMAT EXAMPLE - schematic, NOT real results (this step needs real data or the internet)**

```
Gene_set                    Term                                        Adjusted P-value
GO_Biological_Process_2021  positive regulation of autophagy (GO:0010508)            0.012
KEGG_2019_Mouse             Lysosome                                                  0.031
(format example with invented terms - requires internet; run it on your real hits)
```

| If you see... | What it means and what to do |
|---|---|
| Background error / zero overlap | Gene symbols in `bg` must be the symbols used by the library (mouse title-case). Without `background=` Enrichr uses the whole genome and inflates significance. |

## Notebook 13 - Failure-mode gallery: what artefacts look like

| Purpose | Optional gallery of failure modes: a true effect, no effect, a depth artefact and a composition artefact, analysed with the same code. Teaches you to recognise artefacts. |
|---|---|
| Needs | sim.py only |
| Produces | figures/fig_failure_modes.pdf |
| Time | 5-8 min |

### Cell 1 - Scenario simulator with a quick pipeline

Four synthetic worlds are generated, each with 3 cohorts: (1) a true decoupling, (2) no effect, (3) a technical depth artefact (old animals have lower depth, so more dropout, but the same latent coupling), (4) a composition artefact (old animals are DAM-rich, the latent coupling is unchanged). The same analysis is run on each. Contaminants and low-quality cells are removed using the planted labels to keep this gallery short.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import sys; sys.path.append('../src')
from mgcoupling.utils import *
from mgcoupling.sim import *
import scanpy as sc, pandas as pd, numpy as np, matplotlib.pyplot as plt

def prep(ad):
    ad.layers['counts'] = ad.X.copy()
    sc.pp.calculate_qc_metrics(ad, inplace=True, percent_top=None)
    ad.obs['log_counts'] = np.log1p(ad.obs.total_counts); ad.obs['n_genes'] = ad.obs.n_genes_by_counts
    ad.obs['state'] = ad.obs['state_true'].astype(str)
    ok = ~ad.obs.state.isin(['BAM', 'neuron']) & ~ad.obs.lowq_true.astype(bool)
    ad = ad[ok.values].copy()
    sc.pp.normalize_total(ad, target_sum=1e4); sc.pp.log1p(ad)
    return ad

wt = lambda ages, g: [(a, g, 'WT', s, 1) for a, s in zip(ages, 'MFMFMF')]
design = wt([3] * 4, 'young') + wt([18] * 4, 'old')
scen = {
  '1 true decoupling':   dict(rho_old=0.20),
  '2 no effect':         dict(rho_old=0.50),
  '3 depth artefact':    dict(rho_old=0.50, depth_by_group={'old': 0.40}),
  '4 composition artefact': dict(rho_old=0.50, state_p_by_group={'old': [0.45, 0.45, 0.05, 0.03, 0.02]}),
}
mods = {'MITOPHAGY': [g for g in mitophagy if g != 'Ulk1'],
» 'LYSOSOME': [g for g in lysosome if g not in ('Ctsd', 'Lamp1', 'Ctsz')],
        'PROTEASOME': proteasome}
data = {}
for sname, kw in scen.items():
    data[sname] = []
    for k in range(3):
        ad, _ = simulate_cohort('c%d' % k, design, 0.50, kw['rho_old'], 600, 100 + k,
                                depth_by_group=kw.get('depth_by_group'),
                                » state_p_by_group=kw.get('state_p_by_group'))
        data[sname].append(prep(ad))
print({s: [a.n_obs for a in v] for s, v in data.items()})
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
{'1 true decoupling': [4453, 4469, 4458], '2 no effect': [4455, 4446, 4463], '3 depth artefact': [4468, 4413,...
```

| If you see... | What it means and what to do |
|---|---|
| MemoryError | Reduce cells per animal (600) or the number of cohorts per scenario (range(3)). |
| Takes several minutes | 12 cohorts are simulated; this is normal on a laptop CPU. |

## Cell 2 - The same analysis on all four worlds

For every scenario: the primary pooled change in coupling, the change within homeostatic cells only (composition check), and the change for a negative-control pair (specificity check). A trustworthy decoupling must show a negative primary effect, a persistent effect inside homeostatic cells, and a flat control.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
def ivw(effs):
    e = np.array([x[0] for x in effs]); s = np.array([x[1] for x in effs]); w = 1 / s ** 2
    return float((w * e).sum() / w.sum()), float(np.sqrt(1 / w.sum()))

def pooled(ads, p, q, state=None, min_cells=100):
    out = []
    for a in ads:
        ac, eff = cohort_effect(a, mods[p], mods[q], state=state, min_cells=min_cells)
        if eff is not None: out.append(eff[:2])
    return ivw(out) if out else (np.nan, np.nan)

rows, acs = [], {}
for sname, ads in data.items():
    main = pooled(ads, 'MITOPHAGY', 'LYSOSOME')
    homeo = pooled(ads, 'MITOPHAGY', 'LYSOSOME', state='homeostatic')
    ctrl = pooled(ads, 'MITOPHAGY', 'PROTEASOME')
    rows.append({'scenario': sname, 'primary': main[0], 'primary_se': main[1],
    » 'homeostatic_only': homeo[0],
                 'homeo_se': homeo[1], 'control_pair': ctrl[0], 'ctrl_se': ctrl[1]})
    acs[sname] = pd.concat([cohort_effect(a, mods['MITOPHAGY'], mods['LYSOSOME'])[0].assign(c=i) for i,
    » a in enumerate(ads)])
gal = pd.DataFrame(rows)
print(gal.set_index('scenario')[['primary', 'homeostatic_only', 'control_pair']].round(3).to_string())
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

```
                        primary  homeostatic_only  control_pair
scenario
1 true decoupling        -0.136            -0.155        -0.036
2 no effect              -0.005             0.004        -0.032
3 depth artefact         -0.076            -0.088        -0.042
4 composition artefact    0.023            -0.020        -0.048
```

### How to read it

Compare each primary effect with its negative control, not with zero. (1) True decoupling: primary -0.136, homeostatic-only -0.155, control -0.036. (2) No effect: all near 0. (3) Depth artefact: primary -0.076 and the control moves too (-0.042). (4) Composition artefact: overall +0.023 but homeostatic-only -0.020, sign flips inside states. These artefacts are subtle here; in real data they can be larger. The diagnostic PATTERN is the lesson.

| If you see... | What it means and what to do |
|---|---|
| NaN for the homeostatic-only effect | Too few animals pass `min_cells` inside the state; lower `min_cells` (not below ~30) or report as not estimable. |

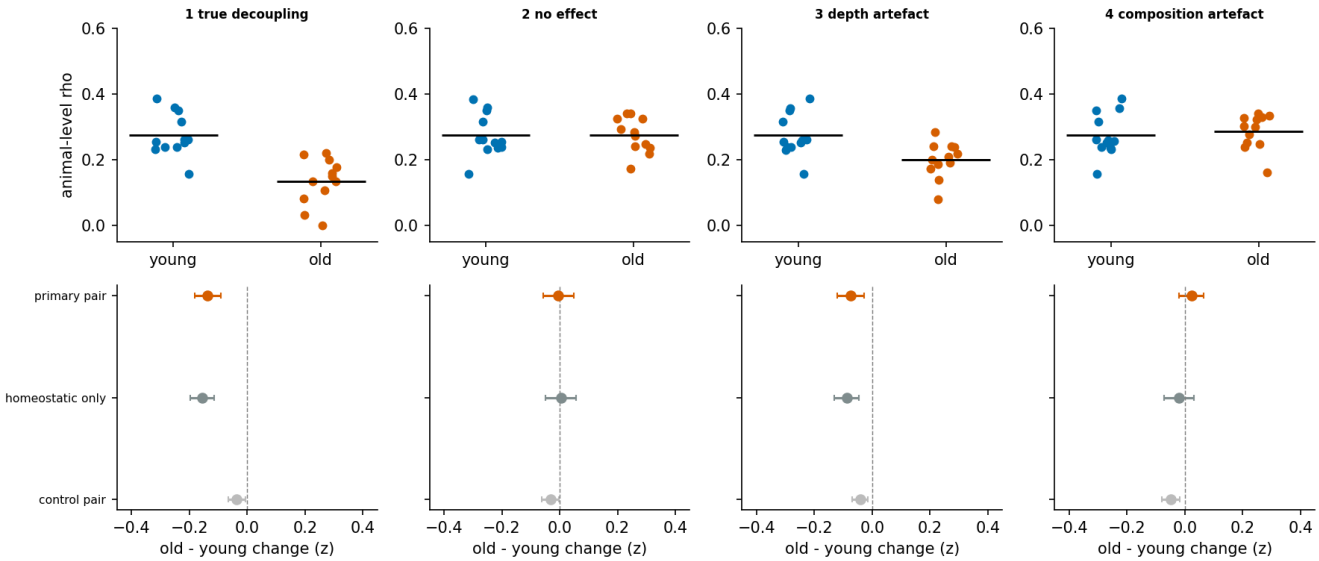
## Cell 3 - Figure: how real effects and artefacts differ

Each column is one world. Top: animal-level coupling (one dot per animal). Bottom: pooled change with 95% CI for the primary pair, the homeostatic-only analysis and the negative-control pair. A real effect is negative everywhere except the control; the depth artefact moves the control as well; the composition artefact vanishes inside homeostatic cells.

*Code [» marks a printed continuation; the notebook holds the line unwrapped]*

```
fig, ax = plt.subplots(2, 4, figsize=(12, 5.2), gridspec_kw={'height_ratios': [1, 1.05]})
for j, (sname, a) in enumerate(acs.items()):
    for k, g in enumerate(['young', 'old']):
        v = a[a.age_group == g].rho
        ax[0, j].scatter(np.full(len(v), k) + np.random.default_rng(j).uniform(-.12, .12, len(v)), v,
        » s=22,
                         color=['#0072B2', '#D55E00'][k])
        ax[0, j].hlines(v.mean(), k - .3, k + .3, color='black', lw=1.4)
    ax[0, j].set_xticks([0, 1]); ax[0, j].set_xticklabels(['young', 'old']); ax[0, j].set_title(sname,
    » fontsize=8, fontweight='bold')
    ax[0, j].set_ylim(-0.05, 0.6)
    r = gal.iloc[j]
    for i, (lab, e, s) in enumerate([('primary pair', r.primary, r.primary_se), ('homeostatic only',
    » r.homeostatic_only, r.homeo_se),
                                     ('control pair', r.control_pair, r.ctrl_se)]):
        ax[1, j].errorbar(e, 2 - i, xerr=1.96 * s, fmt='o', color=['#D55E00', '#7f8c8d', '#bbbbbb'][i],
        » capsize=2)
    ax[1, j].set_yticks([2, 1, 0]); ax[1, j].set_yticklabels(['primary pair', 'homeostatic only',
    » 'control pair'] if j == 0 else [], fontsize=7.5)
    ax[1, j].axvline(0, color='grey', lw=.7, ls='--'); ax[1, j].set_xlim(-0.45, 0.45); ax[1,
    » j].set_xlabel('old - young change (z)')
ax[0, 0].set_ylabel('animal-level rho')
for a_ in ax.ravel(): a_.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.savefig('../results/figures/fig_failure_modes.pdf'); plt.show()
```

**EXPECTED OUTPUT - real output of this exact code on the synthetic benchmark (planted truth known)**

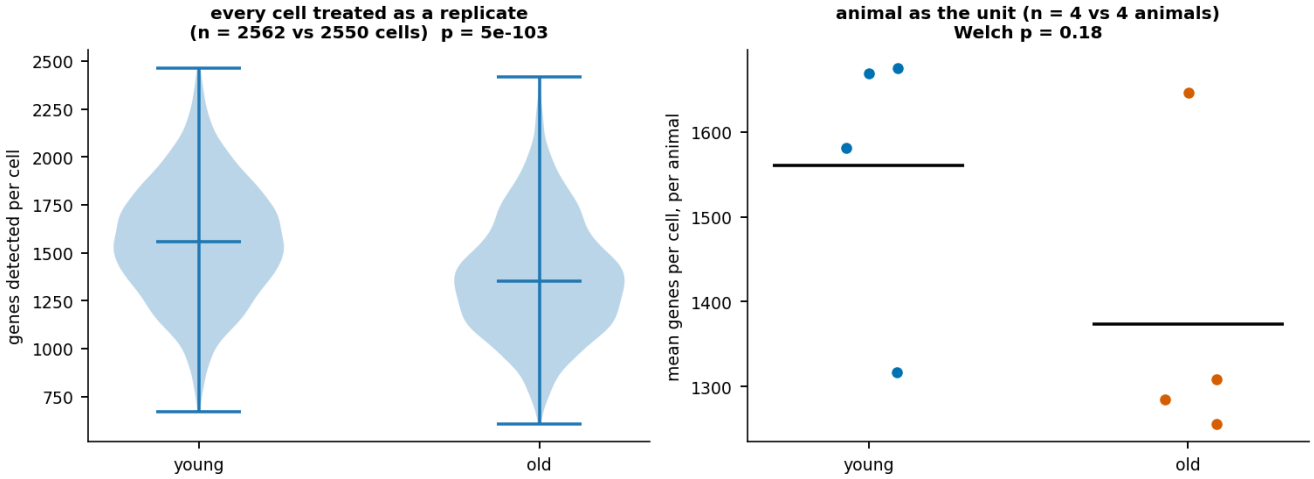



***Expected figure*** *(13_3): real output of this code on the synthetic benchmark.*

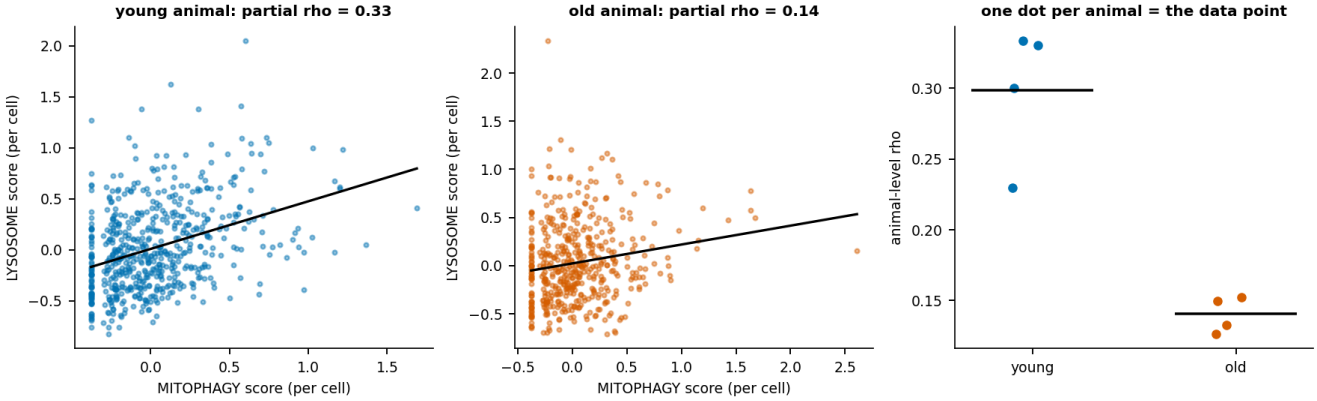
### How to read it

Top row: animal dots. Bottom row: three estimates per world. A real effect is negative in the first two rows of each panel and the control stays near its background; the depth artefact leaves a smaller effect that is not specific; the composition artefact disappears inside homeostatic cells.

# Part 5. Reading your results: real effect or artefact?




***Why animals, not cells, are the replicates.*** *Real numbers from the synthetic data: treating cells as replicates gives p = 5e-103 for a difference in genes detected per cell, while the animal-level comparison gives p = 0.18. Technical variation between animals is invisible to a cell-level test.*

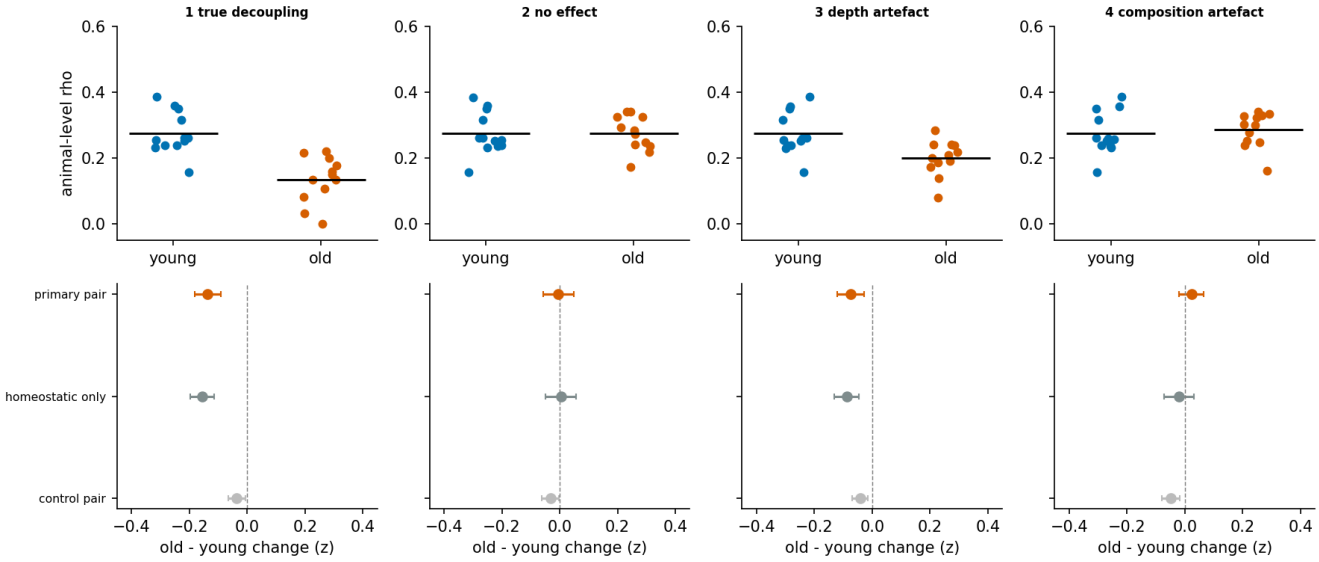



***What "coupling" is.*** *Left and middle: cell-level MITOPHAGY and LYSOSOME scores in one young and one old animal (homeostatic cells), with partial Spearman rho after removing depth effects. Right: the animal-level rho values that enter the statistics (one dot = one animal).*

## 5.1 Decision table

| Pattern in your results | Most likely meaning | What to do |
|---|---|---|
| Primary effect negative; controls near zero; persists within homeostatic cells; CI of pooled effect excludes 0; same sign in >= 75% of cohorts | A specific, intrinsic change in coordination | Proceed to humans, genetics, perturbation (Notebooks 09-12) |
| Primary effect and control pairs move together | Global change in coordination or a technical artefact (depth, dropout) | Check depth covariates and pseudo-cells; report as non-specific (title 4 in Part 9) |
| Overall effect present but vanishes within states | Composition artefact | Report state-linked or composition-driven change |
| Overall effect has opposite sign inside states | Simpson-type composition effect | Report within-state estimates as primary |
| One cohort disagrees strongly (I-squared > 75%) | Platform, region or sex differences | Examine and report; do not pool blindly |
| Matched-null p is large although meta-analysis is "significant" | The change is not specific to these programs | Treat as global; find what differs between cohorts |
| Effect only in the pooled-sample cohort | Batch confound | Do not rely on it |

Notebook 13 reproduces these patterns on synthetic data so that you can recognise them. In that gallery the artefacts are subtle; in real data they may be larger, which is why the specificity (06), state (07) and null (05) analyses are all mandatory.




***Failure-mode gallery*** *(Notebook 13, Cell 3; real output on synthetic worlds). Column 1 a true decoupling, 2 no effect, 3 a depth artefact, 4 a composition artefact. Judge each primary effect against its control pair and its within-homeostatic estimate.*

# Part 6. Optional but decisive: a minimal wet-lab validation module

Computational evidence reaches a ceiling around good mid-tier journals. One small, well-controlled experimental module is what changes the editors' reading from "analysis of public data" to "tested prediction". You do not need to run it alone: collaborate with a lab that keeps aged mice or primary microglia (this could be a PBioC/Frankfurt group, a Pakistani partner lab, or a core facility), and offer co-authorship in exchange for access.

| Experiment | Design (animals are the unit) | Readout | Prediction to preregister |
|---|---|---|---|
| E1 In situ | Young (3 mo) vs old (22-24 mo) WT mouse brain sections, n >= 5 per group, blinded imaging | IBA1+ cells: LAMP1 and TOMM20 colocalisation (mitochondria in lysosomes); LAMP1 volume per cell | State direction of the mito-lysosome overlap and lysosome volume in old microglia |
| E2 Primary microglia | Acutely isolated microglia from young vs old mice, n >= 4 per group | qPCR of 8-10 top rubric genes; TFEB nuclear fraction; LysoTracker and DQ-BSA signal | Top genes change in the direction found by scRNA-seq; TFEB activity matches Notebook 08 |
| E3 Challenge | Same cells +/- CCCP and +/- bafilomycin, 2 doses, time course | Western: PINK1 stabilisation, LC3-II, p62, LAMP1, CTSD; mt-Keima or mito-QC if a reporter line is available | Directional predictions from Notebook 11 |
| E4 Human | iPSC-derived microglia (collaborator), +/- CCCP/bafilomycin | RNA-seq or qPCR of program genes; lysosomal function assays | Same sign as human donor-level analysis |

- Preregister experiments on OSF with sample sizes, exclusion rules and predictions; analyse with animal as the unit (mixed models with random intercept per animal when multiple measures per animal).
- Animal work requires institutional ethics approval; report according to the ARRIVE 2.0 guidelines. Do not start without a named responsible licence holder.
- Even 3-4 concordant, blinded, pre-specified readouts change reviewers' view substantially. Report discordant ones too.

# Part 7. Figures that look like a top-journal paper

| Figure | Content | Main notebook |
|---|---|---|
| 1 | Study design: cohorts, animals per age, pipeline, gates (schematic); UMAP of integrated microglia by state | 03 |
| 2 | Primary result: animal-level coupling by age per cohort; forest plot; matched-null histogram | 05 |
| 3 | Specificity and robustness: negative-control pairs, leave-one-gene-out, pseudo-cells, covariates | 06 |
| 4 | State-resolved analysis: composition, within-state effects, DAM continuum | 07 |
| 5 | Regulators: TF activity vs coupling, pseudobulk DE volcano, genome-wide axis genes | 08 |
| 6 | Human SEA-AD: donor-level coupling vs age and Braak | 09 |
| 7 | Genetics and perturbation: MAGMA betas, perturbation z-scores, screen overlap, evidence rubric, network | 10-12 |
| 8 (optional) | Experimental validation panels | Wet lab |

*Shared figure style and a save function that writes vector files plus the source data many journals require*

```
import matplotlib as mpl, matplotlib.pyplot as plt, pandas as pd
okabe = ['#0072B2', '#E69F00', '#009E73', '#D55E00', '#CC79A7', '#56B4E9', '#F0E442', '#000000']
mpl.rcParams.update({'font.family': 'sans-serif',
                     'font.sans-serif': ['Arial', 'Helvetica', 'DejaVu Sans'],
                     'font.size': 7, 'axes.linewidth': 0.6, 'axes.spines.top': False,
                     'axes.spines.right': False, 'pdf.fonttype': 42, 'svg.fonttype': 'none',
                     'axes.prop_cycle': mpl.cycler(color=okabe)})

def save_fig(fig, name, source_df=None):
    fig.savefig(f'../results/figures/{name}.pdf', bbox_inches='tight')       # vector for journals
    fig.savefig(f'../results/figures/{name}.png', dpi=300, bbox_inches='tight')
    if source_df is not None:                                                # 'Source data' file
        source_df.to_csv(f'../results/tables/source_data_{name}.csv', index=False)
```

- Single-column width 85 mm, double 180 mm; fonts 5-8 pt in the final size; colour-blind-safe palette (Okabe-Ito).
- Show individual animals as points; never show only bars with error bars. State n as animals, not cells, in every legend.
- Assemble multi-panel figures in Inkscape (free) or Affinity; keep the raw plotting code in the repo.
- Add a Source Data spreadsheet (one sheet per figure) - Nature-family and many other journals ask for it.

# Part 8. Releasing everything publicly on GitHub (and getting a DOI)

## 8.1 Repository contents

| Path | Content |
|---|---|
| README.md | Title, abstract, workflow diagram, how to install, how to run, data sources, citation, licence, contact (template in the Appendix) |
| LICENSE | MIT (code). Data derived tables: CC BY 4.0 (state in README) |
| CITATION.cff | Machine-readable citation so GitHub shows a "Cite this repository" button (Appendix) |
| environment.yml, environment_final.yml | Pinned environments |
| notebooks/01...12 | Executed notebooks, numbered in order; clear of large outputs |
| src/mgcoupling/ | utils.py and any helper modules |
| gene_sets/modules.csv, modules_human.csv | The frozen programs with provenance |
| config/ and docs/ | Cohort decisions, data_sources.md, preregistration link, cluster_decisions_*.csv |
| results/tables, results/figures | All result tables, figure PDFs and source data |
| run_all.sh | One script that re-executes all notebooks |

*run_all.sh*

```
#!/usr/bin/env bash
# run_all.sh - re-execute the whole pipeline in order
set -e
for nb in notebooks/0*.ipynb notebooks/1*.ipynb; do
  echo "Running $nb"
  jupyter nbconvert --to notebook --execute --inplace --ExecutePreprocessor.timeout=-1 "$nb"
done
```

## 8.2 Zenodo DOI and release

1. Go to https://zenodo.org and log in with GitHub. Open your profile > GitHub, find the repository and switch it ON.
2. On GitHub: Releases > Draft a new release > tag `v1.0.0` > title "Submission version" > Publish. Zenodo archives it and mints a DOI within minutes.
3. Add the DOI badge to README and the DOI to CITATION.cff and the manuscript's Code availability statement.
4. Upload large derived data (processed h5ad files, null sets, MAGMA outputs) as a separate Zenodo "Dataset" record (limit 50 GB) and cite it. Raw data stay at GEO / CELLxGENE with accession numbers.
5. When revisions change results, publish v1.1.0 and say so in the response letter. Never silently change a released version.

## 8.3 Preprint

1. Post to bioRxiv (https://www.biorxiv.org/submit-a-manuscript), category Neuroscience, with the GitHub and Zenodo links in the abstract page and Data/Code availability. Most top journals allow preprints; check the target journal's policy.
2. Link the OSF preregistration and the preprint to your ORCID profile.

**Data and code availability statement (adapt)**

All datasets analysed are public: GEO (accessions listed in Table S1), CELLxGENE (SEA-AD, Tabula Muris Senis), GWAS Catalog. Code, frozen gene programs, executed notebooks and result tables are available at https://github.com/YOUR_USERNAME/microglia-mitophagy-lysosome-coupling (archived at Zenodo, DOI: 10.5281/zenodo.XXXXXXX). The analysis plan was preregistered on OSF (DOI: ...) before replication analyses.

# Part 9. Writing the paper and choosing the journal

## 9.1 Titles to use (pick by what your data finally show)

- **If coupling changes specifically with age and replicates:** "Age-dependent uncoupling of mitophagy and lysosomal transcriptional programs in microglia across mouse cohorts and human donors"
- **If the effect is state-linked:** "A disease-associated microglial state carries a distinct mitophagy-lysosome coordination signature across aging and Alzheimer's disease"
- **If a regulator emerges (e.g. TFEB):** "Loss of TFEB-linked coordination between lysosomal and mitochondrial quality-control programs in aged microglia"
- **If it is not specific:** "Global decline of organelle quality-control coordination in aging microglia: a multi-cohort analysis"
- **If genetics are positive:** "Genetic risk for Parkinson's and Alzheimer's disease converges on a microglial mitophagy-lysosome program that loses coordination with age"

## 9.2 Angle that editors respond to

- **Question-first, not dataset-first:** quality-control coordination between organelles as an emergent property of aging immune cells.
- **Rigour as the selling point:** animal-level replication, matched null, preregistration, human replication, genetic anchoring - say it in the abstract.
- **Disease relevance:** Parkinson's (PINK1/PRKN, GBA1, TMEM175, CTSB) and Alzheimer's (TREM2-DAM) converge on this axis; the paper is about where they meet.
- **Resource value:** frozen programs, code and cohort-harmonised tables that others can reuse (this also helps citations).
- **Candour:** one limitations paragraph that names every weakness before the reviewer does.

## 9.3 Manuscript skeleton (about 5,000-6,500 words plus Methods)

| Section | Content | Words |
|---|---|---|
| Abstract | Context, gap, design (cohorts, animals, donors), 3 key results with effect sizes, genetic and perturbation support, implication | 200 |
| Introduction | Microglia aging; PINK1/PRKN and receptor mitophagy; lysosomal biogenesis (TFEB); evidence of coupling elsewhere; the gap; aims and hypotheses | 700 |
| Results 1 | Harmonised microglia atlas across cohorts; animals per age; state map | 600 |
| Results 2 | Coupling changes with age at animal level; meta-analysis; matched null | 700 |
| Results 3 | Specificity and robustness; negative controls; state-resolved analysis | 700 |
| Results 4 | Regulators: TF activity, replicated DE, genome-wide axis genes | 700 |
| Results 5 | Human donors (SEA-AD) | 500 |
| Results 6 | Genetic anchoring; perturbation and screen concordance; evidence rubric | 800 |
| (Results 7) | Experimental validation (if done) | 600 |
| Discussion | Principal finding; relation to PD/AD biology; mechanism hypotheses; limitations; predictions | 1000 |
| Methods | Every dataset, QC rule, scoring, statistics, software versions, code and data availability, preregistration | 2000+ |

## 9.4 Journal strategy

| Evidence level | Journals to consider | Notes |
|---|---|---|
| Computational + human + genetic (no wet lab) | Aging Cell; GeroScience; Molecular Neurodegeneration; Genome Medicine; Neurobiology of Disease; Journal of Neuroinflammation; Acta Neuropathologica Communications; Alzheimer's and Dementia | Check each journal's policy on computational-only research articles and on preprints; choose by fit |
| Plus small wet-lab module | Nature Communications; Cell Reports; EMBO Journal; Aging Cell (strong chance); Molecular Neurodegeneration | Cover letter must highlight preregistration and animal-level replication |
| Plus causal rescue | Neuron; Nature Neuroscience; Nature Aging; Cell | Needs a full experimental lab collaboration |

Before submitting, read the target journal's Instructions for Authors and a few recent computational papers in it. Ask the editor with a pre-submission inquiry (one paragraph plus the abstract) if the journal offers it; that costs nothing and saves months.

## 9.5 Cover letter: five sentences that matter

- The question and why it matters for neurodegeneration.
- What is new: animal-level, multi-cohort, human-replicated and genetically anchored test of coupling (state the specific finding).
- Rigour: preregistration, matched random null, negative controls, open code with DOI.
- The result in one number with confidence interval.
- Why this journal's readership needs it.

## 9.6 Reviewer questions to answer in advance

| Likely reviewer question | Where your answer lives |
|---|---|
| Is age confounded with batch? | Multiple cohorts with several animals per age; cohort as random effect; Section 5 |
| Pseudoreplication? | Animal is the unit; mixed model as robustness; Notebook 05 |
| Is it just technical correlation or depth? | Partial Spearman with depth covariates; pseudo-cells; matched null; Notebooks 05-06 |
| Is it specific to these pathways? | Negative-control programs; Notebook 06 |
| Is it a state composition artefact? | Within-state and DAM-bin analyses; Notebook 07 |
| Do your gene lists make it circular? | Database provenance, exclusivity, DAM and identity exclusion, frozen before analysis; Notebook 04 |
| Does it hold in humans? | SEA-AD donors; Notebook 09 |
| Transcripts are not mitophagy flux | Stated limitation; proteomic concordance; wet-lab module if available |
| Why should I trust the regulator? | TF activity + replicated DE + genome-wide axis test + genetics + perturbation; rubric with robustness to weights |
| Reproducibility? | Preregistration, pinned environment, GitHub release with DOI, run_all.sh |

# Part 10. Troubleshooting and what to do with each outcome

| Symptom | Likely cause and fix |
|---|---|
| Environment creation fails or hangs | Create with `conda create -n mgcoupling python=3.10`, then install in pieces (conda packages first, then pip). Use `mamba` for speed. |
| Out of memory on SEA-AD or TMS | Load with `backed="r"`, subset to microglia first, drop unneeded layers, use `float32`. Or run in Google Colab Pro / a university cluster. |
| KeyError on a column name | Print `ad.obs.columns` and `ad.var.columns`; every dataset names things differently. Edit the mapping dictionary, never the data. |
| `score_genes` or module has too few genes | Check symbol case (`Gba` vs `Gba1`), species, and that gene symbols (not Ensembl IDs) are the var index. |
| Fewer than 3 animals per age group in a cohort | Move the cohort to the supplement or find another; do not report p-values from n=2. |
| `combine_effects` error about method | Use `method_re="chi2"` (DerSimonian-Laird) as in utils.py. |
| decoupler function not found | Version 2 renamed the API; install `decoupler<2` as pinned. |
| MAGMA reports no genes mapped | SNP IDs must be rsIDs matching the reference `.bim`; check that the annotation step produced a non-empty `.genes.annot`. |
| STRING API returns empty | Check symbols are mouse (species 10090), lower the score to 400, retry later. |
| mixedlm does not converge | Subsample cells per animal, drop the random slope (`re_formula=None`), standardise scores. |

## What each possible outcome becomes

| Outcome | Paper you write |
|---|---|
| Specific, replicated change in coupling; human support; genetic signal | Primary-research paper (titles 1 or 5); aim at the upper tier you can support |
| Change specific but only in a state (e.g. DAM) | State-linked coordination paper (title 2) |
| Change not specific to the pair (controls move too) | Global coordination decline (title 4); still publishable and more honest |
| No change with age after proper controls | Short negative-result report plus resource (methods, programs, code); strengthens the field by killing a tempting artefact |
| Mouse effect not replicated in humans | Species-specific finding; discuss nuclei limitations; propose follow-up |

**Final reality check**

Top journals select on importance and robustness together. This design maximises robustness and gives you the evidence structure reviewers expect. Importance depends on the result you actually get, and no guide can promise that result. If the result is clear, replicated and genetically supported, you have a credible submission. If it is not, the same pipeline gives you an honest and still useful paper.

# Appendix A. src/mgcoupling/utils.py (complete, tested listing)

This file is identical to the one in the bundle. Pure functions: scores, partial Spearman, animal-level coupling, permutation test, meta-analysis, matched random sets, loaders.

*utils.py [» marks a printed continuation; the notebook holds the line unwrapped]*

```
import numpy as np, pandas as pd, scanpy as sc
from scipy import sparse, stats
from scipy.io import mmread
import statsmodels.formula.api as smf
from statsmodels.stats.meta_analysis import combine_effects


def to_dense(X):
    return X.toarray() if sparse.issparse(X) else np.asarray(X)


def mad_outlier(adata, metric, nmads=5):
    m = adata.obs[metric].values
    med, mad = np.median(m), stats.median_abs_deviation(m)
    return (m < med - nmads * mad) | (m > med + nmads * mad)


def module_scores(adata, modules, min_genes=5):
    # mean of gene-wise z-scores over a module (adata.X must be log-normalised, not scaled)
    out = {}
    for name, genes in modules.items():
        idx = [adata.var_names.get_loc(g) for g in genes if g in adata.var_names]
        if len(idx) < min_genes:
            print('WARNING', name, 'only', len(idx), 'genes found')
        sub = to_dense(adata.X[:, idx])
        mu, sd = sub.mean(0), sub.std(0)
        keep = sd > 0
        out[name] = ((sub[:, keep] - mu[keep]) / sd[keep]).mean(1)
    return pd.DataFrame(out, index=adata.obs_names)


def partial_spearman(x, y, covars):
    rx, ry = stats.rankdata(x), stats.rankdata(y)
    C = np.column_stack([np.ones(len(x))] + [stats.rankdata(c) for c in covars])
    rx_res = rx - C @ np.linalg.lstsq(C, rx, rcond=None)[0]
    ry_res = ry - C @ np.linalg.lstsq(C, ry, rcond=None)[0]
    return np.corrcoef(rx_res, ry_res)[0, 1]


def animal_coupling(df, a, b, animal='animal_id', covars=('log_counts', 'n_genes'), min_cells=100):
    rows = []
    for k, g in df.groupby(animal, observed=True):
        if len(g) < min_cells:
            continue
        r = partial_spearman(g[a].values, g[b].values, [g[c].values for c in covars])
        rows.append({animal: k, 'n_cells': len(g), 'rho': r, 'z': np.arctanh(r),
```

```
                    'var_z': 1.0 / (len(g) - 3 - len(covars))})
    return pd.DataFrame(rows)


def perm_diff(a, b, n_perm=20000, seed=0):
    rng = np.random.default_rng(seed)
    a, b = np.asarray(a, float), np.asarray(b, float)
    obs = a.mean() - b.mean()
    pool, n, cnt = np.concatenate([a, b]), len(a), 0
    for _ in range(n_perm):
        rng.shuffle(pool)
        cnt += abs(pool[:n].mean() - pool[n:].mean()) >= abs(obs) - 1e-12
    return obs, (cnt + 1) / (n_perm + 1)


def group_effect(ac, group_col='age_group', hi='old', lo='young'):
    o = ac.loc[ac[group_col] == hi, 'z'].values
    y = ac.loc[ac[group_col] == lo, 'z'].values
    diff = o.mean() - y.mean()
    se = np.sqrt(o.var(ddof=1) / len(o) + y.var(ddof=1) / len(y))
    return diff, se, len(o), len(y)


def meta_random(effects, ses, labels):
    # DerSimonian-Laird random effects ('chi2' in statsmodels)
    return combine_effects(np.array(effects), np.array(ses) ** 2, method_re='chi2', row_names=labels)


def matched_random_sets(adata, genes, n_sets=200, n_bins=20, seed=0, exclude=()):
    rng = np.random.default_rng(seed)
    me = pd.Series(np.asarray(adata.X.mean(axis=0)).ravel(), index=adata.var_names)
    bins = pd.qcut(me.rank(method='first'), n_bins, labels=False)
    present = [g for g in genes if g in me.index]
    ban = set(present) | set(exclude)
    members = {b: [g for g in bins.index[bins == b] if g not in ban] for b in range(n_bins)}
    return [[str(rng.choice(members[bins[g]])) for g in present] for _ in range(n_sets)]


def null_delta(adata, obs_df, sets_a, sets_b, groups, covars=('log_counts', 'n_genes'), min_cells=100):
    # obs_df has animal_id, log_counts, n_genes ; groups has animal_id, age_group
    deltas = []
    for ga, gb in zip(sets_a, sets_b):
        d = obs_df.copy()
        d['a'] = module_scores(adata, {'a': ga}, min_genes=1).iloc[:, 0].values
        d['b'] = module_scores(adata, {'b': gb}, min_genes=1).iloc[:, 0].values
        ac = animal_coupling(d, 'a', 'b', covars=covars, min_cells=min_cells)
```

```
        ac = ac.merge(groups, on='animal_id')
        deltas.append(group_effect(ac)[0])
    return np.array(deltas)


def mixed_interaction(d, y='lyso', x='mito', grp='old', animal='animal_id'):
    m = smf.mixedlm(y + ' ~ ' + x + ' * ' + grp, d, groups=d[animal], re_formula='~' + x)
    return m.fit(method='lbfgs')


def bh_fdr(p):
    p = np.asarray(p, float); n = len(p); o = np.argsort(p)
    q = np.empty(n); prev = 1.0
    for i in range(n - 1, -1, -1):
        prev = min(prev, p[o[i]] * n / (i + 1)); q[o[i]] = prev
    return q


_WT_CACHE = {}


def load_wt(c, base='../data/processed', cache=True):
    # wild-type, young or old animals only; cached in memory so loops are fast
    key = (base, c)
    if cache and key in _WT_CACHE:
        return _WT_CACHE[key]
    a = sc.read_h5ad(f'{base}/{c}_microglia.h5ad')
    keep = (a.obs['condition'].astype(str) == 'WT') & a.obs['age_group'].astype(str).isin(['young',
    » 'old'])
    a = a[keep.values].copy()
    if cache:
        _WT_CACHE[key] = a
    return a


def cohort_effect(a, mito_genes, lyso_genes, covars=('log_counts', 'n_genes'),
                  min_cells=100, state=None):
    sco = module_scores(a, {'mito': mito_genes, 'lyso': lyso_genes}, min_genes=1)
    cols = list(dict.fromkeys(['animal_id', 'age_group', 'sex', 'state'] + list(covars)))
    d = a.obs[cols].join(sco)
    if state is not None:
        d = d[d['state'] == state]
    ac = animal_coupling(d, 'mito', 'lyso', covars=covars, min_cells=min_cells)
    info = d[['animal_id', 'age_group', 'sex']].drop_duplicates('animal_id')
    ac = ac.merge(info, on='animal_id')
    n_o = (ac.age_group == 'old').sum(); n_y = (ac.age_group == 'young').sum()
```

```
    if n_o < 2 or n_y < 2:
        return ac, None
    return ac, group_effect(ac)


def pooled_effect(cohorts, mito_genes, lyso_genes, covars=('log_counts', 'n_genes'), state=None,
                  min_cells=100):
    out = []
    for c in cohorts:
        a = load_wt(c)
        ac, eff = cohort_effect(a, mito_genes, lyso_genes, covars=covars, state=state,
                                min_cells=min_cells)
        if eff is not None:
            out.append((c, eff[0], eff[1]))
    e = np.array([o[1] for o in out]); s = np.array([o[2] for o in out])
    w = 1 / s ** 2
    return float((w * e).sum() / w.sum()), float(np.sqrt(1 / w.sum())), out


# ---------------------------------------------------------------- loaders (used by Notebooks 01 and 09)
def looks_like_counts(X, n=2000):
    d = X.data[:n] if sparse.issparse(X) else np.asarray(X).ravel()[:n]
    return bool(np.all(np.abs(d - np.round(d)) < 1e-6))


def attach_meta(ad, meta):
    ad.obs = ad.obs.join(meta.set_index('sample_id'), on='sample_id', rsuffix='_m')
    print('cells without metadata:', int(ad.obs['animal_id'].isna().sum()))
    return ad[ad.obs['animal_id'].notna().values].copy()


def load_10x_mtx(d, prefix, sample_id):
    genes = pd.read_csv(f'{d}/{prefix}_genes.tsv.gz', header=None, sep='\t')
    bcs = pd.read_csv(f'{d}/{prefix}_barcodes.tsv.gz', header=None, sep='\t')
    ad = sc.AnnData(X=mmread(f'{d}/{prefix}_matrix.mtx.gz').T.tocsr())
    ad.var_names = genes[1].astype(str).values
    ad.obs_names = [f'{sample_id}_{b}' for b in bcs[0]]
    ad.var_names_make_unique()
    ad.obs['sample_id'] = sample_id
    return ad


def load_dge_txt(path, sample_id, genes_in_rows=True):
    # tab-separated digital expression matrix; CHECK orientation with zcat | head first
    df = pd.read_csv(path, sep='\t', index_col=0)
    if not genes_in_rows:

        df = df.T
    ad = sc.AnnData(sparse.csr_matrix(df.T.values.astype(np.float32)))
    ad.var_names = df.index.astype(str)
    ad.obs_names = [f'{sample_id}_{c}' for c in df.columns]
    ad.var_names_make_unique()
    ad.obs['sample_id'] = sample_id
    return ad


def load_cellxgene(path):
    # CELLxGENE .h5ad: raw counts in .raw.X, gene symbols in var['feature_name']
    ad = sc.read_h5ad(path)
    if ad.raw is not None:
        var = ad.raw.var if 'feature_name' in ad.raw.var else ad.var
        ad = sc.AnnData(X=ad.raw.X, obs=ad.obs.copy(), var=var.copy())
    if 'feature_name' in ad.var:
        ad.var_names = ad.var['feature_name'].astype(str).values
    ad.var_names_make_unique()
    print('counts-like:', looks_like_counts(ad.X))
    return ad
```

# Appendix B. src/mgcoupling/sim.py (synthetic generator)

Used only by Notebooks 00 and 13. Reading it shows exactly what was planted: latent mitophagy and lysosome activities, a planted within-animal correlation that is lower in old animals, a regulator set that tracks the coupling axis, depth variation, state shifts, contaminants, low-quality cells and doublets.

*sim.py [» marks a printed continuation; the notebook holds the line unwrapped]*

```
"""Synthetic generator used for the smoke test and failure-mode gallery (Notebook 00 and 13).
Latent mitophagy and lysosome programs with a planted, age-dependent within-animal correlation."""
import numpy as np, pandas as pd, anndata as an
from scipy import sparse


identity = ['Hexb','Cx3cr1','P2ry12','Tmem119','Csf1r','Sall1','Fcrls','Selplg','Siglech',
            'Aif1','Tyrobp','C1qa','C1qb']
mitophagy = ['Pink1','Prkn','Bnip3','Bnip3l','Fundc1','Optn','Calcoco2','Tbk1','Usp30','Mul1',
             'Fkbp8','Bcl2l13','Phb2','Stx17','Ulk1','Park7','Vps13c','Mfn2']
lysosome = ['Mcoln1','Ctsd','Ctsf','Gla','Gba1','Hexa','Neu1','Tpp1','Npc1','Npc2','Lamp1','Lamp2',
            'Atp6v1h','Atp6v0e1','Atp6v1c1','Sgsh','Gns','Galc','Naglu','Asah1','Lipa','Ppt1',
            'Cln3','Cln5','Gusb','Idua','Ids','Man2b1','Fuca1','Ctsa','Ctsz','Smpd1','Gaa']
oxphos = ['Ndufa1','Ndufa9','Ndufb8','Ndufs1','Sdha','Sdhb','Uqcrc1','Uqcrc2','Cox4i1','Cox5a',
          'Cox6a1','Atp5f1a','Atp5f1b','Ndufb5','Ndufs3','Uqcrb','Cox7c','Atp5mc1']
autoph = ['Atg5','Atg7','Atg12','Atg16l1','Becn1','Map1lc3b','Gabarap','Gabarapl1','Atg3','Atg4b','Ulk1']
proteasome = (['Psma%d' % i for i in range(1, 8)] + ['Psmb%d' % i for i in range(1, 8)] +
              ['Psmc%d' % i for i in range(1, 7)] + ['Psmd%d' % i for i in (1, 2, 3, 4, 6, 7, 8, 11,
              » 12, 13, 14)])
ribosome = ['Rpl%d' % i for i in range(3, 33)] + ['Rps%d' % i for i in range(3, 18)]
dam = ['Trem2','Apoe','Lpl','Cst7','Clec7a','Itgax','Spp1','Axl','Cd9','Gpnmb','Ctsd','Lamp1','Ctsz','Cd
» 68']
ifn = ['Ifit1','Ifit2','Ifit3','Irf7','Isg15','Usp18','Oasl2']
prolif = ['Mki67','Top2a','Stmn1']
antig = ['H2-Aa','H2-Ab1','Cd74','H2-Eb1']
acute = ['Il1b','Tnf','Ccl3','Ccl4','Nfkbia']
neuron = ['Snap25','Rbfox3','Syt1']
bam = ['Mrc1','Lyve1','Pf4','Cd163']
mtg = ['mt-Co1','mt-Nd1','mt-Atp6','mt-Cytb','mt-Co2']
regulators = ['Tfeb', 'Tfe3', 'Mitf', 'Ppargc1a']          # planted: track the coupling axis in young
» animals
named = list(dict.fromkeys(g for lst in [identity, mitophagy, lysosome, oxphos, autoph, proteasome,
                           ribosome, dam, ifn, prolif, antig, acute, neuron, bam, mtg,
                           » regulators] for g in lst))
genes = named + ['Gene%04d' % i for i in range(3200 - len(named))]
G = len(genes); gi = {g: i for i, g in enumerate(genes)}


rng0 = np.random.default_rng(1)
base = rng0.lognormal(np.log(2e-4), 1.2, G)

def setbase(lst, mu, sd=0.4):
    for g in lst:
        base[gi[g]] = rng0.lognormal(np.log(mu), sd)

setbase(identity, 1.5e-3); setbase(mitophagy, 4e-5); setbase(lysosome, 2e-4)
```

```
setbase(oxphos, 5e-4); setbase(autoph, 2e-4); setbase(proteasome, 4e-4); setbase(ribosome, 1.2e-3)
setbase(dam, 1e-4); setbase(ifn, 6e-5); setbase(prolif, 3e-5); setbase(antig, 4e-4); setbase(acute, 1e-4)
setbase(neuron, 3e-6); setbase(bam, 3e-6); setbase(mtg, 2.5e-3, 0.2); setbase(regulators, 3e-4)
base = base / base.sum()
idx = lambda lst: np.array([gi[g] for g in lst])


def simulate_animal(rng, n, rho, depth_mult, state_p, reg=0.9):
    states = rng.choice(['homeostatic', 'DAM', 'IFN', 'BAM', 'neuron'], size=n, p=state_p)
    z = rng.multivariate_normal([0, 0], [[1, rho], [rho, 1]], size=n)
    m, l = z[:, 0].copy(), z[:, 1].copy()                  # latent mitophagy / lysosome activity
    gshared, h, o, a = (rng.normal(size=n) for _ in range(4))
    depth = depth_mult * rng.lognormal(np.log(3500), 0.35, n)
    l = l + 1.0 * (states == 'DAM'); m = m + 0.3 * (states == 'DAM')
    L = np.zeros((n, G))                                   # log-fold changes on each gene's mean
    L[:, idx(mitophagy)] += 0.55 * m[:, None];  L[:, idx(lysosome)] += 0.55 * l[:, None]
    L[:, idx(proteasome)] += 0.45 * h[:, None]; L[:, idx(ribosome)] += 0.35 * h[:, None]
    L[:, idx(oxphos)] += 0.45 * o[:, None];     L[:, idx(autoph)] += 0.45 * a[:, None]
    for lst in (mitophagy, lysosome, proteasome, oxphos, autoph):
        L[:, idx(lst)] += 0.18 * gshared[:, None]           # shared factor, NOT age dependent
    L[:, idx(regulators)] += reg * ((m + l) / 2)[:,
    » None]   # regulator tracks the axis; weaker when reg is small
    for g_ in idx(dam): L[states == 'DAM', g_] += 2.2
    for g_ in idx(ifn): L[states == 'IFN', g_] += 3.0
    for g_ in idx(['Hexb','Cx3cr1','P2ry12','Tmem119','Selplg','Siglech','Sall1','Fcrls']):
        L[states == 'DAM', g_] -= 1.2
    for g_ in idx(bam): L[states == 'BAM', g_] += 7.5
    for g_ in idx(neuron): L[states == 'neuron', g_] += 8.0
    for g_ in idx(['P2ry12', 'Tmem119', 'Siglech']): L[states == 'BAM', g_] -= 2.0
    lowq = rng.random(n) < 0.04                            # damaged cells: low depth, high mt fraction
    depth = np.where(lowq, depth * 0.35, depth)
    L[np.ix_(lowq, idx(mtg))] += 2.5
    mu = depth[:, None] * base[None, :] * np.exp(L)
    X = rng.poisson(mu * rng.gamma(8, 1 / 8, size=mu.shape)).astype(np.float32)
    dbl = rng.random(n) < 0.02                             # 2% doublets
    for i in np.where(dbl)[0]:
        X[i] += X[rng.integers(n)]
    obs = pd.DataFrame({'state_true': states, 'doublet_true': dbl, 'lowq_true': lowq,
                        'latent_m': m, 'latent_l': l})
    return sparse.csr_matrix(X), obs


def simulate_cohort(name, design, rho_young, rho_old, n_cells, seed, depth_by_group=None,
» reg_by_group=None,
                    state_p_by_group=None):
    rng = np.random.default_rng(seed); parts, meta = [], []

    sp_by = {'young': [0.90, 0.05, 0.03, 0.015, 0.005], 'mid': [0.82, 0.09, 0.06, 0.02, 0.01],
             'old': [0.70, 0.15, 0.10, 0.03, 0.02]}
    sp_by.update(state_p_by_group or {})
    depth_by_group = depth_by_group or {}
    reg_by_group = reg_by_group or {'young': 0.9, 'mid': 0.55, 'old': 0.2}
    for k, (age_m, group, cond, sex, pooled) in enumerate(design):
        rho = {'young': rho_young, 'old': rho_old, 'mid': (rho_young + rho_old) / 2}[group]
        rho = float(np.clip(rho - (0.12 if cond != 'WT' else 0) + rng.normal(0, 0.05), 0.02, 0.9))
        aid = f'{name}_{k + 1:02d}'
        reg = reg_by_group[group]
        X, obs = simulate_animal(rng, n_cells, rho, rng.lognormal(0, 0.2) * depth_by_group.get(group,
        » 1.0),
                                 sp_by[group], reg=reg)
        for key, val in dict(sample_id='S_' + aid, animal_id=aid, dataset=name, age_months=age_m,
                             age_group=group, condition=cond, sex=sex, region='whole_brain',
                             pooled_n=pooled, true_rho_latent=rho).items():
            obs[key] = val
        obs.index = [f'{aid}_c{i:04d}' for i in range(n_cells)]
        parts.append(an.AnnData(X=X, obs=obs, var=pd.DataFrame(index=genes)))
        meta.append({k_: obs[k_].iloc[0] for k_ in ['dataset', 'sample_id', 'animal_id', 'age_months',
                     'age_group', 'sex', 'region', 'condition', 'pooled_n']})
    ad = an.concat(parts, join='outer', index_unique=None); ad.var_names = genes
    return ad, pd.DataFrame(meta)
```

# Appendix C. environment.yml and repository files

*environment.yml (also in the bundle)*

```
name: mgcoupling
channels:
  - conda-forge
  - bioconda
dependencies:
  - python=3.12
  - numpy
  - pandas
  - scipy
  - statsmodels
  - matplotlib
  - seaborn
  - scikit-learn
  - scanpy>=1.10
  - anndata
  - python-igraph
  - leidenalg
  - scikit-misc
  - jupyterlab
  - requests
  - pip
  - pip:
      - harmonypy
      - gseapy
      - scrublet
      - "pydeseq2>=0.5"
      - pybiomart
      - GEOparse
      - "decoupler<2"
      - scvi-tools   # optional (Notebook 03 Cell 3)
```

**Version conflicts to know about**

`decoupler<2` (the 1.x API used in Notebook 08) pins pandas below 3 and an older numba on some platforms. If pip reports conflicts, install decoupler last, or keep Notebook 08 Cells 1-2 in a second environment. The rest of the project was tested with pandas 3.0 and numpy 2.4.

# Appendix D. Test report: what was actually run

Every cell marked "run" in Part 4 was executed in sequence on the synthetic benchmark and the outputs shown are those results. Tested with: Python 3.12.3; scanpy 1.12.4; anndata 0.13.4; pandas 3.0.2; numpy 2.4.4; scipy 1.17.1; statsmodels 0.15.0; pydeseq2 0.5.4; gseapy 1.3.1; scrublet 0.2.3; harmonypy 2.1.0; leidenalg 0.12.0; python-igraph 1.0.0; scikit-misc 0.5.3; numba 0.68. The `decoupler` 1.8 call signature of Notebook 08 Cell 1-2 was tested on a toy network (the CollecTRI download needs internet); MAGMA, GEO, CELLxGENE, STRING, Enrichr and scVI steps could not be executed here and are shown as format examples.

| Step | Status in the benchmark | Notes |
|---|---|---|
| N00, N02-N09, N11-N13 cells marked run | Executed, no errors | Outputs shown are real |
| N01 (GEO/CELLxGENE loading) | Not executed (needs downloads) | Loader functions in utils.py were imported and the CELLxGENE-style loader was exercised on the synthetic donor file in N09 |
| N03 scVI cell | Not executed (needs PyTorch) | Harmony path executed |
| N04 gseapy library listing | Not executed (needs internet) | Seeds and filters executed |
| N08 CollecTRI / TF activity | API tested on a toy network only | Real run needs internet |
| N10 MAGMA | Not executed | Layouts shown as format examples; the locus overlay cell was executed |
| N12 STRING and Enrichr | Not executed (needs internet) | Rubric and robustness cells executed |

Bugs found and fixed during testing (they are why you can trust the shipped code, and a reminder of what to look for on your own data): (1) Harmony's scanpy wrapper broke with harmonypy 2.x; (2) an automatic cluster-drop rule based on pan-myeloid scores silently deleted the DAM cluster; (3) pooling pseudo-cells across states flipped the sign of the effect; (4) a categorical Braak column made statsmodels fit one coefficient per stage; (5) a `%` in a format string crashed a summary print; (6) brace expansion in `mkdir -p` does nothing in plain `sh`.

# Appendix E. Templates and final checklist

## A. README.md template

```
# Microglia mitophagy-lysosome coupling across aging

Multi-cohort, animal-level analysis of coordination between mitophagy and lysosomal
transcriptional programs in mouse and human microglia.

- Preprint: <bioRxiv link>      - Preregistration: <OSF DOI>      - Archive: <Zenodo DOI>

## Quick start
conda env create -f environment.yml
conda activate mgcoupling
# download data: see docs/data_sources.md, then
bash run_all.sh

## Data
All input data are public (GEO, CELLxGENE, GWAS Catalog). Accessions: docs/data_sources.md.

## Structure
notebooks/ (01-12, run in order) | src/mgcoupling/ | gene_sets/ | results/ | docs/

## Citation
See CITATION.cff.  Licence: MIT (code), CC BY 4.0 (derived tables).
```

## B. CITATION.cff template

```
cff-version: 1.2.0
message: "If you use this software or data, please cite it as below."
title: "Microglia mitophagy-lysosome coupling across aging"
authors:
  - family-names: "YourFamilyName"
    given-names: "YourGivenNames"
    orcid: "https://orcid.org/0000-0000-0000-0000"
version: 1.0.0
doi: 10.5281/zenodo.XXXXXXX
date-released: 2026-12-01
repository-code: "https://github.com/YOUR_USERNAME/microglia-mitophagy-lysosome-coupling"
license: MIT
```

## C. Final pre-submission checklist

| Item | Done |
|---|---|
| Novelty check documented; related papers cited fairly | |
| Preregistration submitted before replication analyses | |
| Every cohort has >= 3 animals per age group, or is supplementary | |
| All statistics use animal/donor as unit; cell-level tests labelled as robustness | |
| Matched random null and negative-control programs reported | |
| Gene programs frozen, with provenance, overlap and DAM exclusion documented | |
| Human replication and sign comparison reported | |
| MAGMA with APOE/MHC removal and conditioning; negative controls included | |
| Perturbation predictions written before testing; failures reported | |
| Limitations paragraph names n, transcript-vs-flux, nuclei vs cells, cohort differences | |
| All figures: individual animals shown, n = animals, source data provided | |
| GitHub release, Zenodo DOI, environment pinned, run_all.sh tested on a clean machine | |
| Data and code availability statements complete; all datasets cited | |
| Preprint posted; cover letter and 3-5 suggested reviewers ready | |